

Tugas Besar 2 IF3270 – Pembelajaran Mesin

Convolutional Neural Network dan Recurrent Neural Network



16 Mei 2026 — Dipersiapkan oleh kelompok 69 K02, “maafin gw ya ghan”:

Mochammad Fariz Rifqi Rizqulloh

13523069@std.stei.itb.ac.id

Muhammad Jibril Ibrahim

13523085@std.stei.itb.ac.id

Nayaka Ghana Subrata

13523090@std.stei.itb.ac.id

Daftar Isi |

Daftar Isi 	2
01 — Deskripsi Persoalan	5
02 — Penjelasan Implementasi	6
└ 2.1 Implementasi CNN	6
2.1.1 Struktur Implementasi dan Deskripsi Kelas	6
2.1.2 Implementasi Utility Functions	8
2.1.3 Implementasi Forward Propagation from scratch	9
2.1.4 Implementasi Pelatihan Model	12
2.1.5 Implementasi Batch Inference	14
2.1.6 Implementasi Feature Map Extraction	14
2.1.7 Implementasi Grad-CAM	15
└ 2.2 Implementasi RNN & LSTM	16
2.2.1 Implementasi Feature Extraction	16
2.2.2 Implementasi Preprocessing Caption	18
2.2.2.1 Pembentukan Split Dataset	19
2.2.2.2 Text Cleaning	19
2.2.2.3 Padding dan Truncation	21
2.2.3 Implementasi Arsitektur Decoder	22
2.2.3.1 Komponen Utama Decoder	23
2.2.3.2 Variasi Recurrent Layer: RNN dan LSTM	24
2.2.3.3 Pre-Inject Decoder	25
2.2.3.4 Init-Inject Decoder	26
2.2.3.5 Multi-Layer Recurrent Decoder	27
2.2.3.6 Output Layer dan Distribusi Vocabulary	28
2.2.4 Implementasi Pelatihan Decoder	29
2.2.5 Implementasi Forward Propagation from scratch	30
2.2.5.2 Forward Propagation pada RNN	33
2.2.5.3 Forward Propagation pada LSTM	34
2.2.5.4 Proyeksi ke Vocabulary	35
2.2.5.5 Greedy Decoding	35
2.2.5.6 Validasi terhadap Model Keras	36
2.2.6 Implementasi Beam Search	37

2.2.7 Implementasi Arsitektur Init-Inject	39
2.2.6.1 Arsitektur Init-Inject	39
2.2.6.2 Image Feature sebagai Initial State	40
2.2.6.3 Init-Inject pada LSTM	41
2.2.6.4 Multi-Layer Init-Inject	42
2.2.6.5 Output Layer	42
2.2.8 Implementasi Backward Propagation from scratch	43
2.2.8.1 Alur Backward Propagation	43
2.2.8.2 Backward Dense Output Layer	44
2.2.8.3 Backward Recurrent Layer	45
2.2.8.4 Backward Image Projection dan Embedding	46
2.2.8.5 Update Bobot From Scratch	47
03 — Hasil Pengujian	48
└─ 3.1 Hasil Pengujian CNN	48
3.1.1 Perbandingan shared vs non-shared parameter	51
3.1.2 Pengaruh jumlah layer konvolusi	53
3.1.3 Pengaruh banyak filter per layer konvolusi	54
3.1.4 Pengaruh ukuran filter per layer konvolusi	55
3.1.5 Pengaruh jenis pooling layer	56
3.1.6 Perbandingan Keras vs from scratch	58
└─ 3.2 Hasil Pengujian Simple RNN dan LSTM	59
3.2.1 Perbandingan jumlah layer & hidden state	59
3.2.2 Perbandingan RNN vs LSTM	60
3.2.3 Perbandingan Keras vs from scratch	61
3.2.4 Pengaruh panjang maksimum caption terhadap BLEU-4	62
3.2.5 Perbandingan 12 variasi konfigurasi decoder Pre-Inject	64
3.2.6 Perbandingan Pre-Inject vs Init-Inject	65
3.2.7 Perbandingan Greedy vs Beam Search	66
04 — Kesimpulan dan Saran	68
└─ 4.1 Kesimpulan	68
└─ 4.2 Saran	68
└─ 4.3 Komentar	69
Pembagian Tugas 	70
Referensi 	73
Lampiran 	75

Form Penggunaan AI	75
scratch_forward.py	77
scratch_backprop.py	84
extract_inception_features.py	84
preprocess_captions.py	87
build_teacher_forcing.py	92
train_decoders.py	96
decoder.py	104
src/cnn/nn/layers.py	112
src/cnn/nn/keras_layers.py	122
src/cnn/core/ops.py	126
src/cnn/data/image.py	130
src/cnn/data/dataset.py	133
src/common/image_io.py	141
src/cnn/training/serialization.py	144
src/cnn/training/train.py	149
src/cnn/training/experiments.py	156
src/cnn/visualization.py	165

01 — Deskripsi Persoalan

Tugas besar ini berfokus pada implementasi dan analisis arsitektur deep learning berbasis Convolutional Neural Network (CNN) dan Recurrent Neural Network (RNN), khususnya Simple RNN dan Long Short-Term Memory (LSTM). Persoalan utama yang dibahas adalah bagaimana model CNN dapat digunakan untuk melakukan klasifikasi citra, serta bagaimana model RNN dan LSTM dapat digunakan sebagai decoder dalam sistem image captioning. Selain menggunakan Keras untuk pelatihan model, tugas ini juga meminta mahasiswa untuk mengimplementasikan forward propagation from scratch.

Pada bagian CNN, persoalan yang diselesaikan adalah klasifikasi gambar menggunakan [dataset Intel](#) Image Classification yang terdiri dari sekitar 25.000 gambar dalam enam kategori, yaitu buildings, forest, glacier, mountain, sea, dan street. Model CNN dilatih menggunakan beberapa variasi arsitektur, seperti jumlah layer konvolusi, jumlah filter, ukuran filter, dan jenis pooling. Selain itu, dilakukan perbandingan antara implementasi CNN dengan shared parameter menggunakan Conv2D dan non-shared parameter menggunakan LocallyConnected2D untuk menganalisis pengaruh parameter sharing terhadap performa dan efisiensi model.

Pada bagian RNN dan LSTM, persoalan yang diselesaikan adalah pembuatan caption otomatis untuk gambar menggunakan [dataset Flickr8k](#). Dataset ini terdiri dari 8.092 gambar, dengan masing-masing gambar memiliki lima caption. Sistem image captioning dibangun menggunakan arsitektur encoder-decoder, yaitu CNN sebagai encoder untuk mengekstraksi fitur visual dari gambar, dan Simple RNN atau LSTM sebagai decoder untuk menghasilkan urutan kata berupa caption.

02 — Penjelasan Implementasi

└ 2.1 Implementasi CNN

2.1.1 Struktur Implementasi dan Deskripsi Kelas

Implementasi CNN pada *repository* ini dipisahkan ke beberapa modul berikut.

Modul	Fungsi Utama
<code>src/cnn/data/</code>	Memuat gambar, membentuk <i>batch</i> , dan menyusun <i>split</i> dataset
<code>src/cnn/core/</code>	Menyediakan abstraksi model dan operasi tensor dasar
<code>src/cnn/nn/</code>	Mengimplementasikan layer CNN from <i>scratch</i> , aktivasi, metrik, dan <i>adapter layer</i> Keras
<code>src/cnn/training/</code>	Menangani pembangunan model Keras, <i>training</i> , eksperimen, evaluasi, dan konversi ke model <i>scratch</i>
<code>src/cnn/visualization.py</code>	Menangani bonus CNN seperti <i>feature map extraction</i> dan Grad-CAM

Fondasi implementasi from scratch dibangun di atas tiga komponen utama yaitu `Layer`, `WeightedLayer`, dan `Sequential`. `Layer` adalah kelas abstrak dasar yang mendefinisikan bahwa setiap *layer* harus memiliki method `forward()`. `WeightedLayer` memperluas `Layer` untuk *layer* yang memiliki bobot dan menyediakan method `get_weights()` serta `set_weights()`. `Sequential` menyimpan daftar *layer* dan mengeksekusi `forward pass` secara berurutan dari *layer* pertama hingga *layer* terakhir.

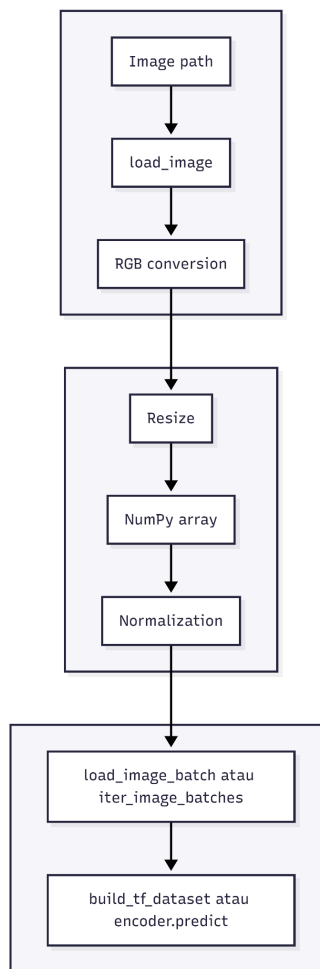
Layer-layer CNN from scratch didefinisikan pada [src/cnn/nn/layers.py](#). Ringkasannya ada pada tabel berikut.

Kelas	Main Attribute	Main Method	Peran
ActivationLayer	activation, activation_fn	forward()	Menerapkan fungsi aktivasi sebagai layer eksplisit
Conv2D	filters, kernel_size, strides, padding, activation, kernel, bias	set_weights(), get_weights(), forward()	Konvolusi 2D dengan <i>shared parameters</i>
LocallyConnected2D	filters, kernel_size, strides, padding, activation, kernel, bias	set_weights(), get_weights(), forward()	Konvolusi lokal tanpa <i>parameter sharing</i>
MaxPooling2D / AveragePooling2D	pool_size, strides, padding	forward()	<i>Pooling</i> lokal dengan operator maksimum atau rata-rata
GlobalAveragePooling2D / GlobalMaxPooling2D	-	forward()	Mereduksi seluruh dimensi spasial menjadi satu nilai per kanal
Flatten	-	forward()	Mengubah tensor spasial menjadi vektor 1D
Dense	units, activation, kernel, bias	set_weights(), get_weights(), forward()	Layer <i>fully connected</i> dan klasifikasi akhir

Selain *layer scratch*, terdapat `KerasLocallyConnected2D` pada [src/cnn/nn/keras_layers.py](https://github.com/fchollet/keras/blob/master/src/cnn/nn/keras_layers.py). Kelas ini digunakan saat melatih model *non-shared* di Keras. Pada level operasi tensor, implementasi menggunakan helper di [src/cnn/core/ops.py](https://github.com/fchollet/keras/blob/master/src/cnn/core/ops.py). Fungsi yang dipakai antara lain `normalize_tuple()`, `ensure_4d_batch()`, `restore_batch_dim()`, `compute_conv_output_length()`, `compute_padding()`, dan `extract_image_patches()`.

2.1.2 Implementasi *Utility Functions*

Implementasi *utility functions* untuk CNN terutama berada pada [src/cnn/data/image.py](#), [src/cnn/data/dataset.py](#), dan [src/common/image_io.py](#). Fungsi-fungsi ini mengubah gambar mentah pada dataset Intel menjadi tensor yang digunakan saat training Keras maupun inferensi *scratch*. *Pipeline utility* CNN terdiri dari beberapa tahap.



Gambar 2.1.2.1 Tahapan *pipeline utility* CNN

Sumber : arsip penulis

Tahap paling dasar adalah *image loader*. Fungsi `load_image()` menerima *file path* gambar, ukuran target, opsi normalisasi, dan *dtype*. Gambar dibaca

menggunakan `PIL.Image.open`, dikonversi ke RGB, di-resize dengan interpolasi bilinear, lalu diubah menjadi `numpy` array. Jika nilai `normalize=True`, nilai piksel dibagi 255 sehingga berada pada rentang `[0, 1]`. *Output* fungsi ini berbentuk `(height, width, 3)` dengan tipe `float32`.

Di atas fungsi tersebut, tersedia `load_image_batch()` dan `iter_image_batches()`. `load_image_batch()` menerima sekumpulan *path* gambar lalu menggabungkannya menjadi satu tensor `numpy` berukuran `(N, H, W, C)`. `iter_image_batches()` menghasilkan *batch* secara bertahap melalui *generator*.

Utility lain yang disiapkan adalah `extract_features()` dan `extract_and_save_features()`. Fungsi ini menerima daftar gambar, *encoder* Keras, ukuran target, dan `batch_size`, lalu menjalankan `encoder.predict()` per *batch*. Jika diperlukan, argumen `preprocess_fn` digunakan untuk memberi *preprocessing* tambahan sebelum gambar masuk ke *encoder*.

Untuk pengelolaan *dataset*, digunakan `resolve_dataset_root()` dan `load_image_classification_dataset()`. Keduanya menangani struktur folder *dataset* Intel seperti `seg_train/seg_train`. Jika folder *validation* tidak tersedia, *split validation* dibentuk dari *train* menggunakan `StratifiedShuffleSplit()` sehingga proporsi kelas tetap terjaga.

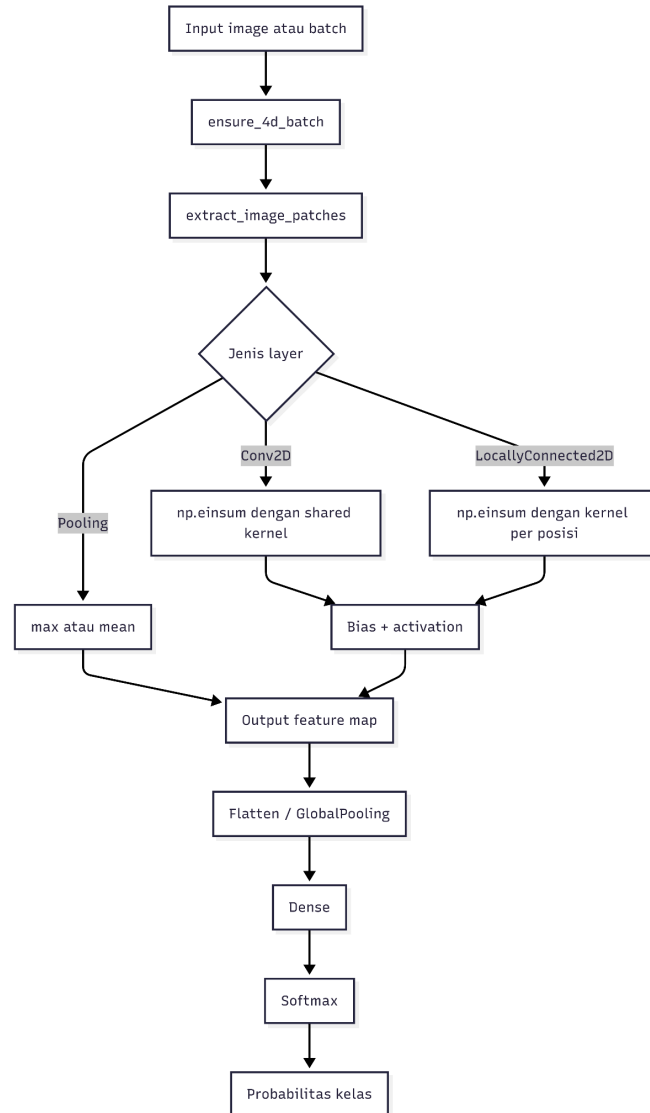
Setelah *split* tersedia, data diubah menjadi *pipeline* TensorFlow melalui `build_tf_dataset()`. Fungsi ini melakukan `read_file`, `decode_image`, `resize`, `cast` ke `float32`, normalisasi, lalu menyusun *pipeline* `tf.data` dengan operasi `map`, `shuffle`, `cache`, `batch`, dan `prefetch`. Metadata *split* kemudian disimpan melalui `save_dataset_manifest()`.

2.1.3 Implementasi *Forward Propagation from scratch*

Implementasi *forward propagation from scratch* diletakkan pada `src/cnn/core/`, `src/cnn/nn/`, dan [src/cnn/training/serialization.py](#).

Bagian ini menghitung proses prediksi CNN secara eksplisit dengan NumPy dari bobot hasil *training* Keras.

Alur *forward propagation* CNN *from scratch* adalah sebagai berikut.



Gambar 2.1.3.1 Alur *forward propagation* CNN *from scratch*

Sumber : arsip penulis

Model *scratch* dibungkus oleh kelas `Sequential` dan `method forward()` menjalankan *layer-layer* secara berurutan. *Output* dari satu *layer* menjadi *input* bagi *layer* berikutnya sampai model menghasilkan distribusi probabilitas kelas. Jika inferensi

dijalankan pada banyak gambar sekaligus, method `predict()` dapat memproses data per *batch*.

Selanjutnya, *layer* `Conv2D` mengimplementasikan konvolusi 2D dengan *shared parameters*. Bobot dibaca dari Keras dalam bentuk kernel 4D (`kH`, `kW`, `C_in`, `C_out`) dan bias 1D. Sebelum konvolusi dihitung, *input* dinormalisasi menjadi tensor 4D melalui `ensure_4d_batch()`. Setelah itu, `extract_image_patches()` digunakan untuk mengekstraksi *patch* lokal sesuai ukuran `kernel`, `stride`, `padding`, dan dilasi.

Operasi konvolusi dilakukan menggunakan `np.einsum`, sehingga setiap *patch* *input* dikalikan dengan kernel yang sama pada seluruh posisi spasial. Setelah hasil perkalian diperoleh, bias ditambahkan dan fungsi aktivasi diterapkan. Secara matematis, untuk setiap posisi spasial (*i*, *j*) dan filter *f*, operasi ini dapat ditulis sebagai berikut.

$$z_{i,j,f} = \sum_{u,v,c} X_{i+u,j+v,c} \times W_{u,v,c,f} + b_f$$

$$a_{i,j,f} = \text{activation}(z_{i,j,f})$$

Jika *input* semula hanya satu gambar, hasil akhir dikembalikan ke bentuk tanpa dimensi *batch* menggunakan `restore_batch_dim()`.

Selanjutnya juga ada *layer* `Conv2D`, terdapat *layer* `LocallyConnected2D` yang mengimplementasikan konvolusi lokal tanpa *parameter sharing*. Tahap awalnya mirip dengan `Conv2D`, yakni *patch* citra diekstraksi lebih dulu, lalu diratakan menjadi bentuk (`batch`, `posisi_output`, `patch_dim`) dan dikalikan dengan kernel 3D berbentuk (`posisi_output`, `patch_dim`, `filters`).

Perbedaan utamanya dengan `Conv2D` adalah setiap posisi *output* memiliki bobot sendiri. Jika pada `Conv2D` satu kernel dipakai berulang untuk seluruh citra, maka pada `LocallyConnected2D` bobot untuk posisi kiri atas, tengah, dan kanan bawah dapat berbeda.

Selain konvolusi, implementasi *scratch* juga mencakup *pooling*, *flatten*, *dense*, dan *softmax*, dengan *pooling* dibangun di atas kelas dasar `_Pooling2D`. Method `forward()` kembali memanfaatkan `extract_image_patches()`, lalu operasi ini didelegasikan ke `_pool()`. Pada `MaxPooling2D`, reduksi dilakukan dengan `np.max`, dan pada `AveragePooling2D`, reduksi dilakukan dengan `np.mean`.

Untuk *global pooling*, yakni `GlobalAveragePooling2D` dan `GlobalMaxPooling2D`, prosesnya langsung mereduksi seluruh dimensi spasial (H, W) menjadi satu nilai per kanal. Setelah itu, `Flatten` mengubah tensor spasial menjadi vektor 1D dengan urutan *row-major* (`C order`) agar konsisten dengan *behaviour* dari Keras.

Layer `Dense` kemudian menghitung proyeksi linear menggunakan `np.tensordot`, menambahkan bias, lalu menerapkan aktivasi. Pada layer klasifikasi akhir, aktivasi yang digunakan adalah *softmax* dengan implementasi berikut.

$$shifted = x - \max(x)$$

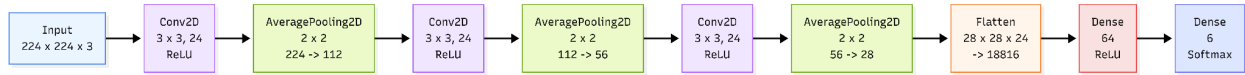
$$softmax(x)_k = \frac{\exp(shifted_k)}{\sum_j \exp(shifted_j)}$$

Agar model hasil *training* Keras dapat diuji dengan implementasi NumPy, dibuat fungsi `keras_to_scratch()`. Fungsi ini membaca arsitektur Keras lalu memetakan *layer-layer* yang didukung seperti `Conv2D`, `KerasLocallyConnected2D`, `Pooling`, `Flatten`, `Dense`, dan `Activation` ke kelas *scratch* yang ekuivalen. Bobot dari masing-masing *layer* kemudian disalin ke format NumPy.

2.1.4 Implementasi Pelatihan Model

Implementasi pelatihan model CNN terutama berada pada [src/cnn/training/train.py](#), dan [src/cnn/training/experiments.py](#). Pada tahap ini, model dibangun menggunakan `tf.keras.Sequential` lalu dilatih untuk melakukan klasifikasi enam kelas pada dataset Intel Image Classification.

Arsitektur model terdiri dari beberapa blok konvolusi. Setiap blok berisi satu *layer* konvolusi beraktivasi ReLU dan satu *layer pooling*. Setelah seluruh blok konvolusi, representasi spasial diratakan menggunakan Flatten, diteruskan ke satu *hidden dense layer*, lalu ke *layer* klasifikasi akhir beraktivasi softmax.



Gambar 2.1.4.1 Alur arsitektur model CNN

Sumber : arsip penulis

Pada eksperimen yang dijalankan, seluruh model *shared* menggunakan *input* $224 \times 224 \times 3$, `padding="same"`, satu *dense layer* berukuran 64, dan *output* 6 kelas. Variasi arsitektur dibentuk melalui `build_experiment_grid()` sehingga total terdapat 16 *shared model*.

Hyperparameter	Variasi
Jumlah <i>layer</i> konvolusi	2, 3
Banyak filter per <i>layer</i>	16, 24
Ukuran kernel per <i>layer</i>	(3, 3) ; (5, 5)
Jenis <i>pooling</i>	max, avg

Dalam implementasi ini, variasi jumlah filter dan ukuran kernel dibuat seragam untuk seluruh *layer* pada satu eksperimen. Sebagai contoh, konfigurasi `cnn_13_f24-24-24_k3x3-3x3-3x3_avg` berarti model memiliki tiga blok konvolusi. Masing-masing blok memakai 24 filter berukuran 3×3 dan seluruh *pooling* menggunakan *average pooling*.

Proses kompilasi model dilakukan pada fungsi `compile_cnn_classifier()`. Optimizer yang digunakan adalah Adam dengan *learning rate* $1e-3$. Fungsi *loss* yang digunakan adalah `SparseCategoricalCrossentropy`. Metrik yang dicatat adalah `SparseCategoricalAccuracy` dan `SparseMacroF1`. Metrik `SparseMacroF1`

dibuat sebagai *wrapper* di atas `tf.keras.metrics.F1Score` dengan cara mengubah label *integer* menjadi *one-hot* terlebih dahulu.

Pipeline data training memanfaatkan `build_tf_dataset()`. *Split train* di-*shuffle*, sedangkan *split validation* dan *test* tidak. Eksperimen dijalankan dengan `seed=42`, `batch_size=4`, dan `epochs=6`. Ukuran dataset yang dipakai adalah 11227 gambar *train*, 2807 gambar *validation*, dan 3000 gambar *test*. Kelas yang digunakan adalah `buildings`, `forest`, `glacier`, `mountain`, `sea`, dan `street`.

Setelah *training* selesai, artefak yang disimpan meliputi model final `.keras`, bobot `.weights.h5`, riwayat *loss* dan metrik dalam JSON, serta file *summary* yang memuat konfigurasi eksperimen, jumlah parameter, nilai validasi, dan *epoch* terbaik.

Arsitektur terbaik dari *shared* kemudian digunakan sebagai dasar eksperimen *non-shared*. Konfigurasinya dibangun ulang melalui `build_local_config()` dengan mengganti semua `Conv2D` menjadi `KerasLocallyConnected2D`, dengan *hyperparameter* lain tetap dipertahankan.

2.1.5 Implementasi *Batch Inference*

Fitur ini diimplementasikan melalui method `predict()` pada kelas `Sequential`. Jika parameter `batch_size` diberikan, *input* dipotong menjadi beberapa *chunk*. Setiap *chunk* diteruskan ke `forward()`, lalu seluruh hasil digabung kembali dengan `np.concatenate`.

Helper `ensure_4d_batch()` dan `restore_batch_dim()` membuat *layer scratch* dapat menerima satu gambar (`H`, `W`, `C`) maupun *batch* gambar (`N`, `H`, `W`, `C`) tanpa mengubah logika utama *layer*.

2.1.6 Implementasi *Feature Map Extraction*

Fitur ini diimplementasikan pada [src/cnn/visualization.py](#) melalui beberapa fungsi berikut.

Fungsi	Kegunaan
--------	----------

<code>list_feature_layers()</code>	Mengembalikan nama semua <i>layer</i> konvolusi / lokal
<code>get_last_feature_layer()</code>	Mengambil <i>layer</i> fitur terakhir untuk visualisasi
<code>extract_feature_maps()</code>	Menghasilkan <i>output</i> aktivasi dari <i>layer-layer</i> fitur terpilih
<code>plot_feature_maps()</code>	Menampilkan beberapa <i>channel feature map</i> dalam bentuk <i>grid</i>

`extract_feature_maps()` membangun model Keras sementara yang *output*-nya adalah aktivasi dari *layer* fitur yang diminta. Aktivasi ini kemudian dipakai untuk melihat pola yang ditangkap oleh kanal-kanal tertentu pada citra *input*.

2.1.7 Implementasi Grad-CAM

Implementasi ini menghitung kontribusi tiap kanal pada *layer* konvolusi terakhir terhadap skor kelas target lalu mengubahnya menjadi *heatmap*. Secara matematis, jika A^k adalah *feature map* kanal ke- k dan y^c adalah skor untuk kelas c , maka bobot penting kanal dihitung sebagai berikut.

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$

Kemudian *heatmap* Grad-CAM dibentuk dengan persamaan berikut.

$$L_{Grad-CAM}^c = ReLU(\sum_k \alpha_k^c A^k)$$

Implementasi tersebut berada pada fungsi `make_gradcam_heatmap()`. Alurnya adalah sebagai berikut.

1. Menyiapkan satu gambar sebagai *batch* melalui `_prepare_image_batch()`.

2. Menentukan *layer* fitur target, *default*-nya *layer* fitur terakhir melalui `get_last_feature_layer()`.
3. Menjalankan `tf.GradientTape()` untuk menghitung gradien skor kelas terhadap *output layer* fitur.
4. Merata-ratakan gradien pada dimensi spasial untuk memperoleh bobot kanal.
5. Menghitung kombinasi linear antara *feature map* dan bobot kanal.
6. Menerapkan ReLU dan normalisasi agar *heatmap* berada pada rentang $[0, 1]$.

Selanjutnya, fungsi `_forward_to_class_scores()` melanjutkan *forward pass* dari *output layer* fitur sampai ke skor kelas sebelum softmax. Setelah *heatmap* diperoleh, fungsi `resize_heatmap()` melakukan interpolasi bilinear ke ukuran citra asli. Fungsi `overlay_heatmap()` menggabungkan *heatmap* berwarna dengan citra *input*. Fungsi `plot_gradcam()` menyusun visualisasi tiga panel yang berisi gambar asli, *heatmap*, dan *overlay*.

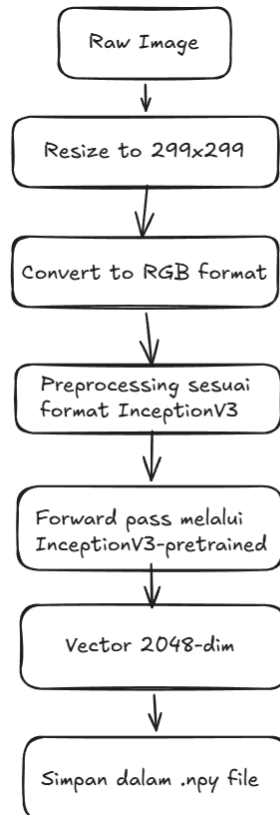
— 2.2 Implementasi RNN & LSTM

2.2.1 Implementasi *Feature Extraction*

Implementasi *Feature Extraction* dapat dilihat pada [lampiran berikut](#). Feature extraction dilakukan untuk mengubah setiap gambar pada dataset Flickr8k menjadi vektor fitur berdimensi tetap yang dapat digunakan sebagai input decoder captioning. Tahap ini diperlukan karena model decoder berbasis RNN atau LSTM tidak menerima gambar mentah secara langsung, melainkan menerima representasi numerik dari gambar. Dengan demikian, proses captioning dipisahkan menjadi dua komponen utama: encoder visual untuk mengekstraksi fitur gambar, dan decoder sequence untuk menghasilkan caption.

Pada implementasi yang kami buat, encoder visual yang digunakan adalah *InceptionV3* dengan bobot pretrained dari ImageNet. Model *InceptionV3* digunakan tanpa classification head dengan konfigurasi `include_top=False` dan `pooling="avg"`. Artinya, layer klasifikasi akhir *InceptionV3* tidak digunakan, sehingga output model bukan berupa probabilitas kelas ImageNet, melainkan vektor fitur hasil global average pooling. Vektor ini merepresentasikan informasi visual tingkat tinggi dari gambar, seperti objek, pola, dan struktur visual yang relevan untuk pembentukan caption.

Secara konseptual, proses feature extraction dapat diringkas sebagai berikut :



Gambar 2.2.1.1 Diagram proses *feature extraction*

Sumber : arsip penulis

Implementasi encoder terdapat pada fungsi *build_encoder* di

```
encoder = InceptionV3(weights="imagenet", include_top=False,
```

```
pooling="avg")
encoder.trainable = False
```

Konfigurasi tersebut menunjukkan bahwa *InceptionV3* digunakan sebagai fixed *feature extractor*. Parameter `weights="imagenet"` berarti model menggunakan bobot hasil pretraining pada dataset ImageNet. Parameter `include_top=False` menghapus classification head bawaan InceptionV3, sedangkan `pooling="avg"` menghasilkan satu vektor fitur untuk setiap gambar melalui global average pooling. Properti `encoder.trainable = False` memastikan bobot encoder tidak diperbarui selama proses training decoder.

Metadata yang disimpan mencakup jenis encoder, bobot yang digunakan, ukuran input, fungsi preprocessing, bentuk fitur, tipe data fitur, jumlah gambar, dan jumlah data pada masing-masing split. Informasi ini membantu memastikan bahwa proses training decoder dapat direproduksi dan bahwa fitur yang digunakan sesuai dengan konfigurasi encoder yang diharapkan.

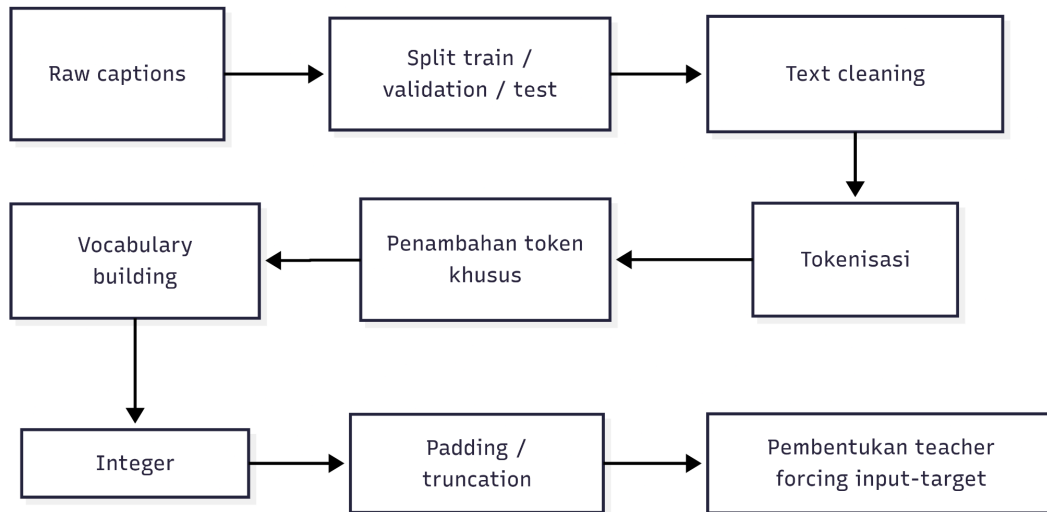
Secara keseluruhan, tahap feature extraction menghasilkan representasi visual yang ringkas dan informatif dari setiap gambar. Representasi ini kemudian digunakan sebagai input visual untuk decoder RNN/LSTM. Dengan memisahkan ekstraksi fitur dari training decoder, proses pelatihan menjadi lebih efisien karena gambar tidak perlu terus-menerus melewati CNN encoder pada setiap epoch. Decoder cukup membaca fitur yang sudah tersimpan, sehingga eksperimen terhadap variasi arsitektur decoder dapat dilakukan dengan biaya komputasi yang lebih rendah.

2.2.2 Implementasi *Preprocessing Caption*

Implementasi *Preprocessing Caption* dapat dilihat pada [lampiran berikut](#). Preprocessing caption dilakukan untuk mengubah caption mentah pada dataset Flickr8k menjadi representasi numerik yang dapat digunakan oleh decoder RNN/LSTM. Karena decoder bekerja pada urutan token diskrit, setiap caption perlu dibersihkan,

ditokenisasi, dikonversi menjadi indeks vocabulary, lalu disusun menjadi pasangan input-target untuk proses pelatihan dengan teacher forcing.

Secara umum, pipeline preprocessing caption terdiri dari beberapa tahap:



Gambar 2.2.2.1 Diagram proses *pipeline preprocessing caption*
Sumber : arsip penulis

2.2.2.1 Pembentukan Split Dataset

Sebelum preprocessing dilakukan, dataset terlebih dahulu dibagi menjadi train, validation, dan test set. Pembagian ini dilakukan berdasarkan daftar gambar yang tersedia dan caption yang memiliki pasangan gambar. Secara default, dataset dibagi menjadi 6000 gambar untuk training, 1000 gambar untuk validation, dan 1000 gambar untuk test. Caption untuk masing-masing split kemudian disimpan sebagai artefak agar tahap preprocessing, training, dan evaluasi menggunakan pembagian data yang konsisten

2.2.2.2 Text Cleaning

Tahap selanjutnya adalah text cleaning. Setiap caption diubah menjadi huruf kecil, lalu karakter selain huruf alfabet dan spasi dihapus atau diganti dengan spasi. Setelah itu, spasi berlebih dirapikan dan caption dipecah menjadi token berdasarkan spasi. Dengan proses ini, variasi akibat kapitalisasi, tanda baca, angka, atau simbol tidak dianggap sebagai token yang berbeda.

Misalnya, caption seperti:

```
"A dog, running on the grass!"
```

akan diproses menjadi:

```
["a", "dog", "running", "on", "the", "grass"]
```

Setelah caption dibersihkan, sistem menambahkan beberapa token khusus.

- Token `<start>` ditambahkan di awal caption sebagai penanda awal sequence,
- Token `<end>` ditambahkan di akhir caption sebagai penanda berhenti.
- Token `<pad>` untuk menyamakan panjang sequence, dan
- Token `<unk>` untuk merepresentasikan kata yang tidak ditemukan di vocabulary.

Dengan demikian, contoh caption sebelumnya menjadi:

```
[  
"<start>", "a", "dog", "running", "on", "the", "grass",  
"<end>"  
]
```

Vocabulary dibangun hanya dari caption pada training set. Keputusan ini penting untuk mencegah information leakage dari validation dan test set ke proses training. Seluruh kata unik pada training captions dikumpulkan setelah proses cleaning, lalu digabungkan dengan token khusus.

Setelah vocabulary terbentuk, setiap caption dikonversi menjadi sequence integer. Setiap token diganti dengan indeks yang sesuai pada vocabulary. Jika terdapat token yang tidak ada di vocabulary, token tersebut diganti dengan indeks `<unk>`. Hal ini memungkinkan model tetap menerima caption validation atau test yang mengandung kata di luar vocabulary training.

Secara konseptual:

```
["<start>", "a", "dog", "running", "<end>"]
```

Akan diubah menjadi


```
[start_idx, idx("a"), idx("dog"), idx("running"), end_idx]
```

Dimana fungsi `idx()` adalah fungsi untuk mendapatkan index dari string `str` pada vocabulary yang telah dibentuk

2.2.2.3 Padding dan Truncation

Tahap selanjutnya adalah padding dan truncation. Decoder membutuhkan input dengan panjang tetap, sehingga semua caption harus diseragamkan panjangnya. Panjang maksimum sequence ditentukan dari caption terpanjang pada training set setelah penambahan token `<start>` dan `<end>`. Caption yang lebih pendek akan ditambahkan token `<pad>` sampai mencapai panjang tersebut. Jika ada caption yang lebih panjang dari panjang maksimum, caption dipotong, dan token terakhir dipastikan menjadi `<end>` agar sequence tetap memiliki penanda akhir.

Contoh padding:

```
[<start>, a, dog, <end>]
```

Akan diubah menjadi

```
[<start>, a, dog, <end>, <pad>, <pad>]
```

Setelah caption berbentuk sequence integer dengan panjang tetap, data disusun menjadi format teacher forcing. Pada teacher forcing, decoder diberi token ground truth sebelumnya untuk memprediksi token berikutnya. Dengan demikian, setiap caption dipisah menjadi dua sequence: `caption_inputs` dan `caption_targets`.

Misalnya sequence caption adalah:

```
[<start>, a, dog, runs, <end>, <pad>]
```

Maka pasangan teacher forcing menjadi:

- `caption_inputs`:

```
[<start>, a, dog, runs, <end>]
```

- `caption_targets`:

```
[a, dog, runs, <end>, <pad>]
```

Artinya, pada setiap timestep, model belajar memprediksi token berikutnya berdasarkan fitur gambar dan token-token sebelumnya. Jika input pada timestep tertentu adalah <start>, target yang diharapkan adalah kata pertama dari caption. Jika inputnya adalah kata terakhir sebelum akhir caption, targetnya adalah <end>.

Selain sequence caption, setiap training sample juga dihubungkan dengan fitur gambar yang sesuai. Karena fitur gambar telah disimpan sebagai matriks hasil feature extraction, setiap sample cukup menyimpan indeks fitur gambar yang menunjuk ke baris fitur dari gambar terkait. Dengan cara ini, satu gambar yang memiliki beberapa caption dapat menghasilkan beberapa pasangan training, semuanya mengacu pada fitur gambar yang sama tetapi memiliki target caption yang berbeda

Secara keseluruhan, preprocessing caption mengubah data teks mentah menjadi representasi terstruktur yang sesuai untuk pelatihan model sequence. Tahap ini memastikan bahwa decoder menerima input dengan format konsisten, vocabulary yang terkontrol, dan target prediksi yang sejajar dengan mekanisme teacher forcing. Dengan preprocessing ini, model RNN/LSTM dapat belajar menghasilkan caption secara autoregressive, yaitu memprediksi kata berikutnya berdasarkan fitur gambar dan kata-kata sebelumnya

2.2.3 Implementasi Arsitektur Decoder

Arsitektur decoder bertugas mengubah fitur gambar dan sequence token caption menjadi distribusi probabilitas kata pada setiap timestep. Pada sistem image captioning ini, encoder CNN hanya digunakan untuk menghasilkan representasi visual gambar, sedangkan proses pembangkitan caption dilakukan oleh decoder berbasis recurrent neural network. Dua jenis recurrent layer yang digunakan adalah Simple RNN dan LSTM.

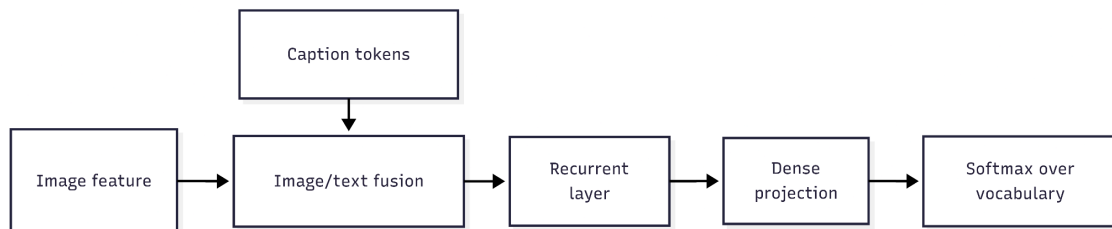
Secara umum, decoder menerima dua input utama:

- image feature

- caption token sequence

Fitur gambar berasal dari hasil ekstraksi InceptionV3, sedangkan caption token sequence berasal dari hasil preprocessing dan teacher forcing. Decoder kemudian menghasilkan output berupa distribusi probabilitas atas vocabulary untuk setiap timestep caption.

Secara konseptual :



Gambar 2.2.3.1 Diagram proses arsitektur decoder

Sumber : arsip penulis

Output decoder memiliki bentuk umum:

- `(batch_size, decoder_timesteps, vocab_size)`

Artinya, untuk setiap data dalam batch dan setiap timestep caption, model menghasilkan probabilitas untuk seluruh kata dalam vocabulary.

2.2.3.1 Komponen Utama Decoder

Decoder terdiri dari beberapa komponen utama:

Komponen	Fungsi
Image feature input	Menerima vektor fitur gambar dari CNN encoder
Token input	Menerima sequence token caption dalam bentuk integer
Embedding layer	Mengubah token integer menjadi vektor embedding
Recurrent layer	Memproses sequence secara berurutan menggunakan Simple RNN atau LSTM
Dense output layer	Memetakan hidden state ke ukuran vocabulary
Softmax	Menghasilkan probabilitas kata untuk setiap timestep

Token caption tidak langsung diproses sebagai integer. Sebelum masuk ke recurrent layer, token terlebih dahulu dilewatkan ke embedding layer. Embedding layer mengubah setiap indeks token menjadi vektor berdimensi tetap sehingga hubungan semantik antar-token dapat dipelajari selama training.

Jika sequence input adalah:

```
[<start>, a, dog, runs]
```

maka embedding layer mengubahnya menjadi:

```
[e_<start>, e_a, e_dog, e_runs]
```

Vektor embedding inilah yang kemudian diproses oleh recurrent layer.

2.2.3.2 Variasi Recurrent Layer: RNN dan LSTM

Terdapat dua jenis decoder recurrent yang diimplementasikan, yaitu decoder berbasis RNN dan decoder berbasis LSTM.

Pada RNN, model menyimpan satu state utama, yaitu hidden state. Hidden state diperbarui pada setiap timestep berdasarkan input saat ini dan hidden state sebelumnya. RNN relatif sederhana dan efisien, tetapi memiliki keterbatasan dalam menyimpan dependensi jangka panjang karena informasi lama harus terus dibawa melalui hidden state yang sama.

Pada LSTM, model memiliki dua state, yaitu hidden state dan cell state. LSTM menggunakan gate untuk mengontrol informasi yang masuk, dipertahankan, dan dikeluarkan. Dengan mekanisme ini, LSTM lebih cocok untuk sequence yang membutuhkan dependensi lebih panjang, misalnya caption dengan struktur kalimat yang lebih kompleks.

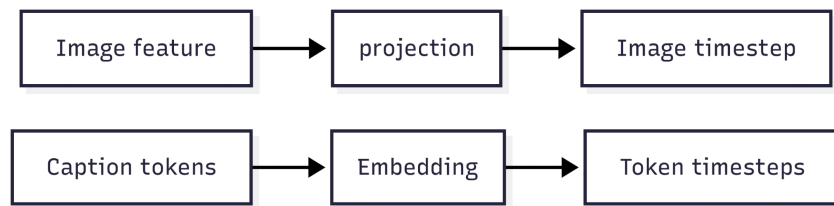
Pada implementasi, pemilihan jenis recurrent layer dilakukan melalui fungsi builder decoder. Fungsi `build_simple_rnn_decoder` membangun decoder berbasis Simple RNN, sedangkan `build_lstm_decoder` membangun decoder berbasis LSTM.

Keduanya menggunakan struktur umum yang sama, tetapi berbeda pada jenis recurrent layer yang digunakan

2.2.3.3 Pre-Inject Decoder

Pada strategi pre-inject, fitur gambar diperlakukan sebagai bagian dari sequence input. Fitur gambar terlebih dahulu diproyeksikan ke dimensi embedding menggunakan dense layer dengan aktivasi tanh, kemudian disisipkan sebagai timestep pertama sebelum token-token caption.

Secara konseptual:



Gambar 2.2.3.2 Diagram proses *Pre-Inject Decoder*

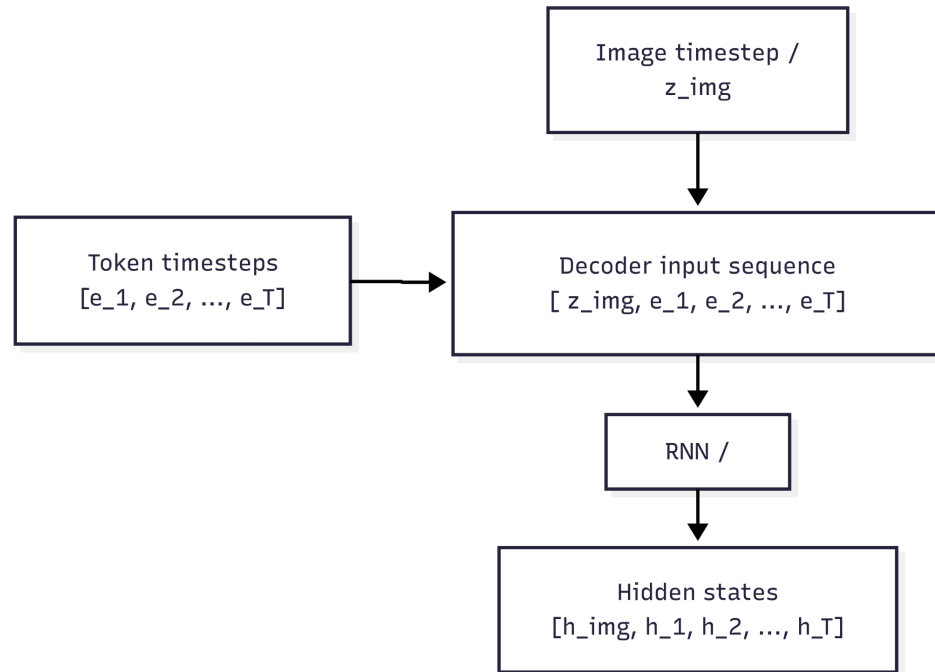
Sumber : arsip penulis

decoder input sequence

```
[image timestep, token_1, token_2, ..., token_T]
```

Dengan strategi ini, recurrent layer membaca representasi gambar terlebih dahulu. Informasi gambar kemudian dibawa melalui hidden state ke timestep-timestep berikutnya saat decoder memproses token caption.

Alurnya dapat ditulis sebagai:



Gambar 2.2.3.3 Diagram proses *Pre-Inject Decoder 2*

Sumber : arsip penulis

Karena timestep pertama adalah timestep gambar, output pada timestep tersebut tidak digunakan sebagai prediksi kata. Prediksi kata hanya dihitung dari timestep setelahnya, yaitu timestep yang sejajar dengan caption tokens.

Kelebihan pre-inject adalah implementasinya sederhana dan intuitif: gambar diperlakukan seperti “token konteks” pertama. Namun, informasi gambar harus dipertahankan oleh recurrent state sepanjang sequence, sehingga pada sequence yang panjang informasi visual dapat melemah

2.2.3.4 Init-Inject Decoder

Pada strategi init-inject, fitur gambar tidak dimasukkan sebagai timestep tambahan. Sebaliknya, fitur gambar digunakan untuk membentuk state awal recurrent layer. Dengan kata lain, recurrent layer memulai proses decoding dari state yang sudah mengandung informasi visual.

Untuk RNN, fitur gambar diproyeksikan menjadi initial hidden state:

- image feature -> Dense + tanh -> h_0

Kemudian caption token embedding diproses mulai dari state awal tersebut:

- tokens -> Embedding -> [e_1, e_2, ..., e_T]
- RNN([e_1, e_2, ..., e_T], initial_state=h_0)

Untuk LSTM, karena LSTM memiliki hidden state dan cell state, fitur gambar diproyeksikan menjadi dua initial state:

- image feature -> Dense + tanh -> h_0
- image feature -> Dense + tanh -> c_0

Kemudian LSTM memproses sequence token dengan initial state tersebut:

- LSTM([e_1, e_2, ..., e_T], initial_state=[h_0, c_0])

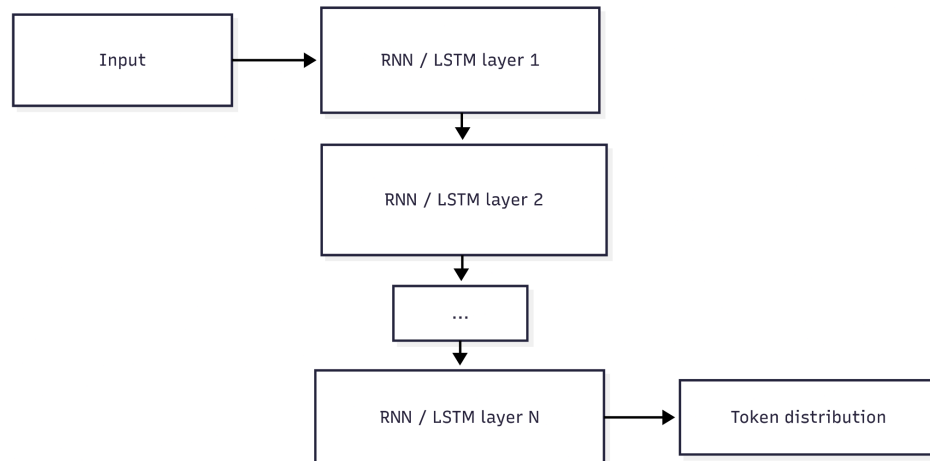
Berbeda dari pre-inject, init-inject tidak menambahkan timestep gambar ke sequence. Karena itu, seluruh output recurrent layer langsung sejajar dengan timestep caption dan tidak perlu ada proses membuang timestep pertama.

Kelebihan init-inject adalah informasi visual langsung menjadi kondisi awal decoder. Pendekatan ini lebih eksplisit dalam memposisikan gambar sebagai konteks awal pembangkitan caption, bukan sebagai token tambahan.

2.2.3.5 Multi-Layer Recurrent Decoder

Arsitektur decoder juga mendukung penggunaan lebih dari satu recurrent layer. Jika jumlah layer lebih dari satu, output sequence dari recurrent layer pertama menjadi input sequence bagi recurrent layer berikutnya.

Secara konseptual:



Gambar 2.2.3.4 Diagram proses *Multi-Layer Recurrent Decoder*

Sumber : arsip penulis

Setiap recurrent layer dikonfigurasi dengan `return_sequences=True`, sehingga layer tersebut menghasilkan output untuk seluruh timestep, bukan hanya output timestep terakhir. Hal ini penting karena image captioning membutuhkan prediksi kata pada setiap timestep. Dengan demikian, model menghasilkan satu hidden state untuk setiap posisi caption, sehingga setiap hidden state dapat dipetakan menjadi distribusi kata.

2.2.3.6 Output Layer dan Distribusi Vocabulary

Setelah sequence diproses oleh recurrent layer, setiap hidden state pada timestep caption dilewatkan ke dense layer dengan ukuran output sebesar vocabulary size. Dense layer ini menghasilkan skor untuk setiap token dalam vocabulary. Skor tersebut kemudian dinormalisasi menggunakan softmax sehingga menjadi distribusi probabilitas.

Secara matematis:

- $\text{logits}_t = h_t W_y + b_y$
- $p_t = \text{softmax}(\text{logits}_t)$

Dengan :

- H_t : hidden state pada timestep ke- t
- W_y : bobot dense output
- b_y : bias dense output
- p_t : distribusi probabilitas kata pada timestep ke- t

Token dengan probabilitas tertinggi dapat dipilih sebagai prediksi model pada time step tersebut. Pada training, distribusi ini dibandingkan dengan target token menggunakan fungsi loss. Pada inference, distribusi ini digunakan untuk memilih token berikutnya, misalnya dengan greedy decoding atau beam search.

2.2.4 Implementasi Pelatihan Decoder

Implementasi Pelatihan *Decoder* dapat dilihat pada [lampiran berikut](#). Pelatihan decoder dilakukan dengan memanfaatkan TensorFlow/Keras sebagai framework utama. Pada tahap ini, model decoder berbasis Simple RNN atau LSTM dilatih untuk menghasilkan caption berdasarkan fitur gambar dan urutan token caption yang diberikan sebagai input.

Data training yang digunakan berasal dari dua tahap sebelumnya. Pertama, fitur gambar diperoleh dari proses feature extraction menggunakan *InceptionV3*. Kedua, caption yang telah diproses digunakan untuk membentuk pasangan `caption_inputs` dan `caption_targets`. Fitur gambar berperan sebagai konteks visual, sedangkan `caption_inputs` berperan sebagai urutan token yang diberikan ke decoder.

Pelatihan dilakukan menggunakan skema teacher forcing. Dalam skema ini, decoder tidak langsung diberi hasil prediksi dari timestep sebelumnya, tetapi diberi token ground truth dari caption asli. Model kemudian diminta memprediksi token berikutnya.

Sebagai contoh, jika caption adalah :

[<start>, a, dog, runs, <end>]

maka input dan target training menjadi:

- `caption_inputs`:

[<start>, a, dog, runs]

- caption_targets:

```
[a, dog, runs, <end>]
```

Dengan cara ini, model belajar menghasilkan kata berikutnya berdasarkan konteks visual gambar dan kata-kata sebelumnya dalam caption.

Eksperimen dilakukan pada beberapa variasi decoder, yaitu decoder berbasis Simple RNN, LSTM, serta variasi jumlah recurrent layer dan hidden units. Selain itu, arsitektur decoder juga mencakup dua strategi injeksi fitur gambar, yaitu pre-inject dan init-inject, sebagaimana dijelaskan pada bagian arsitektur.

Selama proses training, Keras menangani proses optimisasi model, termasuk perhitungan loss, perhitungan gradien, dan pembaruan bobot. Validation set digunakan untuk memantau performa model pada data yang tidak digunakan untuk update bobot. Setelah training selesai, bobot model, riwayat training, konfigurasi eksperimen, dan ringkasan hasil disimpan sebagai artefak untuk keperluan evaluasi dan inference.

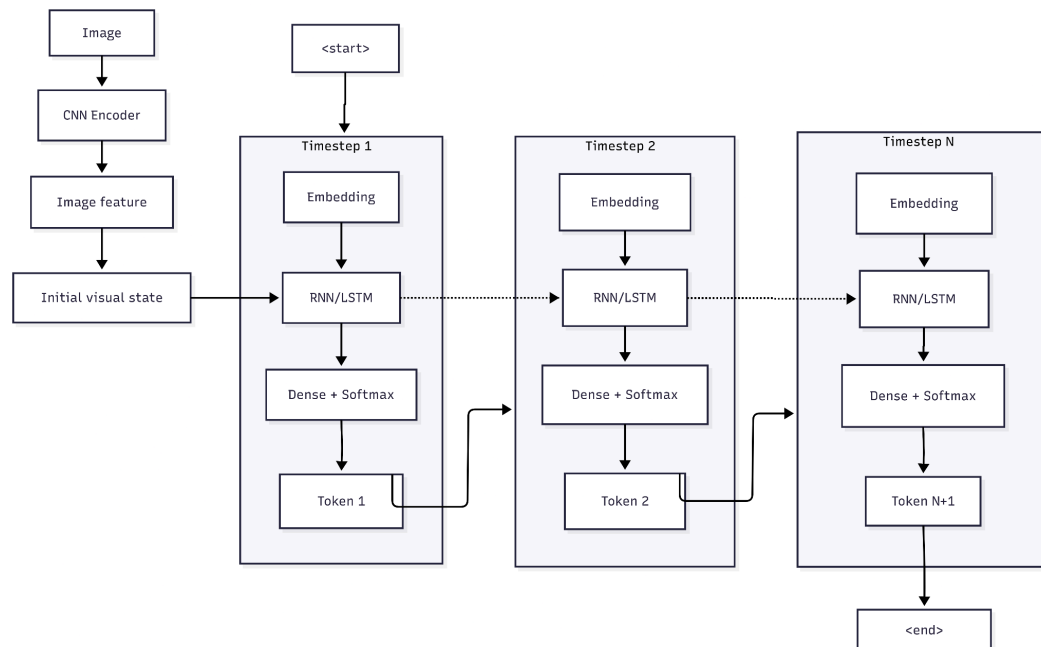
2.2.5 Implementasi *Forward Propagation from scratch*

Implementasi *Forward Propagation from scratch* dapat dilihat pada [lampiran berikut](#). Forward propagation from scratch diimplementasikan untuk mereplikasi proses inferensi decoder RNN/LSTM secara eksplisit menggunakan NumPy. Tujuan utama dari bagian ini bukan untuk melatih model ulang, melainkan untuk menunjukkan bahwa proses prediksi decoder dapat dihitung langsung dari bobot hasil training dan persamaan matematis RNN/LSTM tanpa bergantung pada abstraksi internal TensorFlow/Keras.

Pada tahap training, model dibangun dan dilatih menggunakan Keras. Setelah training selesai, bobot model disimpan. Implementasi from scratch kemudian membaca bobot tersebut dan menjalankan forward pass secara manual, mulai dari proyeksi fitur gambar, embedding token, perhitungan recurrent state, hingga pembentukan distribusi probabilitas kata.

Pada tahap inference, model tidak menerima caption lengkap sebagai input. Input awal yang tersedia hanya gambar dan token khusus <start>. Gambar terlebih dahulu diproses oleh CNN encoder menjadi image feature, sedangkan token <start> digunakan sebagai penanda awal proses pembangkitan caption. Setelah itu, caption dihasilkan secara bertahap: token yang diprediksi pada timestep sebelumnya digunakan sebagai input untuk timestep berikutnya.

Secara konseptual, proses decoding berjalan secara autoregressive:



Gambar 2.2.5.1 Diagram proses *forward propagation*

Sumber : arsip penulis

Dengan demikian, input langsung untuk RNN/LSTM pada suatu timestep bukan seluruh caption, melainkan embedding token pada timestep tersebut dan state dari timestep sebelumnya. State tersebut membawa informasi dari input-input sebelumnya, termasuk konteks visual dari gambar.

Token input, seperti <start> atau token hasil prediksi sebelumnya, diubah menjadi embedding melalui fungsi embedding. Sebagai contoh, untuk arsitektur pre-inject, urutan input recurrent dapat dipahami sebagai:

```

x_0 = z_img
x_1 = embedding(<start>)
x_2 = embedding(predicted_token_1)
x_3 = embedding(predicted_token_2)
...

```

Recurrent layer kemudian memproses sequence tersebut secara berurutan:

```

h_0 = RNN/LSTM(x_0, initial_state)
h_1 = RNN/LSTM(x_1, h_0)
h_2 = RNN/LSTM(x_2, h_1)
...

```

Output pada timestep akan diproyeksikan ke vocabulary untuk menghasilkan probabilitas kata pada time-step tersebut

$$p_i = \text{softmax}(h_i W_y + b_y)$$

Token dengan probabilitas tertinggi kemudian dipilih sebagai prediksi pertama dan digunakan sebagai input pada timestep berikutnya.

Pada implementasi ini, terdapat dua cara memasukkan image feature ke decoder, yaitu pre-inject dan init-inject. Pada arsitektur pre-inject, image feature diproyeksikan terlebih dahulu ke dimensi embedding, lalu diperlakukan sebagai timestep pertama sebelum token <start>. Secara matematis :

$$z_{\text{img}} = \tanh(v_{\text{img}} W_{\text{img}} + b_{\text{img}})$$

Pada arsitektur init-inject, image feature tidak diperlakukan sebagai timestep. Sebaliknya, image feature digunakan untuk membentuk state awal decoder. Untuk Simple RNN :

$$h_0 = \tanh(v_{\text{img}} W_h + b_h)$$

Sementara untuk LSTM, image feature digunakan untuk membentuk dua state awal, yaitu hidden state dan cell state:

$$h_0 = \tanh(v_{img} W_h + b_h)$$

$$c_0 = \tanh(v_{img} W_c + b_c)$$

2.2.5.2 Forward Propagation pada RNN

Pada Simple RNN, model mempertahankan satu state utama, yaitu hidden state. Hidden state menyimpan ringkasan informasi dari timestep-timestep sebelumnya. Pada awal sequence, hidden state diinisialisasi sebagai vektor nol.

Untuk setiap timestep, Simple RNN menghitung hidden state baru dengan persamaan:

$$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$$

Dimana,

X_t : Input pada timestep ke-t

H_t : Hidden state pada timestep ke-t

h_{t-1} : adalah Hidden state timestep sebelumnya

W_x : Bobot input ke hidden state

W_h : Bobot recurrent dari hidden state sebelumnya

b : Bias

Secara implementasi, proses ini dilakukan dengan melakukan iterasi sepanjang sequence. Pada setiap timestep, input saat ini dikombinasikan dengan hidden state sebelumnya, lalu dilewatkan ke fungsi aktivasi tanh. Hidden state yang dihasilkan kemudian disimpan sebagai output timestep tersebut dan digunakan kembali pada timestep berikutnya. Dengan mekanisme ini, RNN membaca sequence secara berurutan. Informasi gambar yang berada pada timestep pertama diteruskan melalui hidden state ke timestep-timestep caption berikutnya.

2.2.5.3 Forward Propagation pada LSTM

LSTM menggunakan mekanisme yang lebih kompleks dibandingkan Simple RNN. Selain hidden state, LSTM memiliki cell state yang berfungsi sebagai jalur memori jangka panjang. LSTM juga menggunakan beberapa gate untuk mengatur informasi yang masuk, dilupakan, dan dikeluarkan.

State utama pada LSTM adalah:

Jenis Gate	Fungsi
Hidden State [h _t]	Output representasi pada timestep saat ini
Cell State [c _t]	Menyimpan memori jangka panjang
Input gate [i _t]	Mengatur informasi baru yang masuk ke Cell State
Forget Gate [f _t]	Mengatur informasi lama yang dipertahankan
Cell candidate [g _t]	Kandidat informasi baru yang dapat ditambahkan ke memori
Output Gate	Mengatur informasi yang menjadi hidden state

Secara matematis, perhitungan LSTM dapat ditulis sebagai:

```
[z_i, z_f, z_g, z_o] = x_t W_x + h_{t-1} W_h + b
i_t = sigmoid(z_i)
f_t = sigmoid(z_f)
g_t = tanh(z_g)
o_t = sigmoid(z_o)
c_t = f_t * c_{t-1} + i_t * g_t
h_t = o_t * tanh(c_t)
```

Pada implementasi from scratch, seluruh nilai gate dihitung dalam satu operasi linear, kemudian dipisahkan menjadi empat bagian: input gate, forget gate, cell candidate, dan output gate. Masing-masing bagian kemudian diberi fungsi aktivasi sesuai teori. Gate menggunakan sigmoid karena nilainya berperan sebagai pengontrol dengan rentang 0 sampai 1, sedangkan cell candidate menggunakan tanh karena merepresentasikan kandidat informasi baru.

Dengan cell state dan gate, LSTM dapat mempertahankan informasi penting lebih stabil dibandingkan Simple RNN. Dalam konteks image captioning, mekanisme ini membantu decoder menjaga konteks visual dan struktur kalimat sepanjang proses pembangkitan caption.

2.2.5.4 Proyeksi ke Vocabulary

Setelah sequence diproses oleh Simple RNN atau LSTM, model menghasilkan hidden state untuk setiap timestep. Pada skema pre-inject, hidden state timestep pertama berasal dari fitur gambar, sehingga tidak digunakan sebagai prediksi kata. Oleh karena itu, timestep gambar diabaikan dan model hanya menggunakan hidden state yang sejajar dengan token caption. Nilai logits_t akan dihitung menggunakan nilai dari hidden state pada timestep t yang diproyeksikan menuju size vocabulary

$$\text{logits}_t = h_t W_y + b_y$$

Nilai logits_t masih berupa skor mentah untuk setiap kata dalam vocabulary. Agar dapat ditafsirkan sebagai probabilitas, skor tersebut dinormalisasi menggunakan softmax:

$$p_t = \text{softmax}(\text{logits}_t)$$

Softmax menghasilkan distribusi probabilitas token pada setiap timestep. Token dengan probabilitas tertinggi dapat dipilih sebagai prediksi model.

2.2.5.5 Greedy Decoding

Untuk menghasilkan caption, implementasi from scratch menggunakan greedy decoding. Pada greedy decoding, model selalu memilih token dengan probabilitas tertinggi pada timestep saat ini.

Alurnya adalah:

1. Isi token pertama dengan <start>
2. Jalankan forward propagation
3. Ambil token dengan probabilitas tertinggi pada timestep saat ini
4. Masukkan token tersebut sebagai input timestep berikutnya
5. Ulangi sampai token <end> muncul atau panjang maksimum tercapai

Jika token yang dipilih adalah <end>, proses decoding dihentikan. Jika bukan, token tersebut ditambahkan ke caption dan digunakan sebagai bagian dari input untuk prediksi berikutnya.

Pendekatan ini sederhana dan deterministik. Namun, karena hanya memilih token terbaik pada setiap timestep, greedy decoding tidak selalu menghasilkan caption global terbaik. Alternatif seperti beam search dapat mempertimbangkan beberapa kandidat sequence sekaligus, tetapi greedy decoding cukup untuk menunjukkan bahwa forward propagation from scratch menghasilkan prediksi yang konsisten dengan model Keras.

2.2.5.6 Validasi terhadap Model Keras

Karena implementasi from scratch membaca bobot hasil training Keras, hasil forward pass seharusnya mendekati atau sama dengan hasil inference model Keras untuk input yang sama. Oleh karena itu, validasi dilakukan dengan membandingkan prediksi dari decoder Keras dan decoder NumPy from scratch.

Secara umum, validasi dilakukan dengan:

1. Input fitur gambar dan token yang sama
2. Inference menggunakan model Keras
3. Inference menggunakan implementasi forward from scratch
4. Bandingkan token atau caption yang dihasilkan

Jika kedua model menghasilkan token yang sama pada setiap timestep, maka implementasi forward propagation from scratch berhasil mereplikasi perilaku model Keras. Jika terdapat perbedaan, perbedaan tersebut dapat dianalisis pada timestep pertama yang menghasilkan token berbeda.

Validasi ini penting karena menunjukkan bahwa persamaan matematis RNN/LSTM yang digunakan pada implementasi NumPy sudah sesuai dengan bobot dan urutan operasi pada model Keras.

2.2.6 Implementasi Beam Search

Implementasi *Beam Search* dapat dilihat pada [lampiran berikut](#). Beam search digunakan sebagai alternatif dari greedy decoding pada tahap inference captioning. Jika greedy decoding hanya memilih token dengan probabilitas tertinggi pada setiap

timestep, beam search menyimpan beberapa kandidat caption terbaik secara bersamaan. Dengan cara ini, model tidak langsung mengunci satu pilihan token sejak awal, tetapi mempertimbangkan beberapa kemungkinan sequence sebelum memilih caption akhir.

Secara high level, alur beam search adalah:

```
inisialisasi kandidat dengan token <start>
untuk setiap timestep:
    untuk setiap kandidat aktif:
        jalankan forward propagation
        ambil top-k token berikutnya
        buat kandidat baru dari setiap token tersebut
    hitung skor setiap kandidat
    simpan k kandidat terbaik
berhenti jika semua kandidat sudah menghasilkan <end>
pilih kandidat terbaik sebagai caption final
```

Nilai k disebut sebagai beam width. Pada implementasi ini, nilai default beam width adalah 3, sehingga pada setiap timestep sistem mempertahankan maksimal tiga kandidat caption terbaik.

Secara konseptual, satu kandidat dapat dipandang sebagai satu kemungkinan caption parsial. Misalnya pada timestep tertentu, beam dapat berisi kandidat seperti:

- Candidate 1: "dog" score = -0.42
- Candidate 2: "black" score = -0.55
- Candidate 3: "the" score = -0.70

Kandidat-kandidat ini kemudian diperluas pada timestep berikutnya.

Beam search dimulai dari satu kandidat kosong. Sequence token diisi dengan <pad>, lalu posisi pertama diisi dengan token <start>. Pada tahap awal, belum ada kata yang dihasilkan, sehingga words masih kosong dan log_probability bernilai 0. Pada setiap timestep, beam search memproses seluruh kandidat yang masih aktif. Untuk setiap kandidat, decoder menjalankan forward propagation berdasarkan fitur gambar dan sequence token kandidat tersebut. Hasil forward propagation adalah distribusi

probabilitas token pada timestep saat ini. Dari distribusi tersebut, sistem mengambil `beam_width` token dengan probabilitas tertinggi. Jika `beam_width=3`, maka setiap kandidat aktif dapat menghasilkan sampai tiga kandidat baru. Sebagai contoh:

```
candidate = "a dog"
top-3 token berikutnya : ["running", "standing", "playing"]
Maka, 3 kandidat baru yang akan dibuat adalah :
["a dog running", "a dog playing", "a dog standing"]
```

Beam search memilih kandidat berdasarkan akumulasi probabilitas token. Karena probabilitas banyak token jika dikalikan akan menghasilkan nilai yang sangat kecil, implementasi menggunakan log-probability. Dengan log-probability, perkalian probabilitas berubah menjadi penjumlahan:

$$P(\text{sequence}) = P(w_1) * P(w_2) * \dots * P(w_T)$$
$$\log P(\text{sequence}) = \log P(w_1) + \log P(w_2) + \dots + \log P(w_T)$$

Setiap kali kandidat diperluas dengan token baru, skor kandidat diperbarui dengan menambahkan log-probability token tersebut:

$$\text{new_score} = \text{old_score} + \log(P(\text{next_token}))$$

Namun, jika hanya menggunakan total log-probability, caption yang lebih pendek cenderung lebih diuntungkan karena memiliki lebih sedikit faktor probabilitas. Untuk mengurangi bias terhadap caption pendek, implementasi menggunakan normalisasi berdasarkan panjang caption:

$$\text{candidate_score} = \log_probability / \text{length}$$

Dengan demikian, skor kandidat adalah rata-rata log-probability per kata, bukan total log-probability mentah. Jika kandidat menghasilkan token `<end>`, kandidat tersebut dianggap selesai. Kandidat yang sudah selesai tidak diperluas lagi pada timestep berikutnya. Namun, kandidat tersebut tetap dipertahankan dalam beam agar masih dapat bersaing dengan kandidat lain yang belum selesai. Setelah proses selesai, baik karena semua kandidat menghasilkan `<end>` atau karena panjang maksimum tercapai,

kandidat dengan skor terbaik dipilih sebagai hasil akhir. Output akhirnya adalah sequence kata dari kandidat terbaik.

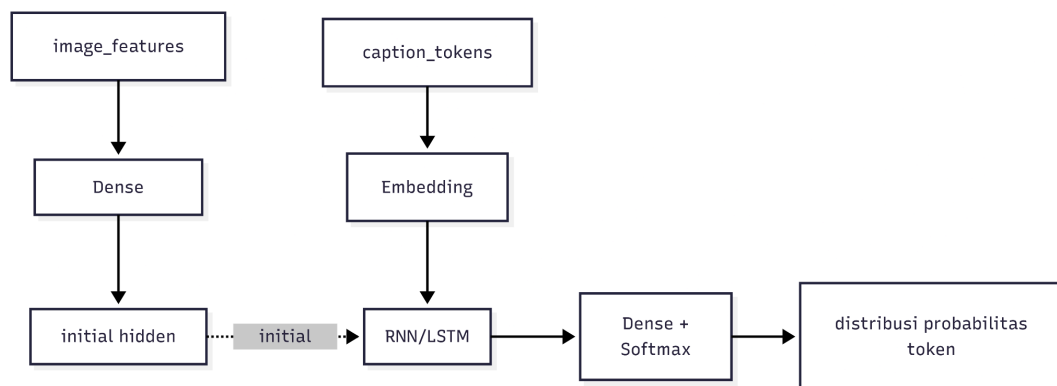
2.2.7 Implementasi Arsitektur *Init-Inject*

Implementasi arsitektur *Init-Inject* dapat dilihat pada [lampiran berikut](#). Init-inject merupakan arsitektur alternatif image captioning yang memasukkan informasi gambar sebagai initial state dari decoder recurrent. Berbeda dengan pre-inject yang menempatkan feature gambar sebagai timestep pertama sebelum token <start>, init-inject menggunakan feature gambar untuk membentuk hidden state awal decoder. Dengan demikian, caption sequence yang masuk ke RNN/LSTM hanya berisi token caption, sedangkan konteks visual diberikan melalui state awal recurrent layer.

Implementasi init-inject pada repository dibuat di `src/captioning/decoder.py`, yaitu melalui fungsi `build_init_inject_simple_rnn_decoder()` dan `build_init_inject_lstm_decoder()`. Kedua fungsi tersebut memanggil helper `_build_init_inject_decoder()` dengan jenis recurrent layer yang berbeda. Untuk RNN digunakan `layers.SimpleRNN`, sedangkan untuk LSTM digunakan `layers.LSTM`.

2.2.6.1 Arsitektur Init-Inject

Secara umum, alur arsitektur init-inject adalah sebagai berikut:



Gambar 2.2.5.1 Diagram proses *Init-Inject*
Sumber : arsip penulis

Model memiliki dua input utama, yaitu `image_features` dan `caption_tokens`. `image_features` adalah feature vector hasil ekstraksi CNN InceptionV3 dengan dimensi 2048. `caption_tokens` adalah sequence token caption hasil preprocessing dan teacher forcing.

Pada init-inject, feature gambar tidak digabungkan ke sequence input. Caption token langsung diproses oleh embedding layer:

```
x = layers.Embedding(  
    vocab_size,  
    embed_dim,  
    mask_zero=False,  
    name="token_embedding",  
) (caption_tokens)
```

Embedding layer mengubah setiap token integer menjadi vektor berdimensi `embed_dim`. Sequence embedding inilah yang menjadi input utama recurrent layer.

2.2.6.2 Image Feature sebagai Initial State

Perbedaan utama init-inject terletak pada cara image feature digunakan. Untuk setiap recurrent layer, image feature diproyeksikan menggunakan Dense layer agar dimensinya sesuai dengan jumlah hidden unit pada layer tersebut.

Untuk SimpleRNN, feature gambar diproyeksikan menjadi initial hidden state:

```
initial_hidden = layers.Dense(  
    units,  
    activation="tanh",  
    name=f"image_to_h{layer_index}",  
) (image_features)
```

Hasil Dense ini digunakan sebagai `initial_state` untuk layer SimpleRNN:

```
initial_state = [initial_hidden]
```

Kemudian recurrent layer dipanggil dengan initial state tersebut:

```
x = recurrent_layer(
    units,
    return_sequences=True,
    name=f"{recurrent_name}_{layer_index}",
)(x, initial_state=initial_state)
```

Dengan cara ini, hidden state awal h_0 tidak lagi berupa nol, tetapi berasal dari representasi visual gambar. Decoder sejak timestep pertama sudah dikondisikan oleh informasi gambar.

2.2.6.3 Init-Inject pada LSTM

Untuk LSTM, initial state terdiri dari dua komponen, yaitu hidden state h_0 dan cell state c_0 . Oleh karena itu, feature gambar diproyeksikan menjadi dua vektor berbeda.

- image feature diproyeksikan menjadi initial hidden state:

```
initial_hidden = layers.Dense(
    units,
    activation="tanh",
    name=f"image_to_h{layer_index}",
)(image_features)
```

- image feature juga diproyeksikan menjadi initial cell state:

```
initial_cell = layers.Dense(
    units,
    activation="tanh",
    name=f"image_to_c{layer_index}",
)(image_features)
```

Keduanya kemudian dimasukkan sebagai initial state LSTM:

```
initial_state = [initial_hidden, initial_cell]
```

Dengan demikian, LSTM menerima informasi visual tidak hanya pada hidden state awal, tetapi juga pada cell state awal. Cell state berfungsi sebagai jalur memori jangka panjang, sehingga pemberian image feature pada c_0 membantu model mempertahankan konteks visual sepanjang proses decoding.

2.2.6.4 Multi-Layer Init-Inject

Implementasi init-inject juga mendukung lebih dari satu recurrent layer. Jumlah layer ditentukan oleh parameter `hidden_units`, yang dapat berupa integer tunggal atau `sequence`. Fungsi `_normalize_hidden_units()` mengubah konfigurasi tersebut menjadi tuple agar dapat diproses secara seragam.

Untuk setiap layer recurrent, image feature diproyeksikan ulang ke initial state layer tersebut. Artinya, jika model memiliki 3 recurrent layer, maka setiap layer memiliki Dense projection sendiri dari image feature ke initial state.

Nama layer dibuat berbeda menggunakan indeks layer, misalnya:

- `image_to_h1`
- `image_to_h2`
- `image_to_h3`

Untuk LSTM, juga terdapat:

- `image_to_c1`
- `image_to_c2`
- `image_to_c3`

Pendekatan ini membuat setiap recurrent layer menerima conditioning langsung dari image feature, bukan hanya layer pertama.

2.2.6.5 Output Layer

Setelah `sequence caption` diproses oleh recurrent layer, output setiap timestep diteruskan ke Dense output layer:

```
token_distribution = layers.Dense(  
    vocab_size,  
    activation="softmax",  
    name="token_distribution",  
) (x)
```

Layer ini menghasilkan distribusi probabilitas terhadap seluruh vocabulary untuk setiap timestep caption.

Karena init-inject tidak menambahkan timestep image ke input sequence, tidak diperlukan proses membuang timestep pertama seperti pada pre-inject. Output recurrent langsung sejajar dengan target caption.

2.2.8 Implementasi *Backward Propagation from scratch*

Implementasi *Backward Propagation from scratch* dapat dilihat pada [lampiran berikut](#). Fungsi utama yang digunakan adalah `backward_batch()`. Fungsi ini menerima `feature_batch`, `tokens`, `targets`, dan `pad_idx`, lalu mengembalikan nilai loss serta seluruh gradient model dalam bentuk `ScratchGradients`. Backward propagation hanya diimplementasikan untuk batch. Hal ini membuat perhitungan gradient lebih konsisten dengan proses training dan memudahkan agregasi gradient dari banyak sampel sekaligus.

Struktur gradient disimpan dalam dataclass `ScratchGradients`, yang berisi:

- `image_projection`
- `token_embedding`
- `recurrent_layers`
- `token_distribution`

Gradient recurrent layer disimpan dalam `RecurrentGradients`, yang berisi gradient untuk kernel, `recurrent_kernel`, dan bias.

2.2.8.1 Alur Backward Propagation

Pada awal `backward_batch()`, sistem menjalankan forward pass manual dengan cache. Image feature diproyeksikan menggunakan `dense_tanh_forward()`, lalu hasilnya digabungkan dengan token embedding. Berbeda dari forward inference biasa, forward pada backpropagation menyimpan nilai-nilai antara yang diperlukan untuk menghitung turunan.

Untuk recurrent layer, sistem menggunakan fungsi forward khusus dengan cache, yaitu:

- `simple_rnn_forward_batch_with_cache()`
- `lstm_forward_batch_with_cache()`

Cache SimpleRNN menyimpan input sequence, hidden state sebelumnya, dan output hidden state tiap timestep. Sementara itu, cache LSTM menyimpan input sequence, hidden state sebelumnya, cell state sebelumnya, cell state saat ini, input gate, forget gate, cell candidate, output gate, dan output hidden state.

Setelah recurrent forward selesai, timestep image dibuang, lalu dense output layer menghasilkan logits. Loss dan gradient terhadap logits dihitung dengan `masked_softmax_cross_entropy_backward()`. Loss yang digunakan adalah masked sparse categorical cross-entropy. Token <pad> tidak ikut dihitung dalam loss maupun gradient. Mask dibuat dari kondisi:

```
targets != pad_idx
```

Jika jumlah token valid adalah N, maka loss dihitung sebagai rata-rata negative log likelihood hanya pada token valid:

```
loss = -sum(log(p_target) * mask) / N
```

Gradient terhadap logits diperoleh dari turunan softmax cross-entropy:

```
dlogits = probabilities
dlogits[target] -= 1
dlogits *= mask
dlogits /= valid_count
```

2.2.8.2 Backward Dense Output Layer

Gradient dense output layer dihitung dari output recurrent pada timestep caption dan dlogits.

```
dW_out = caption_steps^T dlogits
db_out = sum(dlogits)
dcaption_steps = dlogits W_out^T
```

Karena timestep image sebelumnya dibuang pada forward output, gradient recurrent sequence dibuat kembali dengan shape penuh. Gradient untuk timestep caption ditempatkan pada posisi 1:, sedangkan timestep image di posisi 0 awalnya bernilai nol.

2.2.8.3 Backward Recurrent Layer

Backward recurrent dilakukan dari layer terakhir ke layer pertama. Jika model memiliki beberapa recurrent layer, gradient dari layer atas menjadi input gradient bagi layer di bawahnya. Proses ini dilakukan dengan urutan terbalik terhadap forward propagation.

Untuk SimpleRNN, fungsi `simple_rnn_backward_batch()` melakukan backpropagation through time dari timestep terakhir ke timestep pertama. Pada setiap timestep, gradient hidden state berasal dari dua sumber, yaitu gradient output timestep saat ini dan gradient recurrent dari timestep berikutnya.

Turunan aktivasi tanh dihitung dengan:

$$da = dh * (1 - h_t^2)$$

Kemudian gradient parameter dihitung sebagai:

```
dW_x += x_t^T da
dW_h += h_{t-1}^T da
db += sum(da)
dx_t = da W_x^T
dh_next = da W_h^T
```

Untuk LSTM, fungsi `lstm_backward_batch()` juga melakukan backpropagation through time dari timestep terakhir ke timestep pertama. Gradient mengalir melalui hidden state dan cell state. Pada setiap timestep, gradient terhadap output gate, cell state, forget gate, input gate, dan cell candidate dihitung berdasarkan persamaan forward LSTM.

Gradient utama LSTM dihitung melalui:

```
dh = doutput_t + dh_next
doutput_gate = dh * tanh(c_t)
dcell = dh * o_t * (1 - tanh(c_t)^2) + dc_next
dforget_gate = dcell * c_{t-1}
dprevious_cell = dcell * f_t
```

```
dinput_gate = dcell * g_t
dcell_candidate = dcell * i_t
```

Kemudian masing-masing gradient gate dikalikan dengan turunan aktivasinya:

```
di *= i_t * (1 - i_t)
df *= f_t * (1 - f_t)
dg *= 1 - g_t^2
do *= o_t * (1 - o_t)
```

Keempat gradient gate digabung kembali menjadi dgates, lalu digunakan untuk menghitung gradient parameter LSTM:

```
dW_x += x_t^T dgates
dW_h += h_{t-1}^T dgates
db += sum(dgates)
dx_t = dgates W_x^T
dh_next = dgates W_h^T
dc_next = dprevious_cell
```

2.2.8.4 Backward Image Projection dan Embedding

Setelah gradient melewati semua recurrent layer, gradient sequence dipisahkan kembali menjadi gradient untuk image timestep dan token embedding. Gradient image timestep berada pada posisi pertama sequence:

```
dprojected_image = dx[:, 0,:]
```

Gradient ini diteruskan ke `dense_tanh_backward()` untuk menghitung gradient Dense projection image feature. Karena proyeksi menggunakan aktivasi tanh, turunan aktivasi dihitung dengan:

```
dlinear = doutput * (1 - output^2)
```

Lalu gradient parameter image projection dihitung sebagai:

```
dW_img = feature_batch^T dlinear
db_img = sum(dlinear)
```

Untuk embedding, gradient berasal dari posisi sequence setelah timestep image:

```
dtoken_embeddings = dx[:, 1:, :]
```

Karena embedding merupakan operasi lookup, beberapa token yang sama dapat muncul berkali-kali dalam batch. Oleh karena itu, akumulasi gradient embedding dilakukan menggunakan `np.add.at()`, sehingga gradient token yang muncul berulang akan dijumlahkan pada baris embedding yang sama.

2.2.8.5 Update Bobot From Scratch

Update bobot dilakukan pada fungsi `train_batch()`. Fungsi ini memanggil `backward_batch()` untuk memperoleh loss dan gradient, lalu menerapkan update menggunakan stochastic gradient descent melalui `apply_sgd_update()`.

Aturan update yang digunakan adalah:

```
W_new = W_old - learning_rate * dW
b_new = b_old - learning_rate * db
```

Update diterapkan pada bobot image projection, token embedding, seluruh recurrent layer, dan dense output layer. Hasil update dikembalikan sebagai objek `ScratchDecoder` baru, sehingga bobot lama tidak dimodifikasi secara langsung.

Selain satu batch update, diimplementasikan juga `fit_scratch()` untuk menjalankan training sederhana dari scratch selama beberapa epoch. Fungsi ini melakukan iterasi batch teacher forcing, menjalankan `train_batch()` pada setiap batch, lalu menyimpan rata-rata loss tiap epoch.

03 — Hasil Pengujian

3.1 Hasil Pengujian CNN

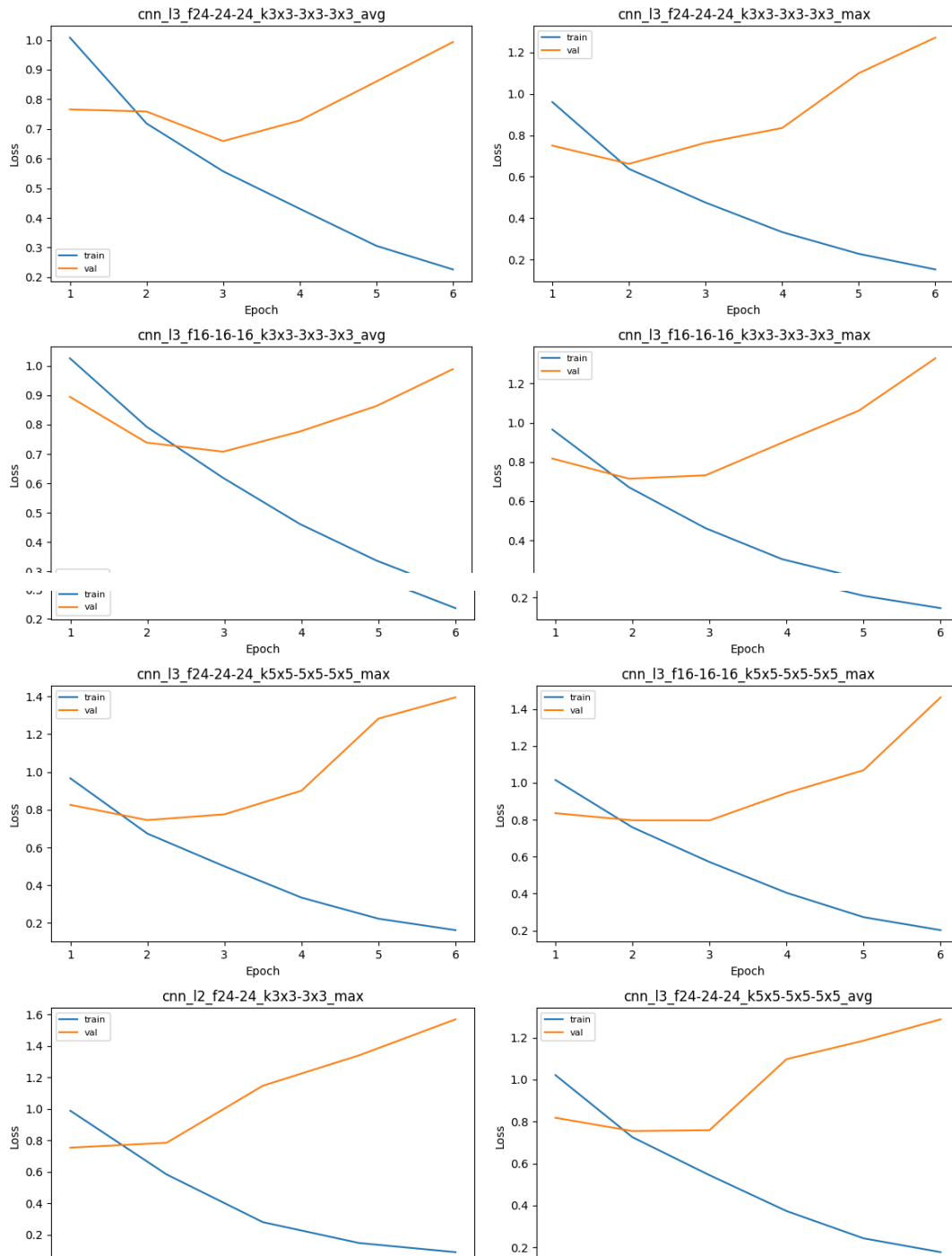
Pengujian CNN dilakukan pada 16 arsitektur *shared* dengan variasi jumlah *layer* konvolusi, jumlah filter, ukuran kernel, dan jenis *pooling*. Model *shared* terbaik adalah `cnn_l3_f24-24-24_k3x3-3x3-3x3_avg` dengan `val_macro_f1 = 0.7765`, `val_accuracy = 0.7756`, dan 1215766 parameter.

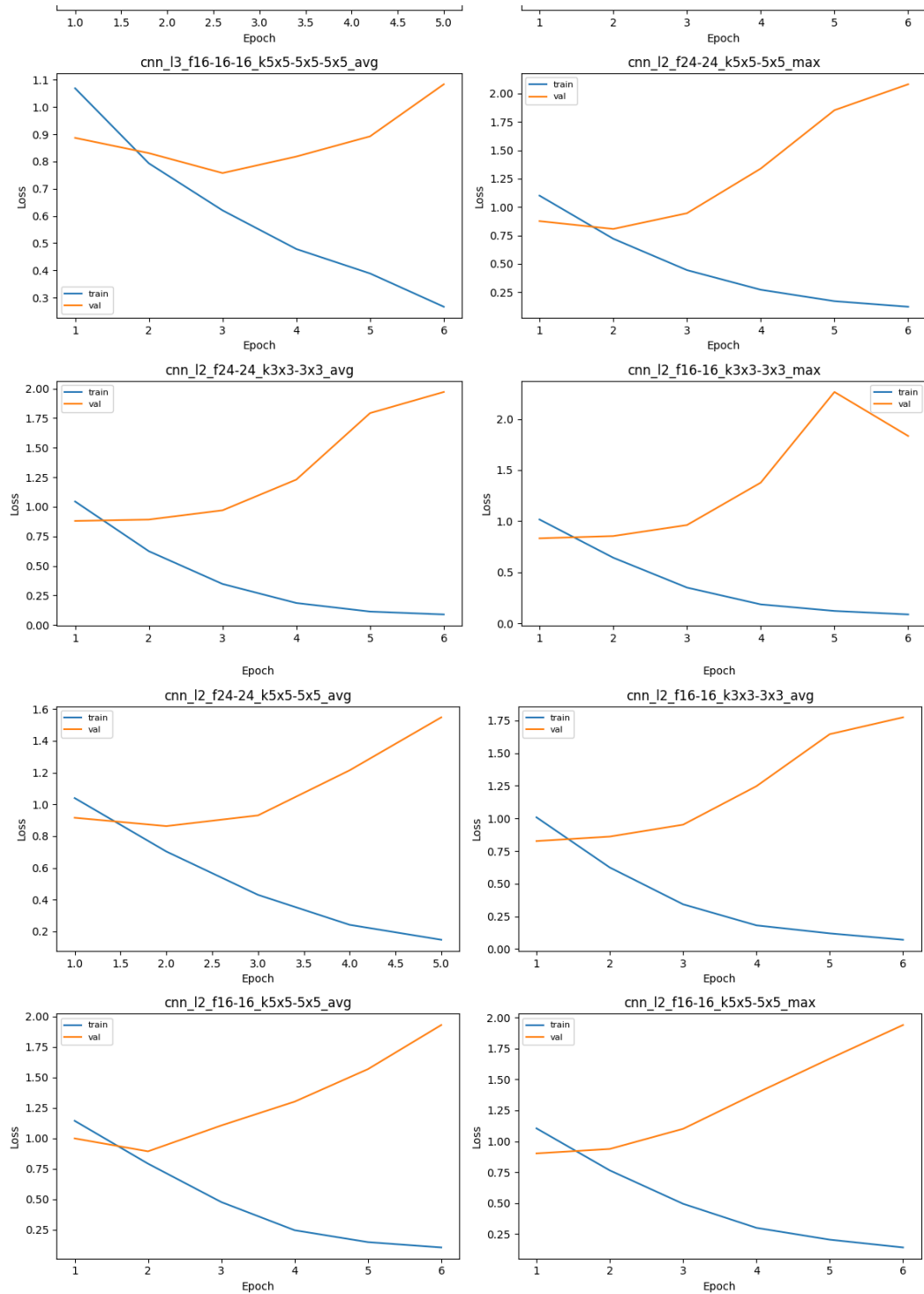
	name	layers	filters	kernels	pooling	val_loss	val_accuracy	val_macro_f1	params	best_epoch
12	cnn_l3_f24-24-24_k3x3-3x3-3x3_avg	3	(24, 24, 24)	((3, 3), (3, 3), (3, 3))	avg	0.659108	0.775561	0.776496	1215766	3
13	cnn_l3_f24-24-24_k3x3-3x3-3x3_max	3	(24, 24, 24)	((3, 3), (3, 3), (3, 3))	max	0.835253	0.766655	0.767425	1215766	4
8	cnn_l3_f16-16-16_k3x3-3x3-3x3_avg	3	(16, 16, 16)	((3, 3), (3, 3), (3, 3))	avg	0.776668	0.762024	0.764288	808358	4
9	cnn_l3_f16-16-16_k3x3-3x3-3x3_max	3	(16, 16, 16)	((3, 3), (3, 3), (3, 3))	max	1.061295	0.752761	0.753414	808358	5
15	cnn_l3_f24-24-24_k5x5-5x5-5x5_max	3	(24, 24, 24)	((5, 5), (5, 5), (5, 5))	max	0.775718	0.747417	0.748608	1235350	3
11	cnn_l3_f16-16-16_k5x5-5x5-5x5_max	3	(16, 16, 16)	((5, 5), (5, 5), (5, 5))	max	0.944949	0.748130	0.747970	817318	4
5	cnn_l2_f24-24_k3x3-3x3_max	2	(24, 24)	((3, 3), (3, 3))	max	0.783844	0.746348	0.747918	4823230	2
14	cnn_l3_f24-24-24_k5x5-5x5-5x5_avg	3	(24, 24, 24)	((5, 5), (5, 5), (5, 5))	avg	1.287300	0.742073	0.741865	1235350	6
10	cnn_l3_f16-16-16_k5x5-5x5-5x5_avg	3	(16, 16, 16)	((5, 5), (5, 5), (5, 5))	avg	0.892486	0.738867	0.740460	817318	5
7	cnn_l2_f24-24_k5x5-5x5_max	2	(24, 24)	((5, 5), (5, 5))	max	0.945574	0.726042	0.725501	4833598	3
4	cnn_l2_f24-24_k3x3-3x3_avg	2	(24, 24)	((3, 3), (3, 3))	avg	1.230199	0.721411	0.721461	4823230	4
1	cnn_l2_f16-16_k3x3-3x3_max	2	(16, 16)	((3, 3), (3, 3))	max	0.962783	0.718561	0.720193	3214486	3
6	cnn_l2_f24-24_k5x5-5x5_avg	2	(24, 24)	((5, 5), (5, 5))	avg	0.862961	0.711792	0.713127	4833598	2
0	cnn_l2_f16-16_k3x3-3x3_avg	2	(16, 16)	((3, 3), (3, 3))	avg	0.951873	0.708942	0.708356	3214486	3
2	cnn_l2_f16-16_k5x5-5x5_avg	2	(16, 16)	((5, 5), (5, 5))	avg	1.300011	0.695048	0.694436	3219350	4
3	cnn_l2_f16-16_k5x5-5x5_max	2	(16, 16)	((5, 5), (5, 5))	max	1.100256	0.684004	0.687603	3219350	3

Gambar 3.1.1 Nilai `val_macro_f1` untuk seluruh arsitektur *shared* yang diuji

Sumber : arsip penulis

Training and Validation Loss Curves for All Shared CNN Architectures



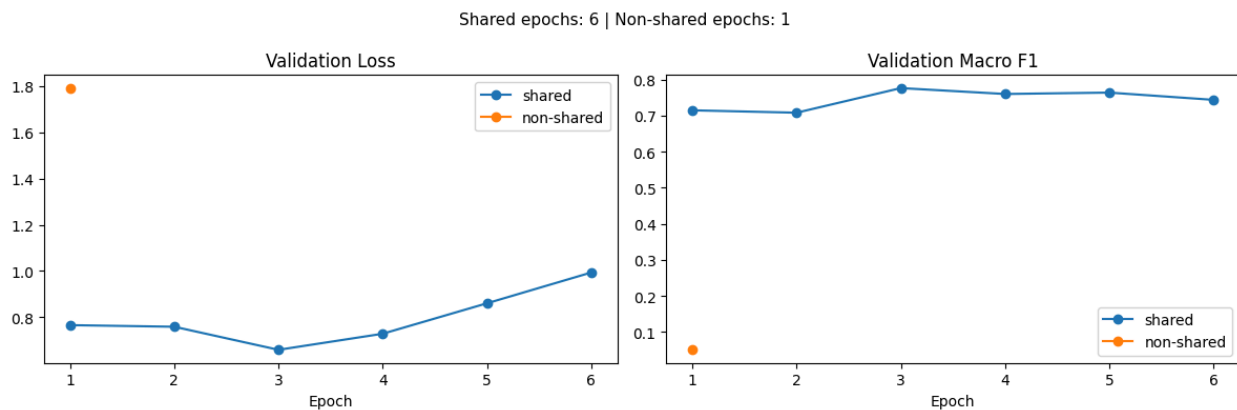


Gambar 3.1.2 Kurva *training loss* dan *validation loss* untuk semua arsitektur *shared*
 Sumber : arsip penulis

Gambar 3.1.1 dan 3.1.2 menunjukkan bahwa hampir semua arsitektur mengalami penurunan *training loss* yang konsisten, tetapi *validation loss* tidak selalu ikut turun. Beberapa model mencapai *validation loss* terbaik pada *epoch* awal lalu kembali naik pada *epoch* berikutnya. Pola ini menjadi dasar pemilihan model terbaik berdasarkan *validation metric*, bukan *training loss* saja.

Selain metrik agregat, penulis juga membandingkan prediksi akhir pada enam gambar *test* yang sama untuk model terbaik dari setiap nilai faktor. Perbandingan ini dipakai sebagai ilustrasi tambahan. Metrik utama tetap *macro F1-score* pada seluruh *split test* atau *validation*.

3.1.1 Perbandingan shared vs non-shared parameter



Gambar 3.1.1.1 Kurva validasi model *shared* dan *non-shared*

Sumber : arsip penulis

Perbandingan ini menggunakan arsitektur dasar yang sama, tetapi model *non-shared* mengganti seluruh *Conv2D* dengan *LocallyConnected2D*. Perubahan ini menghilangkan mekanisme *weight sharing*, sehingga setiap posisi spasial mempunyai bobot sendiri dan dampaknya terlihat langsung pada ukuran model. Pada *test set*, model *shared* memperoleh *macro f1* sebesar 0.7702 dengan 1215766 parameter, sedangkan model *non-shared* hanya memperoleh *macro f1* sebesar 0.0496 dengan 116584390 parameter. Jumlah parameter model *non-shared* berukuran sekitar 95.9 kali lebih besar dibanding dengan model *shared*.

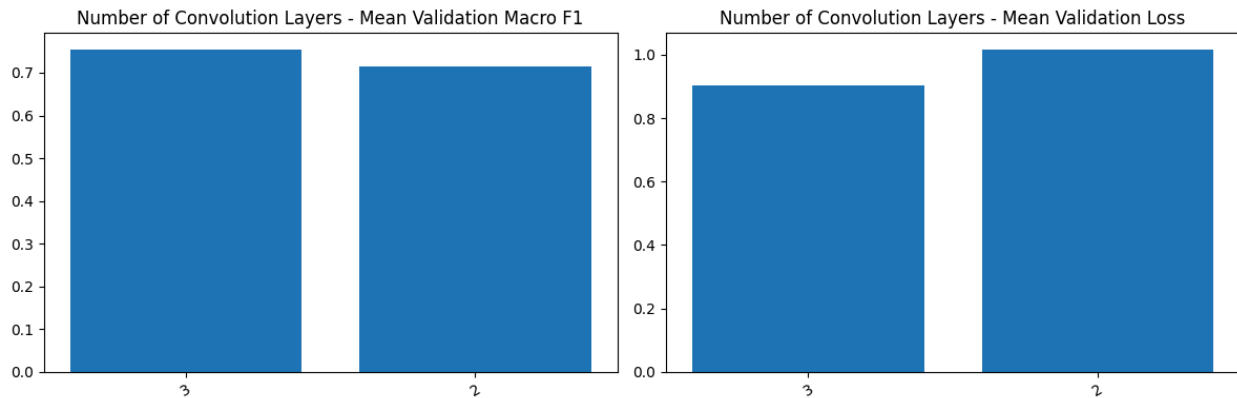
Selisih ini menunjukkan bahwa masalah utama pada model *non-shared* bukan hanya efisiensi memori, tetapi juga efisiensi pembelajaran. Pada *layer* konvolusi biasa,

satu kernel yang sama digunakan ulang di seluruh lokasi gambar, sehingga model dapat mempelajari detektor pola yang berlaku umum, misalnya tepi, tekstur, atau sudut. Pada `LocallyConnected2D`, pola yang sama harus dipelajari ulang untuk banyak lokasi berbeda. Dengan ukuran *input* sebesar 224×224 , pendekatan ini membuat jumlah parameter membesar sangat cepat, sementara jumlah data latih tetap sama. Akibatnya, kapasitas model naik tajam tanpa diimbangi data yang cukup untuk mengestimasi bobot sebanyak itu.

Kurva pada Gambar 3.1.1.1 memperlihatkan perbedaan perilaku *training* pada kedua konfigurasi. Model *shared* masih dapat menurunkan *validation loss* sampai sekitar *epoch* ke-3 dan mencapai *validation macro F1* di angka 0.77. Setelah itu *validation loss* naik kembali, sehingga titik terbaiknya berada di sekitar *epoch* ke-3. Ini menunjukkan bahwa model *shared* benar-benar belajar representasi yang berguna, lalu mulai memasuki fase *overfitting* ringan setelah performa validasi mencapai puncak.

Sebaliknya, pada model *non-shared* proses *training* dihentikan lebih awal karena metrik validasi tidak menunjukkan perbaikan sehingga grafiknya hanya memiliki satu titik dengan *best epoch*-nya adalah *epoch* pertama. Pada titik tersebut, *train accuracy* bernilai sebesar 0.1724, *train macro f1* bernilai sebesar 0.1080, *val accuracy* bernilai sebesar 0.1792, dan *val macro f1* bernilai sebesar 0.0507. Nilai *train* dan *validation* sama-sama rendah, sehingga kegagalannya tidak dapat dijelaskan sebagai *overfitting* saja karena model belum sempat mencapai representasi yang baik bahkan pada data latih. Pada konfigurasi ini, *parameter sharing* bukan hanya lebih hemat, tetapi juga memberi proses optimasi dan generalisasi yang jauh lebih baik.

3.1.2 Pengaruh jumlah layer konvolusi



Gambar 3.1.2.1 Rata-rata *validation macro F1* dan *validation loss* berdasarkan jumlah layer konvolusi

Sumber : arsip penulis

Gambar 3.1.2.1 membandingkan model dengan 2 layer dan 3 layer konvolusi. Rata-rata nilai dari *val macro f1* untuk model 3 layer adalah 0.7551, sedangkan model 2 layer berada pada angka 0.7148. Rata-rata *val loss* juga lebih rendah pada 3 layer, yaitu 0.9041 dibandingkan dengan 2 layer yang memiliki *val loss* sebesar 1.0172. Dengan kata lain, penambahan satu blok konvolusi tidak hanya menaikkan skor klasifikasi, tetapi juga membuat keluaran probabilitas model lebih sesuai terhadap label validasi.

Jika dilihat di dalam masing-masing kelompok, performa 3 layer juga lebih stabil. Pada kelompok 3 layer, rentang nilai *val macro f1* berada di sekitar 0.7405 sampai 0.7765. Pada kelompok 2 layer, rentangnya lebih lebar, yaitu sekitar 0.6876 sampai 0.7479. Ini berarti konfigurasi 3 layer tidak hanya menghasilkan model terbaik, tetapi juga lebih tahan terhadap perubahan filter, kernel, dan *pooling* yang diuji pada eksperimen ini. Model terbaik keseluruhan juga berasal dari kelompok 3 layer, yaitu `cnn_13_f24-24-24_k3x3-3x3-3x3_avg` dengan nilai *val macro f1* sebesar 0.7765.

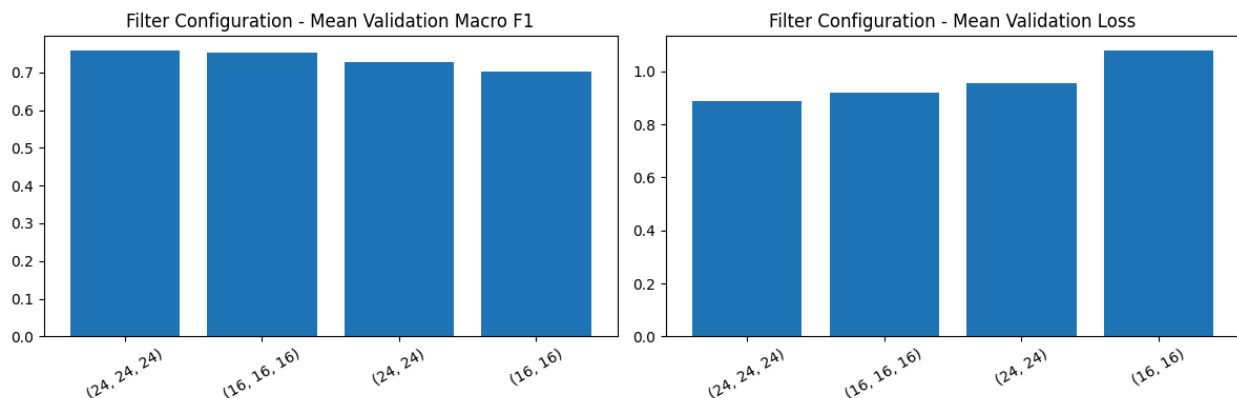
Secara representasional, hasil ini masuk akal karena dua layer konvolusi sudah cukup untuk menangkap pola lokal tingkat rendah dan menengah, tetapi tiga layer memberi satu tahap ekstraksi tambahan sebelum fitur diratakan ke `Flatten` dan

Dense. Tahap tambahan ini membantu model menyusun pola visual yang lebih kompleks, misalnya kombinasi tekstur, kontur, dan struktur objek yang lebih relevan untuk membedakan kelas seperti *buildings*, *street*, dan *sea*. Pada dataset ini, tambahan kedalaman tersebut masih memberi manfaat. Belum terlihat tanda bahwa 3 layer terlalu dalam untuk ukuran eksperimen yang dipakai.

Dari kurva *training* dan *validation loss* per kelompok *layer*, kedua kelompok sama-sama menunjukkan pola umum berupa *training loss* yang terus turun dan *validation loss* yang mencapai minimum pada *epoch* awal lalu naik kembali. Namun, kurva 3 layer biasanya turun ke *validation loss* minimum yang lebih rendah dan mempertahankan *validation macro F1* yang lebih tinggi di sekitar titik terbaiknya. Artinya, model 3 layer tidak menghilangkan risiko *overfitting*, tetapi memberi titik optimum validasi yang lebih baik sebelum *overfitting* mulai dominan.

Pada perbandingan prediksi enam gambar *test* yang sama, model terbaik 2 layer dan 3 layer sama-sama benar pada 4/6 gambar (berdasarkan hasil dari artefak *training*), tetapi kesalahannya berbeda. Model 2 layer salah pada gambar pertama dan keenam, sedangkan model 3 layer salah pada gambar kedua dan keenam. Contoh ini menunjukkan bahwa keunggulan 3 layer tidak selalu muncul pada setiap sampel individual, tetapi terlihat lebih jelas ketika evaluasi dilakukan pada seluruh *validation* dan *test split* melalui *macro F1 score*.

3.1.3 Pengaruh banyak filter per layer konvolusi



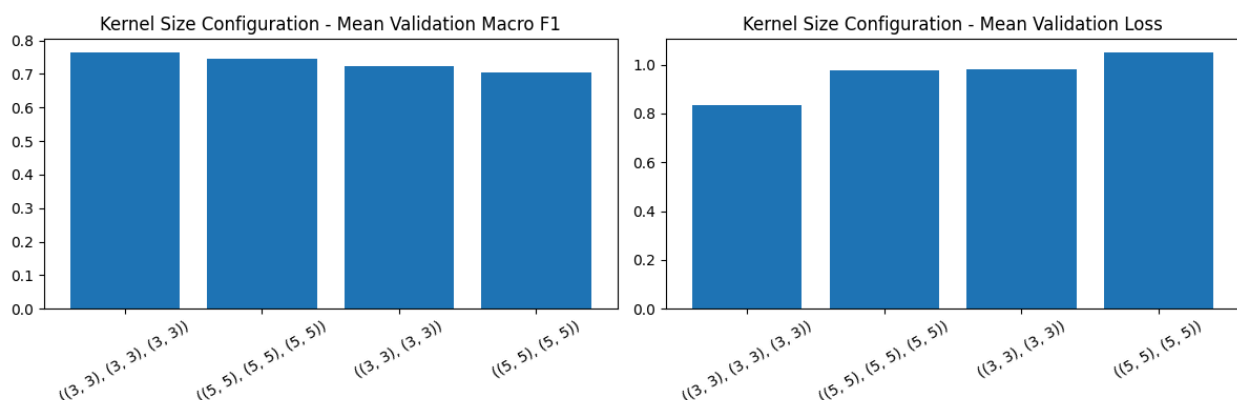
Gambar 3.1.3.1 Rata-rata *validation macro F1* dan *validation loss* berdasarkan konfigurasi jumlah filter konvolusi
Sumber : arsip penulis

Gambar 3.1.3.1 membandingkan hasil validasi untuk empat konfigurasi jumlah filter yang diuji. Konfigurasi dengan 24 filter per *layer* memberikan hasil lebih baik daripada 16 filter per *layer*. Untuk model 2 *layer*, konfigurasi (24, 24) memiliki rata-rata nilai *val macro f1* sebesar 0.7270, sedangkan (16, 16) berada pada nilai 0.7026. Untuk model 3 *layer*, konfigurasi (24, 24, 24) mencapai nilai rata-rata 0.7586, sedikit di atas konfigurasi (16, 16, 16) yang mencapai nilai 0.7515.

Pola yang sama terlihat pada *validation loss*. Konfigurasi dengan 24 filter konsisten memberi *loss* yang lebih rendah pada 2 *layer* maupun 3 *layer*. Kenaikannya tidak sebesar pengaruh jumlah *layer*, tetapi cukup konsisten. Pada eksperimen ini, 24 filter per *layer* memberi representasi fitur yang lebih baik daripada 16 filter per *layer*.

Berdasarkan artefak *training*, pada enam gambar *test* yang sama, perbedaan prediksi antar konfigurasi filter terlihat lebih besar. Model terbaik (16, 16) salah pada seluruh 6/6 gambar sampel, sedangkan model terbaik (16, 16, 16) benar pada 5/6 gambar. Model terbaik (24, 24) dan (24, 24, 24) sama-sama benar pada 4/6 gambar. Hasil ini sejalan dengan kurva *loss* yang menunjukkan konfigurasi 3 *layer* lebih stabil daripada 2 *layer*, terutama saat jumlah filter lebih kecil.

3.1.4 Pengaruh ukuran filter per layer konvolusi



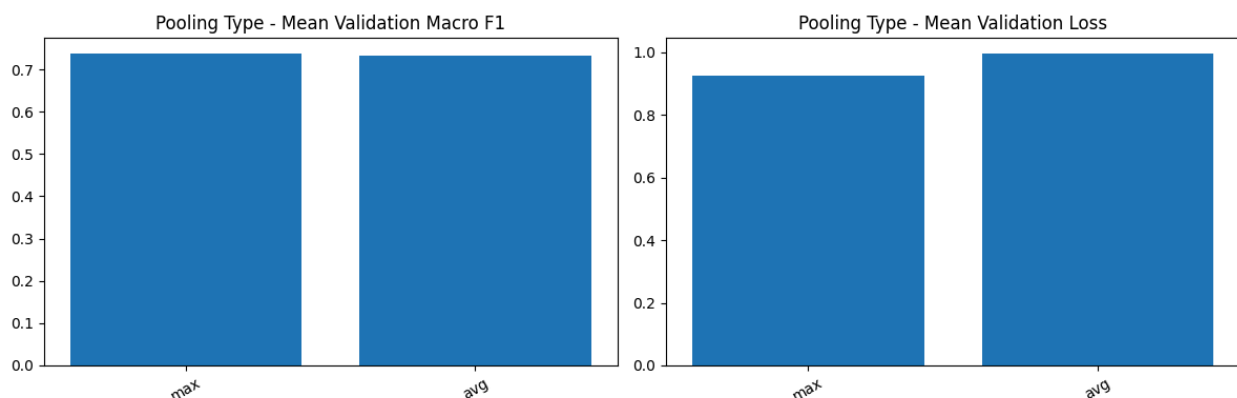
Gambar 3.1.4.1 Rata-rata *validation macro F1* dan *validation loss* berdasarkan ukuran kernel
Sumber : arsip penulis

Perbandingan kernel 3 x 3 dan 5 x 5 menunjukkan bahwa kernel 3 x 3 memberi hasil lebih baik pada kedua jumlah *layer*. Untuk model 2 *layer*, konfigurasi 3 x 3 memiliki rata-rata nilai *val macro f1* sebesar 0.7245, sedangkan 5 x 5 berada pada nilai 0.7052. Untuk model 3 *layer*, konfigurasi 3 x 3 mencapai nilai sebesar 0.7654, sedangkan untuk ukuran 5 x 5 berada pada nilai 0.7447.

Validation loss juga lebih rendah pada konfigurasi 3 x 3. Selisihnya cukup jelas pada model 3 *layer*, yaitu 0.8331 untuk 3 x 3 dan 0.9751 untuk 5 x 5. Pada arsitektur ini, kernel 3 x 3 lebih efisien dalam menangkap pola lokal tanpa menambah parameter sebanyak kernel 5 x 5.

Berdasarkan artefak *training*, pada enam gambar test yang sama, model terbaik 3 x 3 dengan 2 *layer* dan 3 *layer* sama-sama benar pada 4/6 gambar. Untuk kernel 5 x 5, model 2 *layer* benar pada 3/6 gambar, sedangkan model 3 *layer* benar pada 5/6 gambar. Walaupun hasil ini menunjukkan bahwa kernel 5 x 5 masih bisa memberikan prediksi yang baik pada model tertentu, kurva *loss* dan rata-rata *macro F1 score* tetap menunjukkan bahwa konfigurasi 3 x 3 lebih konsisten pada keseluruhan eksperimen.

3.1.5 Pengaruh jenis pooling layer



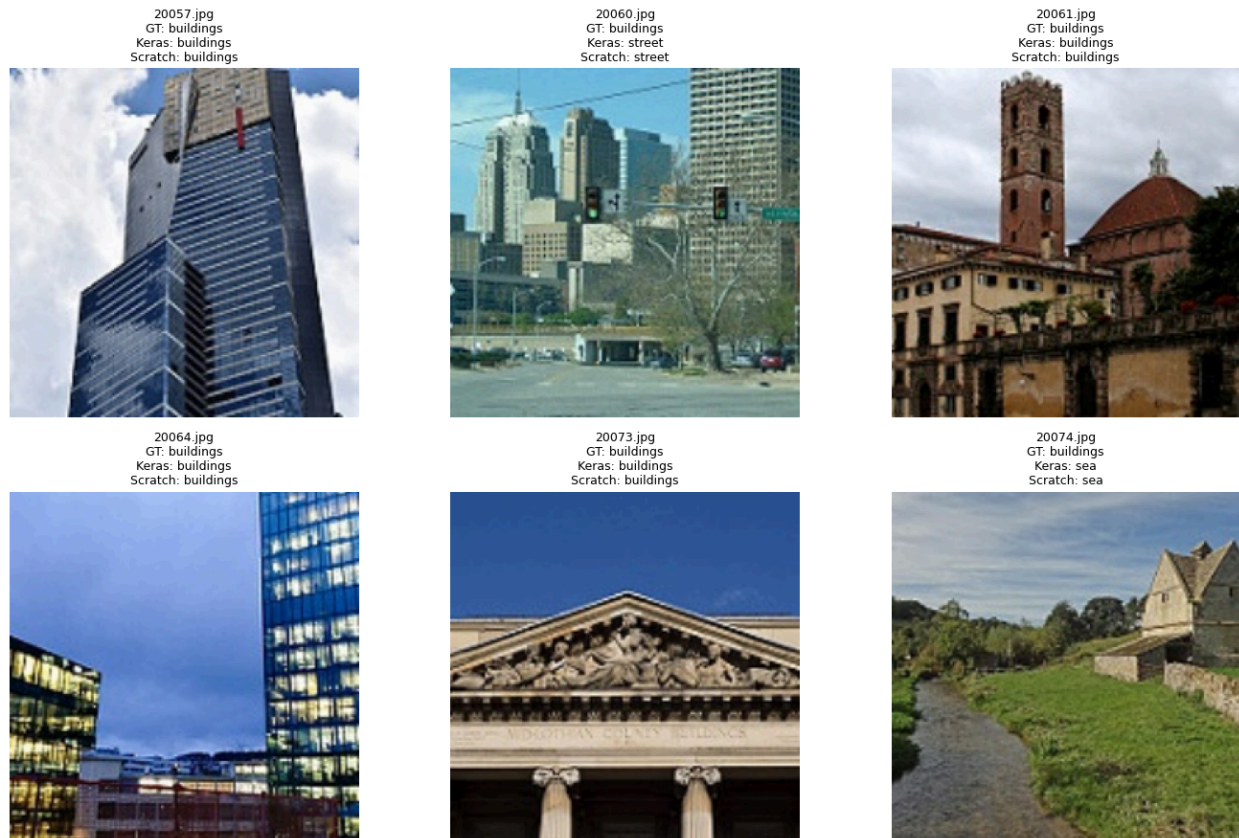
Gambar 3.1.5.1 Rata-rata *validation macro F1* dan *validation loss* berdasarkan jenis pooling
Sumber : arsip penulis

Secara rata-rata, perbedaan antara *max pooling* dan *average pooling* tidak sebesar faktor lain. Jika seluruh konfigurasi digabung, *max pooling* memiliki rata-rata nilai *val macro f1* sebesar 0.7373, sedikit di atas *average pooling* yang berada di angka 0.7326. Rata-rata *val loss* untuk *max pooling* juga lebih rendah, yaitu senilai 0.9262 dibanding 0.9951.

Namun model terbaik keseluruhan justru menggunakan *average pooling*, yaitu `cnn_13_f24-24-24_k3x3-3x3-3x3_avg` dengan nilai *val macro f1* sebesar 0.7765. Pada model 2 layer, *max pooling* lebih unggul atas *average pooling*. Pada model 3 layer, selisih keduanya hampir sama, dan *average pooling* sedikit lebih baik. Jadi, pengaruh jenis *pooling* bergantung pada kombinasi *hyperparameter* lain dan pengaruhnya tidak sekuat jumlah layer, filter, atau ukuran kernel.

Berdasarkan artefak *training*, pada enam gambar test yang sama, model terbaik *average pooling* benar pada 4/6 gambar, sedangkan model terbaik *max pooling* benar pada 3/6 gambar. Prediksi kedua model sama pada lima gambar dan hanya berbeda pada satu gambar. Dari kurva *loss*, keduanya memiliki pola *training* yang mirip, tetapi model *average pooling* memberi *validation loss* minimum dan nilai *macro F1-score* yang sedikit lebih baik pada konfigurasi terbaik.

3.1.6 Perbandingan Keras vs from scratch



Gambar 3.1.6.1 Contoh prediksi Keras dan model *from scratch* pada data uji
Sumber : arsip penulis

Perbandingan ini dilakukan pada model *shared* terbaik setelah bobot Keras dikonversi ke implementasi *scratch*. Pada *test set*, model Keras memperoleh nilai *macro f1* sebesar 0.7702, sedangkan model *scratch* memperoleh nilai *macro f1* sebesar 0.7702 dengan selisih absolut sekitar 0.00007. Pada level metrik klasifikasi, implementasi *forward propagation scratch* sudah konsisten dengan model Keras.

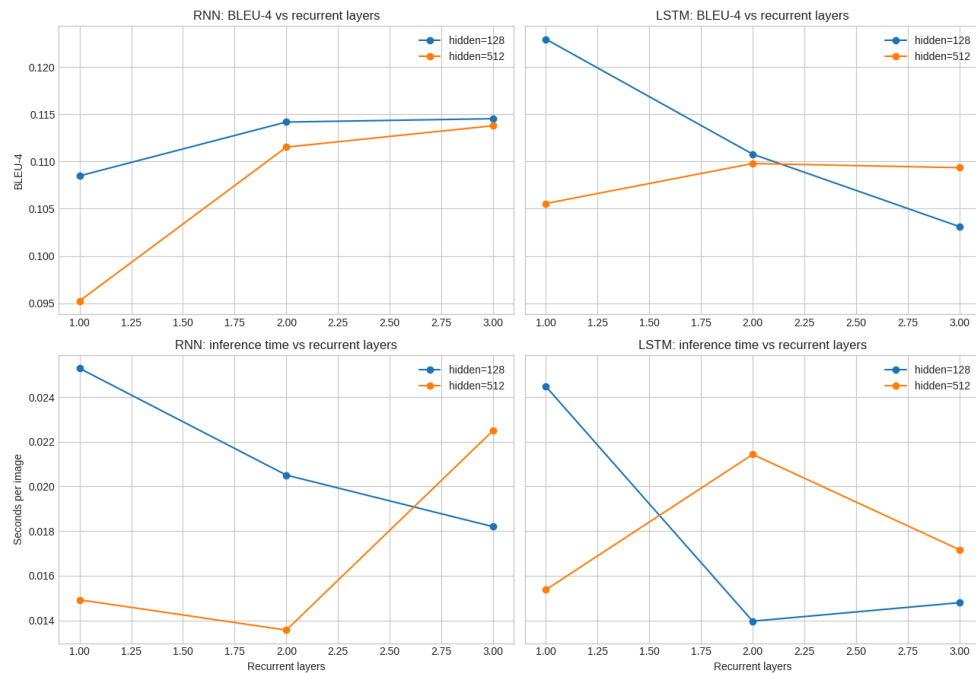
Perbandingan probabilitas juga mendukung hasil tersebut. Nilai rata-rata *probability gap* antara keluaran Keras dan *scratch* adalah 0.00455, sedangkan nilai untuk *max probability gap*-nya adalah 0.2316. Selisih maksimum menunjukkan ada beberapa sampel dengan distribusi probabilitas yang tidak identik, tetapi pada contoh di Gambar 3.1.6.1 prediksi kelas akhir tetap sama. Secara praktis, implementasi *scratch* dapat mereproduksi perilaku model Keras dengan baik.

3.2 Hasil Pengujian Simple RNN dan LSTM

3.2.1 Perbandingan jumlah layer & hidden state

model	hidden_units	bleu4		meteor		seconds_per_image	
		128	512	128	512	128	512
	recurrent_layers						
LSTM	1	0.1230	0.1056	0.3221	0.3158	0.0245	0.0154
	2	0.1108	0.1098	0.3208	0.3211	0.0140	0.0214
	3	0.1031	0.1093	0.3131	0.3114	0.0148	0.0172
RNN	1	0.1085	0.0952	0.3081	0.3127	0.0253	0.0149
	2	0.1142	0.1115	0.3275	0.3108	0.0205	0.0136
	3	0.1145	0.1138	0.3163	0.3209	0.0182	0.0225

Gambar 3.2.1.1 Hasil inference untuk semua konfigurasi layer & hidden state
Sumber : arsip penulis



Gambar 3.2.1.1b Kurva BLEU- 4 dan inference time untuk semua konfigurasi layer & hidden state
Sumber : arsip penulis

Pada LSTM, hasil terbaik justru diperoleh oleh konfigurasi 1 layer hidden 128 dengan BLEU-4 0.1230. Penambahan layer menurunkan BLEU-4, terutama pada

hidden 128. Ini menunjukkan bahwa LSTM 1 layer sudah cukup untuk dataset Flickr8k, sedangkan model yang lebih dalam tidak selalu memberi generalisasi lebih baik.

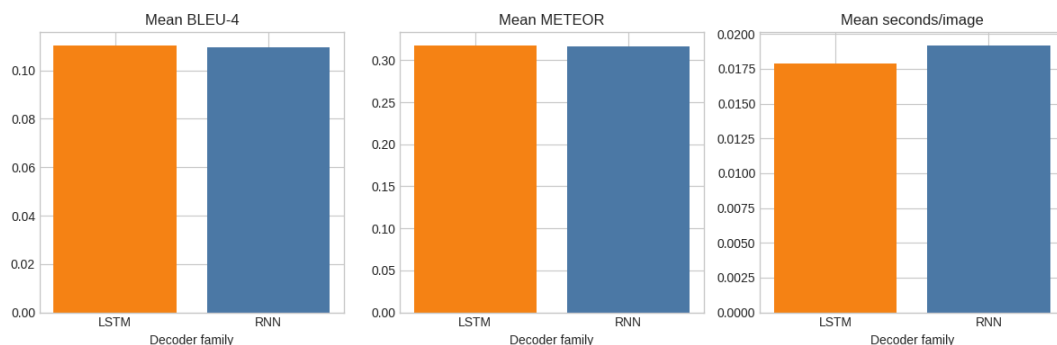
Hidden state 512 juga tidak memberikan peningkatan yang konsisten. Walaupun kapasitas model lebih besar, hasilnya sering lebih rendah dibanding hidden 128. Hal ini kemungkinan disebabkan oleh overfitting, karena dataset relatif kecil dan caption relatif pendek.

Dari grafik waktu inferensi, waktu tidak sepenuhnya naik linear terhadap jumlah layer karena dipengaruhi runtime dan panjang caption aktual. Namun, secara umum model yang lebih besar memiliki potensi komputasi lebih mahal.

3.2.2 Perbandingan RNN vs LSTM

	model	mean_bleu4	best_bleu4	mean_meteor	best_meteor	mean_seconds_per_image
0	LSTM	0.1103	0.1230	0.3174	0.3221	0.0179
1	RNN	0.1096	0.1145	0.3160	0.3275	0.0192

Gambar 3.2.2.1 Hasil inference untuk model RNN dan LSTM
Sumber : arsip penulis



Gambar 3.2.2.1b Graf mean BLEU-4, meteor, dan sec/img untuk model RNN dan LSTM
Sumber : arsip penulis

Secara teori, LSTM lebih tahan terhadap vanishing gradient karena memiliki input gate, forget gate, output gate, dan cell state yang membantu mempertahankan informasi jangka panjang. Hasil eksperimen mendukung sebagian teori ini: LSTM terbaik memberi BLEU-4 tertinggi pada konfigurasi utama pre-inject. Namun RNN

tetap kompetitif karena caption Flickr8k relatif pendek dan banyak pola caption berupa frasa visual sederhana seperti subjek-aksi-objek.

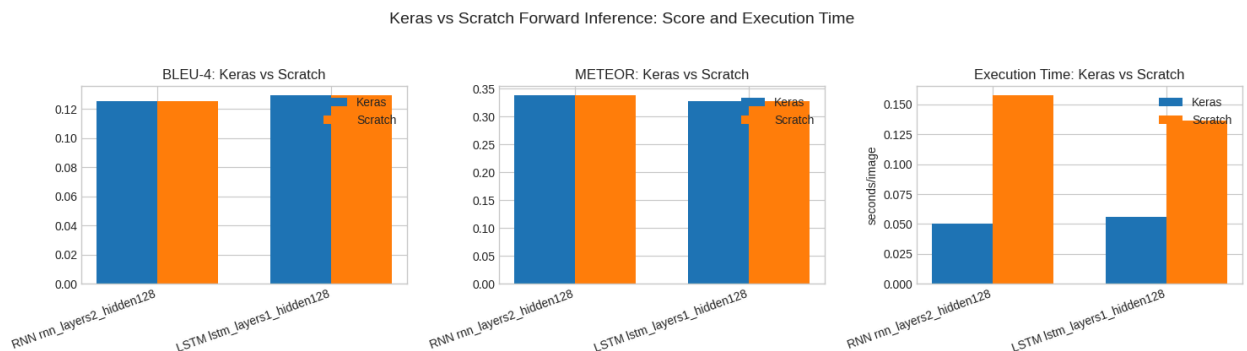
Perbandingan ini sebaiknya ditulis sebagai trade-off, bukan klaim mutlak. LSTM unggul pada model terbaik berbasis BLEU-4, sedangkan RNN rnn_layers2_hidden128 memberikan METEOR terbaik pada pre-inject dan waktu inference yang lebih rendah dibanding LSTM terbaik BLEU-4. Jadi, jika prioritasnya kualitas n-gram, pilih LSTM; jika prioritasnya METEOR dan efisiensi, RNN terpilih masih masuk akal.

3.2.3 Perbandingan Keras vs from scratch

	model	images	exact_matches	match_rate	keras_bleu4	scratch_bleu4	keras_meteor	scratch_meteor	keras_seconds_per_image	scratch_seconds_per_image
0	RNN rnn_layers2_hidden128	500	500	1.0	0.1250	0.1250	0.3375	0.3375	0.0503	0.1573
1	LSTM lstm_layers1_hidden128	500	500	1.0	0.1291	0.1291	0.3283	0.3283	0.0561	0.1362

Gambar 3.2.3.1 Perbandingan Inference Keras vs *from scratch*

Sumber : arsip penulis



Gambar 3.2.3.1b Perbandingan Inference Keras vs *from scratch*

Sumber : arsip penulis

Perbandingan Keras vs NumPy scratch menunjukkan exact sentence match seratus persen untuk kedua model terpilih pada 500 gambar test. RNN menghasilkan rata-rata BLEU-4 0.1250 untuk kedua keras dan *from scratch*, sedangkan LSTM menghasilkan nilai 0.1291 untuk kedua-nya juga. Selisih score nol pada artifact terbaru, sehingga implementasi forward propagation scratch dapat dianggap konsisten dengan model Keras.

Dari sisi waktu pada 500 gambar test, Keras lebih cepat daripada implementasi *from scratch* dengan RNN Keras memakai waktu 0.0315 detik per gambar dan *from scratch* memakai waktu 0.0652 detik per gambar, sedangkan LSTM Keras memakai waktu 0.0293 detik per gambar dan *from scratch* memakai waktu 0.0769 detik per gambar. Perbedaan ini wajar karena Keras memakai operasi tensor yang lebih teroptimasi dan batching yang lebih efisien, sedangkan *scratch* NumPy memprioritaskan transparansi forward propagation.

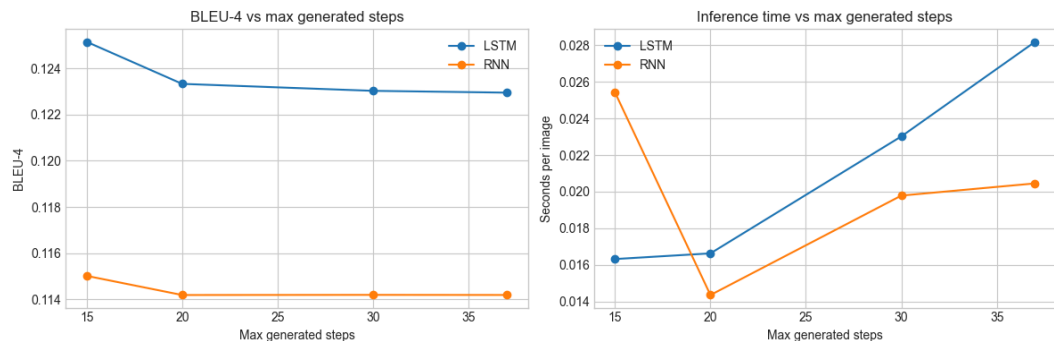
Forward propagation *from scratch* yang dibuat sudah valid karena menghasilkan output identik dengan Keras. Perbedaannya ada pada efisiensi: Keras lebih cepat, sedangkan *scratch* lebih berguna untuk menunjukkan correctness dan pemahaman detail perhitungan forward propagation.

3.2.4 Pengaruh panjang maksimum caption terhadap BLEU-4

	model	experiment_id	max_generated_steps	images	bleu4	meteor	seconds_per_image
0	RNN	rnn_layers2_hidden128	15	1000	0.115009	0.326666	0.025422
1	RNN	rnn_layers2_hidden128	20	1000	0.114179	0.327275	0.014352
2	RNN	rnn_layers2_hidden128	30	1000	0.114186	0.327387	0.019785
3	RNN	rnn_layers2_hidden128	37	1000	0.114181	0.327470	0.020450
4	LSTM	lstm_layers1_hidden128	15	1000	0.125160	0.322801	0.016318
5	LSTM	lstm_layers1_hidden128	20	1000	0.123338	0.322252	0.016628
6	LSTM	lstm_layers1_hidden128	30	1000	0.123036	0.322091	0.023019
7	LSTM	lstm_layers1_hidden128	37	1000	0.122957	0.322081	0.028183

Gambar 3.2.4.1 Perbandingan hasil Inference dengan semua konfigurasi panjang maksimum caption

Sumber : arsip penulis



Gambar 3.2.4.1b Grafik BLEU-4 dan inference time terhadap panjang maksimum caption
Sumber : arsip penulis

Pada LSTM terpilih, batas 15 timestep memberi BLEU-4 tertinggi dengan nilai 0.1252 dibanding 20, 30, dan 37 timestep. Pada RNN terpilih, batas 15 juga sedikit lebih baik dengan nilai 0.1150 dibanding batas yang lebih panjang. Hal ini menunjukkan bahwa memberi ruang caption lebih panjang tidak otomatis memperbaiki kualitas berdasarkan BLEU-4. Penyebab utamanya adalah BLEU-4 menghukum token tambahan yang tidak cocok dengan ground truth. Batas yang terlalu panjang memberi decoder kesempatan menghasilkan kata generik atau repetitif setelah inti caption selesai.

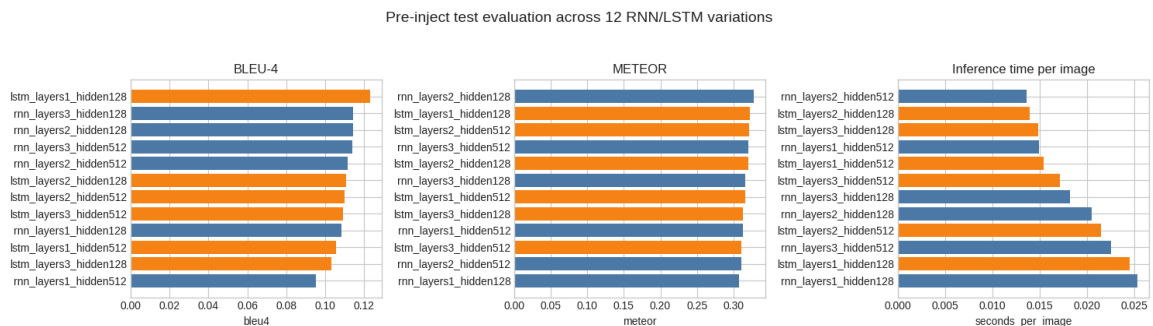
Karena itu, panjang maksimum caption tidak sebaiknya ditentukan hanya dengan mengikuti caption terpanjang pada data training. Walaupun data training memiliki caption dengan panjang hingga 37 token, hasil evaluasi menunjukkan bahwa batas yang lebih pendek, yaitu 15 timestep, justru menghasilkan BLEU-4 terbaik pada model LSTM maupun RNN. Hal ini menunjukkan bahwa `max_caption_length` berperan sebagai hyperparameter decoding yang mempengaruhi kualitas output akhir model.

Dengan demikian, pemilihan `max_caption_length` perlu dilakukan melalui eksperimen dan evaluasi, bukan hanya berdasarkan panjang maksimum caption pada dataset. Dalam kasus ini, batas 15 timestep menjadi pilihan yang lebih optimal karena mampu menjaga caption tetap padat, mengurangi risiko token tambahan yang tidak relevan, dan menghasilkan skor BLEU-4 tertinggi dibandingkan batas yang lebih panjang.

3.2.5 Perbandingan 12 variasi konfigurasi decoder Pre-Inject

	rank_bleu4	experiment_id	model	recurrent_layers	hidden_units	images	bleu4	meteor	seconds_per_image
0	1	lstm_layers1_hidden128	LSTM	1	128	1000	0.122957	0.322081	0.024476
1	2	rnn_layers3_hidden128	RNN	3	128	1000	0.114533	0.316314	0.018194
2	3	rnn_layers2_hidden128	RNN	2	128	1000	0.114181	0.327470	0.020506
3	4	rnn_layers3_hidden512	RNN	3	512	1000	0.113789	0.320883	0.022503
4	5	rnn_layers2_hidden512	RNN	2	512	1000	0.111525	0.310792	0.013562
5	6	lstm_layers2_hidden128	LSTM	2	128	1000	0.110781	0.320847	0.013961
6	7	lstm_layers2_hidden512	LSTM	2	512	1000	0.109788	0.321058	0.021433
7	8	lstm_layers3_hidden512	LSTM	3	512	1000	0.109349	0.311399	0.017163
8	9	rnn_layers1_hidden128	RNN	1	128	1000	0.108479	0.308107	0.025289
9	10	lstm_layers1_hidden512	LSTM	1	512	1000	0.105557	0.315834	0.015371
10	11	lstm_layers3_hidden128	LSTM	3	128	1000	0.103104	0.313073	0.014790
11	12	rnn_layers1_hidden512	RNN	1	512	1000	0.095238	0.312683	0.014917

Gambar 3.2.5.1 Perbandingan hasil Inference dengan semua konfigurasi variasi decoder Pre-Inject
Sumber : arsip penulis



Gambar 3.2.5.1b Grafik BLEU-4, METEOR, waktu inference hasil Inference dengan semua konfigurasi variasi decoder Pre-Inject
Sumber : arsip penulis

Dari 12 variasi pre-inject, model dengan BLEU-4 tertinggi adalah variasi lstm_layers1_hidden128 dengan BLEU-4 bernilai 0.1230, METEOR 0.3221, dan waktu inference 0.0245 detik per gambar. Model dengan METEOR tertinggi adalah rnn_layers2_hidden128 dengan METEOR bernilai 0.3275, BLEU-4 0.1142, dan waktu 0.0205 detik per gambar. Ini menunjukkan bahwa pemilihan model terbaik bergantung pada prioritas metrik: BLEU-4 lebih mengutamakan kecocokan n-gram panjang, sementara METEOR lebih toleran terhadap variasi bentuk kata dan overlap unigram.

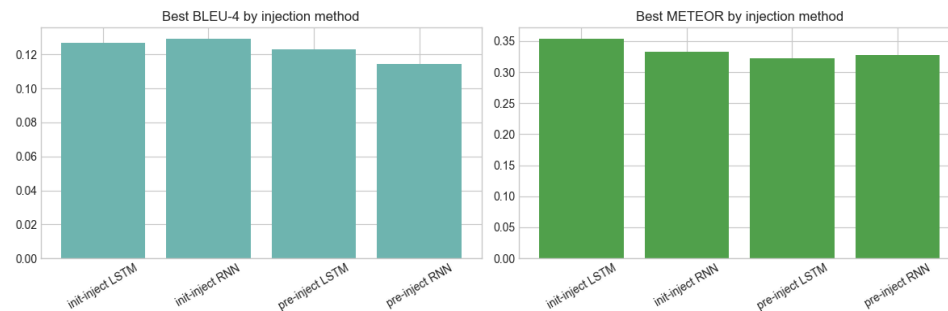
Secara rata-rata, LSTM sedikit unggul dari RNN pada pre-inject dengan BLEU-4 0.1103 vs 0.1096, METEOR 0.3174 vs 0.3160 dan sedikit lebih cepat pada rata-rata waktu dengan nilai 0.0179 vs 0.0192 detik per gambar. Akan tetapi, selisih antara kedua model kecil sehingga kesimpulan utama sebaiknya tidak hanya menyatakan LSTM dominan, tetapi bahwa LSTM terbaik unggul pada BLEU-4 sementara RNN tertentu tetap kompetitif, terutama pada METEOR dan trade-off waktu.

3.2.6 Perbandingan Pre-Inject vs Init-Inject

	injection_method	model	mean_bleu4	best_bleu4	mean_meteor	best_meteor	mean_seconds_per_image
0	init-inject	LSTM	0.1222	0.1267	0.3430	0.3540	0.0228
1	init-inject	RNN	0.1147	0.1293	0.3214	0.3329	0.0280
2	pre-inject	LSTM	0.1103	0.1230	0.3174	0.3221	0.0179
3	pre-inject	RNN	0.1096	0.1145	0.3160	0.3275	0.0192

Gambar 3.2.6.1 Perbandingan hasil Inference Pre-Inject vs Init-Inject

Sumber : arsip penulis



Gambar 3.2.6.1b Grafik BLEU-4 & METEOR hasil Inference

Pre-Inject vs Init-Inject

Sumber : arsip penulis

Berdasarkan tabel, init-inject menghasilkan performa yang lebih baik dibanding pre-inject pada eksperimen ini. BLEU-4 terbaik secara keseluruhan diperoleh oleh initinject_rnn_layers3_hidden128 dengan skor 0.1293, lebih tinggi dibanding model pre-inject terbaik yaitu lstm_layers1_hidden128 dengan BLEU-4 0.1230. Untuk METEOR, init-inject juga unggul. Skor METEOR terbaik diperoleh oleh initinject_lstm_layers3_hidden512 dengan nilai 0.3540, sedangkan pre-inject terbaik hanya mencapai 0.3275.

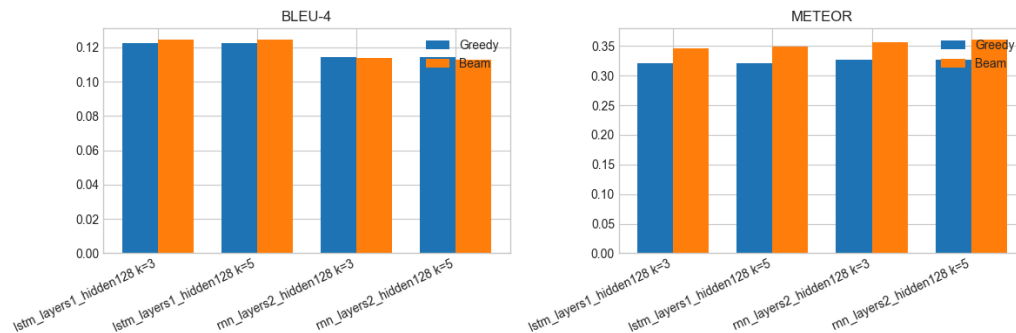
Hasil ini menunjukkan bahwa cara memasukkan informasi visual ke decoder berpengaruh terhadap kualitas caption. Pada init-inject, feature gambar langsung membentuk state awal decoder, sehingga proses generasi caption sejak timestep pertama sudah dikondisikan oleh informasi visual. Pada LSTM, conditioning ini lebih kuat karena gambar memengaruhi hidden state dan cell state.

3.2.7 Perbandingan Greedy vs Beam Search

	experiment_id	beam_width	images	greedy_bleu4	beam_bleu4	greedy_meteor	beam_meteor
0	lstm_layers1_hidden128	3	1000	0.1224	0.1248	0.3216	0.3463
1	lstm_layers1_hidden128	5	1000	0.1224	0.1247	0.3216	0.3488
2	rnn_layers2_hidden128	3	1000	0.1142	0.1136	0.3273	0.3562
3	rnn_layers2_hidden128	5	1000	0.1142	0.1126	0.3273	0.3615

Gambar 3.2.7.1 Perbandingan hasil Inference Greedy vs Beam Search

Sumber : arsip penulis



Gambar 3.2.7.1b Grafik BLEU-4 & METEOR hasil

Inference Greedy vs Beam Search

Sumber : arsip penulis

Pada LSTM, beam search sedikit meningkatkan BLEU-4 dari 0.1224 menjadi sekitar 0.1247-0.1248. METEOR juga naik cukup jelas, dari 0.3216 menjadi 0.3463 pada beam width 3 dan 0.3488 pada beam width 5. Ini menunjukkan bahwa beam search membantu LSTM menemukan caption dengan overlap kata yang lebih baik terhadap ground truth.

Pada RNN, hasilnya berbeda. Beam search meningkatkan METEOR secara signifikan, dari 0.3273 menjadi 0.3562 pada beam width 3 dan 0.3615 pada beam width 5. Namun, BLEU-4 justru sedikit turun dari 0.1142 menjadi 0.1136 dan 0.1126.

Hal ini menunjukkan bahwa caption hasil beam search memiliki overlap kata yang lebih baik, tetapi susunan 4-gram-nya tidak selalu lebih cocok dengan ground truth.

Perbedaan BLEU-4 dan METEOR ini terjadi karena beam search memilih caption berdasarkan probabilitas model, bukan berdasarkan metrik evaluasi. Caption dengan probabilitas tinggi dapat lebih panjang atau lebih umum. Caption seperti ini dapat meningkatkan METEOR karena memiliki lebih banyak overlap unigram, tetapi dapat menurunkan BLEU-4 karena precision 4-gram menjadi lebih rendah.

04 — Kesimpulan dan Saran

└ 4.1 Kesimpulan

Model berbasis CNN, RNN, dan LSTM berhasil diimplementasikan untuk dua persoalan utama, yaitu klasifikasi citra dan image captioning. Pada bagian CNN, model berhasil digunakan untuk melakukan klasifikasi gambar pada dataset Intel Image Classification. Implementasi juga mencakup variasi arsitektur seperti jumlah layer konvolusi, jumlah filter, ukuran kernel, jenis pooling, serta perbandingan antara shared parameter menggunakan Conv2D dan non-shared parameter menggunakan LocallyConnected2D.

Implementasi forward propagation CNN from scratch juga berhasil mereplikasi proses prediksi dari model Keras dengan membaca bobot hasil training dan menjalankan operasi layer secara eksplisit menggunakan NumPy. Hal ini menunjukkan bahwa proses inferensi CNN dapat dipahami dan dihitung langsung dari operasi dasar seperti konvolusi, pooling, flatten, dense, dan softmax tanpa bergantung sepenuhnya pada abstraksi Keras.

Pada bagian RNN dan LSTM, sistem image captioning berhasil dibangun menggunakan arsitektur encoder-decoder. CNN InceptionV3 digunakan sebagai encoder untuk mengekstraksi fitur visual gambar, sedangkan Simple RNN dan LSTM digunakan sebagai decoder untuk menghasilkan caption. Caption mentah diproses melalui tahap cleaning, tokenisasi, pembentukan vocabulary, padding, dan teacher forcing agar dapat digunakan sebagai data pelatihan decoder.

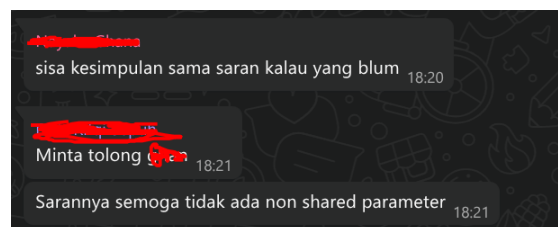
Hasil pengujian menunjukkan bahwa LSTM lebih sesuai untuk tugas image captioning dibandingkan Simple RNN karena memiliki cell state dan mekanisme gate yang lebih baik dalam mempertahankan informasi jangka panjang. Selain itu, variasi strategi injeksi fitur gambar, yaitu pre-inject dan init-inject, memberikan pendekatan berbeda dalam memasukkan konteks visual ke decoder. Pre-inject memperlakukan fitur gambar sebagai timestep pertama, sedangkan init-inject menggunakan fitur gambar sebagai initial state recurrent layer.

Implementasi forward propagation from scratch pada decoder RNN/LSTM juga berhasil dibuat untuk mereplikasi proses inferensi dari model Keras. Selain itu, backward propagation from scratch pada RNN/LSTM berhasil diimplementasikan dengan menghitung gradient secara manual untuk dense output layer, recurrent layer, image projection, dan embedding. Verifikasi menggunakan gradient checking dan perbandingan dengan Keras GradientTape menunjukkan bahwa implementasi from scratch sudah mengikuti persamaan matematis yang sesuai.

— 4.2 Saran

Berdasarkan hasil pengujian, beberapa saran untuk pengembangan lebih lanjut adalah sebagai berikut:

1. Pada bagian CNN, eksperimen juga dapat dikembangkan dengan arsitektur yang lebih kompleks atau pretrained model seperti ResNet, EfficientNet, atau MobileNet. Dengan demikian, performa klasifikasi dapat dibandingkan antara model CNN sederhana yang dibuat sendiri dan model pretrained yang lebih kuat.
2. Pada decoder RNN/LSTM, eksperimen dapat diperluas dengan variasi embedding dimension, dropout pada recurrent layer, optimizer yang berbeda, serta strategi decoding lain. Beam search juga dapat diuji dengan beberapa nilai beam width untuk melihat pengaruhnya terhadap kualitas caption dan waktu inferensi.
3. evaluasi image captioning sebaiknya tidak hanya mengandalkan metrik otomatis seperti BLEU-4 atau METEOR, tetapi juga dilengkapi dengan evaluasi kualitatif. Hal ini penting karena caption yang baik secara makna belum tentu selalu memperoleh skor metrik yang tinggi.
4. Sama ini:



└ 4.3 Komentar

~Fariz~

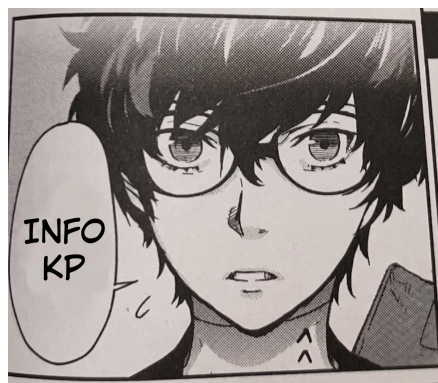


~Jibril~



“Easier to make than one might think”

~Ghana~



“Aku juga mau KP ❤️”

Pembagian Tugas |

Github Repository: <https://github.com/Fariz36/Tubes2-ML>

NIM	Tugas
13523069	Berkontribusi pada pembuatan RNN-LSTM, laporan, dan notebook
13523085	Berkontribusi pada pembuatan RNN-LSTM, laporan, dan notebook
13523090	Berkontribusi pada pembuatan CNN, laporan, dan notebook

Task List Spesifikasi Wajib

Task List	Dikerjakan	Tidak Dikerjakan
[CNN] Utility image processing dan feature extraction untuk dataset gambar	[✓]	[]
[CNN] Implementasi forward propagation CNN from scratch yang dapat load bobot Keras	[✓]	[]
[CNN] Training model CNN Keras untuk klasifikasi Intel Image Classification	[✓]	[]
[CNN] Evaluasi dan analisis CNN: macro F1-score, Keras vs scratch, shared vs non-shared, grafik loss, dan jumlah parameter	[✓]	[]

[RNN/LSTM] Feature extraction Flickr8k dengan CNN encoder frozen dan preprocessing caption	[✓]	[]
[RNN/LSTM] Implementasi decoder Keras pre-inject untuk SimpleRNN dan LSTM	[✓]	[]
[RNN/LSTM] Training 12 variasi decoder: 6 SimpleRNN dan 6 LSTM	[✓]	[]
[RNN/LSTM] Implementasi forward propagation decoder from scratch untuk RNN dan LSTM	[✓]	[]
[RNN/LSTM] Pipeline image captioning from scratch dari raw image sampai caption	[✓]	[]
[RNN/LSTM] Evaluasi dan analisis RNN/LSTM: BLEU-4, METEOR, waktu eksekusi, Keras vs scratch, RNN vs LSTM, grafik loss, contoh kualitatif, dan panjang maksimum caption	[✓]	[]

Task List Spesifikasi Bonus

Task List	Dikerjakan	Tidak Dikerjakan
[CNN] Visualisasi intermediate feature maps dari layer konvolusi	[✓]	[]
[CNN] Grad-CAM untuk region gambar yang paling berpengaruh terhadap prediksi	[✓]	[]
[RNN/LSTM] Implementasi image captioning init-inject sebagai alternatif pre-inject dan membandingkan hasilnya	[✓]	[]

[RNN/LSTM] Implementasi beam search decoder dengan k = 3 atau k = 5 dan membandingkan hasilnya	[✓]	[]
[Semua] Batch inference untuk seluruh forward propagation from scratch dengan batch_size	[✓]	[]
[Semua] Backward propagation from scratch untuk seluruh layer yang digunakan	[✓] Hanya di RNN/LSTM	[]

Referensi |

- [1] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Long short-term memory (LSTM). Dive into Deep Learning. Diakses pada 30 April 2026 dari https://d2l.ai/chapter_recurrent-modern/lstm.html
- [2] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Deep recurrent neural networks. Dive into Deep Learning. Diakses pada 30 April 2026 dari https://d2l.ai/chapter_recurrent-modern/deep-rnn.html
- [3] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Bidirectional recurrent neural networks. Dive into Deep Learning. Diakses pada 30 April 2026 dari https://d2l.ai/chapter_recurrent-modern/bi-rnn.html
- [4] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Recurrent neural networks. Dive into Deep Learning. Diakses pada 30 April 2026 dari https://d2l.ai/chapter_recurrent-neural-networks/index.html
- [5] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Convolutional neural networks. Dive into Deep Learning. Diakses pada 30 April 2026 dari https://d2l.ai/chapter_convolutional-neural-networks/index.html
- [6] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Image captioning. Dive into Deep Learning. Diakses pada 30 April 2026 dari https://d2l.ai/chapter_computer-vision/image-captioning.html
- [7] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Beam search. Dive into Deep Learning. Diakses pada 1 Mei 2026 dari https://d2l.ai/chapter_recurrent-modern/beam-search.html
- [8] Vinyals, O., Toshev, A., Bengio, S., & Erhan, D. (2015). Show and tell: A neural image caption generator. arXiv. Diakses pada 30 April 2026 dari <https://arxiv.org/abs/1411.4555>
- [9] Tanti, M., Gatt, A., & Camilleri, K. P. (2017). Where to put the image in an image caption generator. arXiv. Diakses pada 30 April 2026 dari <https://arxiv.org/abs/1703.09137>

- [10] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2016). Grad-CAM: Visual explanations from deep networks via gradient-based localization. arXiv. Diakses pada 7 Mei 2026 dari <https://arxiv.org/abs/1610.02391>
- [11] Karpathy, A. (2015). The unreasonable effectiveness of recurrent neural networks. Andrej Karpathy Blog. Diakses pada 8 Mei 2026 dari <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>
- [12] Stanford University. (2024). CS231n lecture 10: RNNs & image captioning. Diakses pada 10 Mei 2026 dari https://cs231n.stanford.edu/slides/2024/lecture_10.pdf
- [13] Aditya Jain. (n.d.). Flickr8k dataset. Kaggle. Diakses pada 16 Mei 2026 dari <https://www.kaggle.com/datasets/adityajain105/flickr8k>
- [14] Puneet Bansal. (n.d.). Intel image classification. Kaggle. Diakses pada 16 Mei 2026 dari <https://www.kaggle.com/datasets/puneet6060/intel-image-classification>

Lampiran |

Form Penggunaan AI

Format Deklarasi Penggunaan AI Surat Pernyataan Penggunaan AI

Dengan ini, ~~saya~~/kami:

Nama/Nim:

1. Mochammad Fariz Rifqi Rizqulloh / 13523069
2. Muhammad Jibril Ibrahim / 13523085
3. Nayaka Ghana Subrata / 13523090

Fakultas / Sekolah : Sekolah Teknik Elektro dan Informatika

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas / Proposal : Convolutional Neural Network dan Recurrent Neural Network

– Tugas Besar 2 IF3270 Pembelajaran Mesin

Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak menggunakan~~ AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/ Kecerdasan buatan yang kami gunakan adalah :

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	1 - No AI
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2 - AI Assisted idea generation and structuring
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti	Ya	2 - AI Assisted idea generation and structuring

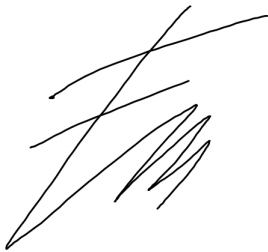
ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.		
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	1 - No AI
Pembuatan Kode : menggunakan tools seperti ChatGPT, Gemini, Copilot, atau sejenisnya.	Ya	2 - AI Assisted idea generation and structuring

Kami juga menyatakan bahwa setiap penggunaan AI/Kecerdasan buatan yang dilakukan pada mata kuliah tersebut dinyatakan di dalam surat ini.

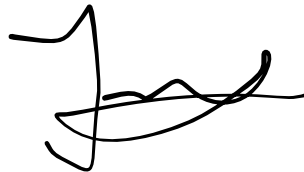
Yogyakarta, 16 Mei 2026

Bandung, 16 Mei 2026

Bandung, 16 Mei 2026



Mochammad Fariz Rifqi
Rizqulloh
13523069



Muhammad Jibril Ibrahim
13523085



Nayaka Ghana Subrata
13523090

scratch_forward.py

```
scratch_forward.py

from __future__ import annotations

import numpy as np

from captioning.scratch_decoder import BeamCandidate, RecurrentWeights

def decoder_forward(decoder, feature: np.ndarray, tokens: np.ndarray) ->
np.ndarray:
    projected_image = np.tanh(feature @ decoder.image_projection.kernel +
decoder.image_projection.bias)
    token_embeddings = decoder.token_embedding[tokens]
    x = np.concatenate([projected_image[None, :], token_embeddings], axis=0)

    for weights in decoder.recurrent_layers:
        if decoder.model_kind == "rnn":
            x = simple_rnn_forward(x, weights)
        elif decoder.model_kind == "lstm":
            x = lstm_forward(x, weights)
        else:
            raise ValueError(f"Unsupported model_kind: {decoder.model_kind}")

    caption_steps = x[1:, :]
    logits = caption_steps @ decoder.token_distribution.kernel +
decoder.token_distribution.bias
    return softmax(logits)

def decoder_forward_batch(decoder, feature_batch: np.ndarray, tokens:
np.ndarray) -> np.ndarray:
    projected_image = np.tanh(feature_batch @ decoder.image_projection.kernel
+ decoder.image_projection.bias)
    token_embeddings = decoder.token_embedding[tokens]
```

```

    x = np.concatenate([projected_image[:, None, :], token_embeddings],
axis=1)

    for weights in decoder.recurrent_layers:
        if decoder.model_kind == "rnn":
            x = simple_rnn_forward_batch(x, weights)
        elif decoder.model_kind == "lstm":
            x = lstm_forward_batch(x, weights)
        else:
            raise ValueError(f"Unsupported model_kind: {decoder.model_kind}")

    caption_steps = x[:, 1:, :]
    logits = caption_steps @ decoder.token_distribution.kernel +
decoder.token_distribution.bias
    return softmax(logits)

def greedy_decode(decoder, feature: np.ndarray) -> str:
    pad_idx = decoder.word_to_idx["<pad>"]
    start_idx = decoder.word_to_idx["<start>"]
    end_idx = decoder.word_to_idx["<end>"]
    tokens = np.full((decoder.max_steps,), pad_idx, dtype=np.int32)
    tokens[0] = start_idx
    generated: list[str] = []

    for step in range(decoder.max_steps):
        predictions = decoder_forward(decoder, feature, tokens)
        next_idx = int(np.argmax(predictions[step]))
        if next_idx == end_idx:
            break

        word = decoder.idx_to_word.get(str(next_idx), "<unk>")
        if word not in {"<pad>", "<start>", "<unk>"}:
            generated.append(word)
        if step + 1 < decoder.max_steps:
            tokens[step + 1] = next_idx

```

```

        return " ".join(generated) if generated else "<empty>"

def greedy_decode_batch(decoder, feature_batch: np.ndarray) -> list[str]:
    pad_idx = decoder.word_to_idx["<pad>"]
    start_idx = decoder.word_to_idx["<start>"]
    end_idx = decoder.word_to_idx["<end>"]
    batch_size = feature_batch.shape[0]
    tokens = np.full((batch_size, decoder.max_steps), pad_idx, dtype=np.int32)
    tokens[:, 0] = start_idx
    generated: list[list[str]] = [[] for _ in range(batch_size)]
    finished = np.zeros((batch_size,), dtype=bool)

    for step in range(decoder.max_steps):
        predictions = decoder_forward_batch(decoder, feature_batch, tokens)
        next_indices = np.argmax(predictions[:, step, :],
axis=1).astype(np.int32)

        for row_index, next_idx in enumerate(next_indices):
            if finished[row_index]:
                continue
            if next_idx == end_idx:
                finished[row_index] = True
                continue

            word = decoder.idx_to_word.get(str(int(next_idx)), "<unk>")
            if word not in {"<pad>", "<start>", "<unk>"}:
                generated[row_index].append(word)
            if step + 1 < decoder.max_steps:
                tokens[row_index, step + 1] = next_idx

    if finished.all():
        break

    return [" ".join(words) if words else "<empty>" for words in generated]

```

```

def beam_search_decode(decoder, feature: np.ndarray, beam_width: int = 3) ->
str:
    if beam_width < 1:
        raise ValueError("beam_width must be at least 1")

    pad_idx = decoder.word_to_idx["<pad>"]
    start_idx = decoder.word_to_idx["<start>"]
    end_idx = decoder.word_to_idx["<end>"]
    blocked_indices = {pad_idx, start_idx, decoder.word_to_idx.get("<unk>")}
    tokens = np.full((decoder.max_steps,), pad_idx, dtype=np.int32)
    tokens[0] = start_idx
    candidates = [BeamCandidate(tokens=tokens, words=(), log_probability=0.0,
ended=False)]

    for step in range(decoder.max_steps):
        expanded: list[BeamCandidate] = []
        for candidate in candidates:
            if candidate.ended:
                expanded.append(candidate)
                continue

            probabilities = decoder_forward(decoder, feature,
candidate.tokens)[step]
            probabilities = probabilities.copy()
            for blocked_idx in blocked_indices:
                if blocked_idx is not None:
                    probabilities[int(blocked_idx)] = 0.0

            top_indices = np.argsort(probabilities)[-beam_width:][::-1]
            for next_idx in top_indices:
                next_idx = int(next_idx)
                next_probability = max(float(probabilities[next_idx]), 1e-12)
                next_tokens = candidate.tokens.copy()
                if step + 1 < decoder.max_steps:
                    next_tokens[step + 1] = next_idx

                if next_idx == end_idx:

```

```

        expanded.append(
            BeamCandidate(
                tokens=next_tokens,
                words=candidate.words,
                log_probability=candidate.log_probability +
float(np.log(next_probability)),
                ended=True,
            )
        )
        continue

    word = decoder.idx_to_word.get(str(next_idx), "<unk>")
    expanded.append(
        BeamCandidate(
            tokens=next_tokens,
            words=(*candidate.words, word),
            log_probability=candidate.log_probability +
float(np.log(next_probability)),
            ended=False,
        )
    )

    candidates = sorted(expanded, key=beam_candidate_score,
reverse=True)[:beam_width]
    if all(candidate.ended for candidate in candidates):
        break

    best = max(candidates, key=beam_candidate_score)
    return " ".join(best.words) if best.words else "<empty>"

def simple_rnn_forward(sequence: np.ndarray, weights: RecurrentWeights) ->
np.ndarray:
    hidden_units = weights.recurrent_kernel.shape[0]
    hidden = np.zeros((hidden_units,), dtype=np.float32)
    outputs = []
    for timestep in sequence:

```

```

        hidden = np.tanh(timestep @ weights.kernel + hidden @
weights.recurrent_kernel + weights.bias)
        outputs.append(hidden)
    return np.stack(outputs, axis=0)

def simple_rnn_forward_batch(sequence: np.ndarray, weights: RecurrentWeights)
-> np.ndarray:
    batch_size = sequence.shape[0]
    hidden_units = weights.recurrent_kernel.shape[0]
    hidden = np.zeros((batch_size, hidden_units), dtype=np.float32)
    outputs = []
    for step in range(sequence.shape[1]):
        hidden = np.tanh(sequence[:, step, :] @ weights.kernel + hidden @
weights.recurrent_kernel + weights.bias)
        outputs.append(hidden)
    return np.stack(outputs, axis=1)

def lstm_forward(sequence: np.ndarray, weights: RecurrentWeights) ->
np.ndarray:
    hidden_units = weights.recurrent_kernel.shape[0]
    hidden = np.zeros((hidden_units,), dtype=np.float32)
    cell = np.zeros((hidden_units,), dtype=np.float32)
    outputs = []
    for timestep in sequence:
        gates = timestep @ weights.kernel + hidden @ weights.recurrent_kernel
+ weights.bias
        input_gate, forget_gate, cell_candidate, output_gate = np.split(gates,
4)

        input_gate = sigmoid(input_gate)
        forget_gate = sigmoid(forget_gate)
        cell_candidate = np.tanh(cell_candidate)
        output_gate = sigmoid(output_gate)
        cell = forget_gate * cell + input_gate * cell_candidate
        hidden = output_gate * np.tanh(cell)
        outputs.append(hidden)

```

```

        return np.stack(outputs, axis=0)

def lstm_forward_batch(sequence: np.ndarray, weights: RecurrentWeights) ->
np.ndarray:
    batch_size = sequence.shape[0]
    hidden_units = weights.recurrent_kernel.shape[0]
    hidden = np.zeros((batch_size, hidden_units), dtype=np.float32)
    cell = np.zeros((batch_size, hidden_units), dtype=np.float32)
    outputs = []
    for step in range(sequence.shape[1]):
        gates = sequence[:, step, :] @ weights.kernel + hidden @
weights.recurrent_kernel + weights.bias
        input_gate, forget_gate, cell_candidate, output_gate = np.split(gates,
4, axis=1)
        input_gate = sigmoid(input_gate)
        forget_gate = sigmoid(forget_gate)
        cell_candidate = np.tanh(cell_candidate)
        output_gate = sigmoid(output_gate)
        cell = forget_gate * cell + input_gate * cell_candidate
        hidden = output_gate * np.tanh(cell)
        outputs.append(hidden)
    return np.stack(outputs, axis=1)

def beam_candidate_score(candidate: BeamCandidate) -> float:
    length = max(1, len(candidate.words))
    return candidate.log_probability / length

def sigmoid(x: np.ndarray) -> np.ndarray:
    return 1.0 / (1.0 + np.exp(-x))

def softmax(x: np.ndarray) -> np.ndarray:
    shifted = x - np.max(x, axis=-1, keepdims=True)
    exp = np.exp(shifted)

```



```
return exp / np.sum(exp, axis=-1, keepdims=True)
```

scratch_backprop.py

```
scratch_backprop.py
```

```
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import numpy as np

from captioning.scratch_decoder import (
    FEATURE_BASENAME,
    FEATURE_DIR,
    REPO_ROOT,
    TEACHER_FORCING_DIR,
    DenseWeights,
    RecurrentWeights,
    ScratchDecoder,
)
from captioning.scratch_forward import sigmoid, softmax

SCRATCH_MODEL_DIR = REPO_ROOT / "artifacts" / "captioning" / "scratch_models"

@dataclass(frozen=True)
class RecurrentGradients:
    kernel: np.ndarray
    recurrent_kernel: np.ndarray
    bias: np.ndarray

@dataclass(frozen=True)
```

```

class ScratchGradients:
    image_projection: DenseWeights
    token_embedding: np.ndarray
    recurrent_layers: tuple[RecurrentGradients, ...]
    token_distribution: DenseWeights

def evaluate_batch(
    decoder: ScratchDecoder,
    feature_batch: np.ndarray,
    tokens: np.ndarray,
    targets: np.ndarray,
    pad_idx: int,
) -> float:
    probabilities = decoder.forward_batch(feature_batch, tokens)
    return masked_softmax_cross_entropy_loss(probabilities, targets, pad_idx)

def backward_batch(
    decoder: ScratchDecoder,
    feature_batch: np.ndarray,
    tokens: np.ndarray,
    targets: np.ndarray,
    pad_idx: int,
) -> tuple[float, ScratchGradients]:
    projected_image, image_projection_cache = dense_tanh_forward(
        feature_batch,
        decoder.image_projection,
    )
    token_embeddings = decoder.token_embedding[tokens]
    x = np.concatenate([projected_image[:, None, :], token_embeddings],
axis=1)

    recurrent_caches = []
    for weights in decoder.recurrent_layers:
        if decoder.model_kind == "rnn":
            x, cache = simple_rnn_forward_batch_with_cache(x, weights)

```

```

        elif decoder.model_kind == "lstm":
            x, cache = lstm_forward_batch_with_cache(x, weights)
        else:
            raise ValueError(f"Unsupported model_kind: {decoder.model_kind}")
        recurrent_caches.append(cache)

caption_steps = x[:, 1:, :]
logits = caption_steps @ decoder.token_distribution.kernel +
decoder.token_distribution.bias
loss, dlogits = masked_softmax_cross_entropy_backward(logits, targets,
pad_idx)

token_distribution_grad = DenseWeights(
    kernel=np.einsum("bth,btv->hv", caption_steps, dlogits),
    bias=np.sum(dlogits, axis=(0, 1)),
)
dcaption_steps = dlogits @ decoder.token_distribution.kernel.T
dx = np.zeros_like(x)
dx[:, 1:, :] = dcaption_steps

recurrent_grads: list[RecurrentGradients] = []
for weights, cache in reversed(list(zip(decoder.recurrent_layers,
recurrent_caches, strict=True))):
    if decoder.model_kind == "rnn":
        dx, grad = simple_rnn_backward_batch(dx, weights, cache)
    else:
        dx, grad = lstm_backward_batch(dx, weights, cache)
    recurrent_grads.append(grad)
recurrent_grads.reverse()

dprojected_image = dx[:, 0, :]
dtoken_embeddings = dx[:, 1:, :]
_, image_projection_grad = dense_tanh_backward(
    dprojected_image,
    decoder.image_projection,
    image_projection_cache,
)

```

```

token_embedding_grad = np.zeros_like(decoder.token_embedding)
np.add.at(token_embedding_grad, tokens, dtoken_embeddings)

return loss, ScratchGradients(
    image_projection=image_projection_grad,
    token_embedding=token_embedding_grad,
    recurrent_layers=tuple(recurrent_grads),
    token_distribution=token_distribution_grad,
)

def train_batch(
    decoder: ScratchDecoder,
    feature_batch: np.ndarray,
    tokens: np.ndarray,
    targets: np.ndarray,
    pad_idx: int,
    learning_rate: float,
) -> tuple[float, ScratchDecoder, ScratchGradients]:
    loss, gradients = backward_batch(decoder, feature_batch, tokens, targets,
pad_idx)
    return loss, apply_sgd_update(decoder, gradients, learning_rate),
gradients

def dense_tanh_forward(x: np.ndarray, weights: DenseWeights) ->
tuple[np.ndarray, dict[str, np.ndarray]]:
    linear = x @ weights.kernel + weights.bias
    output = np.tanh(linear)
    return output, {"input": x, "output": output}

def dense_tanh_backward(
    doutput: np.ndarray,
    weights: DenseWeights,
    cache: dict[str, np.ndarray],

```

```

) -> tuple[np.ndarray, DenseWeights]:
    x = cache["input"]
    output = cache["output"]
    dlinear = doutput * (1.0 - output * output)
    return dlinear @ weights.kernel.T, DenseWeights(
        kernel=x.T @ dlinear,
        bias=np.sum(dlinear, axis=0),
    )

def masked_softmax_cross_entropy_backward(
    logits: np.ndarray,
    targets: np.ndarray,
    pad_idx: int,
) -> tuple[float, np.ndarray]:
    probabilities = softmax(logits)
    mask = targets != pad_idx
    valid_count = int(np.sum(mask))
    if valid_count == 0:
        return 0.0, np.zeros_like(logits)

    batch_indices = np.arange(targets.shape[0])[:, None]
    time_indices = np.arange(targets.shape[1])[None, :]
    clipped = np.clip(probabilities[batch_indices, time_indices, targets],
1e-12, 1.0)
    loss = -float(np.sum(np.log(clipped) * mask) / valid_count)

    dlogits = probabilities.copy()
    dlogits[batch_indices, time_indices, targets] -= 1.0
    dlogits *= mask[..., None]
    dlogits /= valid_count
    return loss, dlogits

def masked_softmax_cross_entropy_loss(
    probabilities: np.ndarray,
    targets: np.ndarray,

```

```

        pad_idx: int,
    ) -> float:
        mask = targets != pad_idx
        valid_count = int(np.sum(mask))
        if valid_count == 0:
            return 0.0

        batch_indices = np.arange(targets.shape[0])[:, None]
        time_indices = np.arange(targets.shape[1])[None, :]
        clipped = np.clip(probabilities[batch_indices, time_indices, targets],
1e-12, 1.0)
        return -float(np.sum(np.log(clipped) * mask) / valid_count)

def simple_rnn_forward_batch_with_cache(
    sequence: np.ndarray,
    weights: RecurrentWeights,
) -> tuple[np.ndarray, dict[str, np.ndarray]]:
    batch_size = sequence.shape[0]
    hidden_units = weights.recurrent_kernel.shape[0]
    hidden = np.zeros((batch_size, hidden_units), dtype=sequence.dtype)
    outputs = []
    previous_hiddens = []
    for step in range(sequence.shape[1]):
        previous_hiddens.append(hidden)
        hidden = np.tanh(sequence[:, step, :] @ weights.kernel + hidden @
weights.recurrent_kernel + weights.bias)
        outputs.append(hidden)
    output = np.stack(outputs, axis=1)
    return output, {
        "input": sequence,
        "previous_hiddens": np.stack(previous_hiddens, axis=1),
        "outputs": output,
    }

def simple_rnn_backward_batch(

```

```

    doutputs: np.ndarray,
    weights: RecurrentWeights,
    cache: dict[str, np.ndarray],
) -> tuple[np.ndarray, RecurrentGradients]:
    sequence = cache["input"]
    previous_hiddens = cache["previous_hiddens"]
    outputs = cache["outputs"]

    dsequence = np.zeros_like(sequence)
    dkernel = np.zeros_like(weights.kernel)
    drecurrent_kernel = np.zeros_like(weights.recurrent_kernel)
    dbias = np.zeros_like(weights.bias)
    dh_next = np.zeros((sequence.shape[0], weights.recurrent_kernel.shape[0]),
dtype=sequence.dtype)

    for step in range(sequence.shape[1] - 1, -1, -1):
        dh = doutputs[:, step, :] + dh_next
        da = dh * (1.0 - outputs[:, step, :] * outputs[:, step, :])
        dkernel += sequence[:, step, :].T @ da
        drecurrent_kernel += previous_hiddens[:, step, :].T @ da
        dbias += np.sum(da, axis=0)
        dsequence[:, step, :] = da @ weights.kernel.T
        dh_next = da @ weights.recurrent_kernel.T

    return dsequence, RecurrentGradients(
        kernel=dkernel,
        recurrent_kernel=drecurrent_kernel,
        bias=dbias,
    )

def lstm_forward_batch_with_cache(
    sequence: np.ndarray,
    weights: RecurrentWeights,
) -> tuple[np.ndarray, dict[str, np.ndarray]]:
    batch_size = sequence.shape[0]
    hidden_units = weights.recurrent_kernel.shape[0]

```

```

hidden = np.zeros((batch_size, hidden_units), dtype=sequence.dtype)
cell = np.zeros((batch_size, hidden_units), dtype=sequence.dtype)
outputs = []
previous_hiddens = []
previous_cells = []
cells = []
input_gates = []
forget_gates = []
cell_candidates = []
output_gates = []

for step in range(sequence.shape[1]):
    previous_hiddens.append(hidden)
    previous_cells.append(cell)
    gates = sequence[:, step, :] @ weights.kernel + hidden @
weights.recurrent_kernel + weights.bias
    input_gate, forget_gate, cell_candidate, output_gate = np.split(gates,
4, axis=1)
    input_gate = sigmoid(input_gate)
    forget_gate = sigmoid(forget_gate)
    cell_candidate = np.tanh(cell_candidate)
    output_gate = sigmoid(output_gate)
    cell = forget_gate * cell + input_gate * cell_candidate
    hidden = output_gate * np.tanh(cell)

    input_gates.append(input_gate)
    forget_gates.append(forget_gate)
    cell_candidates.append(cell_candidate)
    output_gates.append(output_gate)
    cells.append(cell)
    outputs.append(hidden)

output = np.stack(outputs, axis=1)
return output, {
    "input": sequence,
    "previous_hiddens": np.stack(previous_hiddens, axis=1),
    "previous_cells": np.stack(previous_cells, axis=1),

```



```

        "cells": np.stack(cells, axis=1),
        "input_gates": np.stack(input_gates, axis=1),
        "forget_gates": np.stack(forget_gates, axis=1),
        "cell_candidates": np.stack(cell_candidates, axis=1),
        "output_gates": np.stack(output_gates, axis=1),
        "outputs": output,
    }

def lstm_backward_batch(
    doutputs: np.ndarray,
    weights: RecurrentWeights,
    cache: dict[str, np.ndarray],
) -> tuple[np.ndarray, RecurrentGradients]:
    sequence = cache["input"]
    previous_hiddens = cache["previous_hiddens"]
    previous_cells = cache["previous_cells"]
    cells = cache["cells"]
    input_gates = cache["input_gates"]
    forget_gates = cache["forget_gates"]
    cell_candidates = cache["cell_candidates"]
    output_gates = cache["output_gates"]

    dsequence = np.zeros_like(sequence)
    dkernel = np.zeros_like(weights.kernel)
    drecurrent_kernel = np.zeros_like(weights.recurrent_kernel)
    dbias = np.zeros_like(weights.bias)
    hidden_units = weights.recurrent_kernel.shape[0]
    dh_next = np.zeros((sequence.shape[0], hidden_units),
dtype=sequence.dtype)
    dc_next = np.zeros((sequence.shape[0], hidden_units),
dtype=sequence.dtype)

    for step in range(sequence.shape[1] - 1, -1, -1):
        cell = cells[:, step, :]
        previous_cell = previous_cells[:, step, :]
        input_gate = input_gates[:, step, :]

```

```

    forget_gate = forget_gates[:, step, :]
    cell_candidate = cell_candidates[:, step, :]
    output_gate = output_gates[:, step, :]

    dh = doutputs[:, step, :] + dh_next
    tanh_cell = np.tanh(cell)
    doutput_gate = dh * tanh_cell
    dcell = dh * output_gate * (1.0 - tanh_cell * tanh_cell) + dc_next
    dforget_gate = dcell * previous_cell
    dprevious_cell = dcell * forget_gate
    dinput_gate = dcell * cell_candidate
    dcell_candidate = dcell * input_gate

    dinput_gate *= input_gate * (1.0 - input_gate)
    dforget_gate *= forget_gate * (1.0 - forget_gate)
    dcell_candidate *= 1.0 - cell_candidate * cell_candidate
    doutput_gate *= output_gate * (1.0 - output_gate)
    dgates = np.concatenate(
        [dinput_gate, dforget_gate, dcell_candidate, doutput_gate],
        axis=1,
    )

    dkernel += sequence[:, step, :].T @ dgates
    drecurrent_kernel += previous_hiddens[:, step, :].T @ dgates
    dbias += np.sum(dgates, axis=0)
    dsequence[:, step, :] = dgates @ weights.kernel.T
    dh_next = dgates @ weights.recurrent_kernel.T
    dc_next = dprevious_cell

    return dsequence, RecurrentGradients(
        kernel=dkernel,
        recurrent_kernel=drecurrent_kernel,
        bias=dbias,
    )

def apply_sgd_update(

```

```

        decoder: ScratchDecoder,
        gradients: ScratchGradients,
        learning_rate: float,
    ) -> ScratchDecoder:
        if learning_rate <= 0:
            raise ValueError("learning_rate must be greater than 0")

        return ScratchDecoder(
            model_kind=decoder.model_kind,
            image_projection=sgd_dense(decoder.image_projection,
gradients.image_projection, learning_rate),
            token_embedding=decoder.token_embedding - learning_rate *
gradients.token_embedding,
            recurrent_layers=tuple(
                sgd_recurrent(weights, grad, learning_rate)
                for weights, grad in zip(decoder.recurrent_layers,
gradients.recurrent_layers, strict=True)
            ),
            token_distribution=sgd_dense(decoder.token_distribution,
gradients.token_distribution, learning_rate),
            word_to_idx=decoder.word_to_idx,
            idx_to_word=decoder.idx_to_word,
            max_steps=decoder.max_steps,
        )

def sgd_dense(weights: DenseWeights, gradients: DenseWeights, learning_rate:
float) -> DenseWeights:
    return DenseWeights(
        kernel=weights.kernel - learning_rate * gradients.kernel,
        bias=weights.bias - learning_rate * gradients.bias,
    )

def sgd_recurrent(
    weights: RecurrentWeights,
    gradients: RecurrentGradients,

```

```

        learning_rate: float,
) -> RecurrentWeights:
    return RecurrentWeights(
        kernel=weights.kernel - learning_rate * gradients.kernel,
        recurrent_kernel=weights.recurrent_kernel - learning_rate *
gradients.recurrent_kernel,
        bias=weights.bias - learning_rate * gradients.bias,
    )

def load_json(path: Path):
    with path.open("r", encoding="utf-8") as file:
        import json

        return json.load(file)

def load_teacher_forcing_batch(
    split: str,
    batch_size: int,
    start: int = 0,
) -> tuple[np.ndarray, np.ndarray, np.ndarray, int]:
    if batch_size <= 0:
        raise ValueError("batch_size must be greater than 0")
    if start < 0:
        raise ValueError("start must be non-negative")

    metadata = load_json(TEACHER_FORCING_DIR /
"teacher_forcing_metadata.json")
    features = np.load(FEATURE_DIR /
f"{FEATURE_BASENAME}_features.npy").astype("float32", copy=False)
    teacher = np.load(TEACHER_FORCING_DIR / f"{split}_teacher_forcing.npz")
    end = start + batch_size
    feature_indices = teacher["feature_indices"][start:end]
    if len(feature_indices) == 0:
        raise ValueError(f"No samples available for split={split!r},
start={start}, batch_size={batch_size}")

```

```

    return (
        features[feature_indices],
        teacher["caption_inputs"][start:end],
        teacher["caption_targets"][start:end],
        int(metadata["pad_idx"]),
    )

def iter_teacher_forcing_batches(
    split: str,
    batch_size: int,
    limit_samples: int | None = None,
):
    if batch_size <= 0:
        raise ValueError("batch_size must be greater than 0")

    metadata = load_json(TEACHER_FORCING_DIR /
"teacher_forcing_metadata.json")
    features = np.load(FEATURE_DIR /
f"{FEATURE_BASENAME}_features.npy").astype("float32", copy=False)
    teacher = np.load(TEACHER_FORCING_DIR / f"{split}_teacher_forcing.npz")
    total = len(teacher["feature_indices"])
    if limit_samples is not None:
        if limit_samples <= 0:
            raise ValueError("limit_samples must be greater than 0")
        total = min(total, limit_samples)

    pad_idx = int(metadata["pad_idx"])
    for start in range(0, total, batch_size):
        end = min(start + batch_size, total)
        feature_indices = teacher["feature_indices"][start:end]
        yield (
            features[feature_indices],
            teacher["caption_inputs"][start:end],
            teacher["caption_targets"][start:end],
            pad_idx,

```

```

        start,
    )

def fit_scratch(
    decoder: ScratchDecoder,
    split: str,
    batch_size: int,
    epochs: int,
    learning_rate: float,
    limit_samples: int | None = None,
) -> tuple[ScratchDecoder, list[dict[str, float]]]:
    if epochs <= 0:
        raise ValueError("epochs must be greater than 0")

    current = decoder
    history: list[dict[str, float]] = []
    for epoch in range(1, epochs + 1):
        losses: list[float] = []
        for feature_batch, tokens, targets, pad_idx, _ in
            iter_teacher_forcing_batches(split, batch_size, limit_samples):
            loss, current, _ = train_batch(
                current,
                feature_batch,
                tokens,
                targets,
                pad_idx,
                learning_rate,
            )
            losses.append(loss)

        epoch_loss = float(np.mean(losses)) if losses else 0.0
        history.append({"epoch": float(epoch), "loss": epoch_loss})
        print(f"Epoch {epoch}/{epochs} - scratch_loss: {epoch_loss:.6f}")

    return current, history

```

```

def save_scratch_weights(decoder: ScratchDecoder, output_path: str | Path) ->
Path:
    path = Path(output_path)
    if path.suffix == ".h5" or path.name.endswith(".weights.h5"):
        raise ValueError("Refusing to save scratch weights to a Keras .h5
file. Use a separate .npz path.")
    if path.suffix != ".npz":
        raise ValueError("Scratch weights must be saved as a .npz file")

    path.parent.mkdir(parents=True, exist_ok=True)
    arrays: dict[str, np.ndarray] = {
        "model_kind": np.array(decoder.model_kind),
        "image_projection.kernel": decoder.image_projection.kernel,
        "image_projection.bias": decoder.image_projection.bias,
        "token_embedding": decoder.token_embedding,
        "token_distribution.kernel": decoder.token_distribution.kernel,
        "token_distribution.bias": decoder.token_distribution.bias,
        "max_steps": np.array(decoder.max_steps, dtype=np.int32),
    }
    for index, weights in enumerate(decoder.recurrent_layers):
        arrays[f"recurrent_layers.{index}.kernel"] = weights.kernel
        arrays[f"recurrent_layers.{index}.recurrent_kernel"] =
weights.recurrent_kernel
        arrays[f"recurrent_layers.{index}.bias"] = weights.bias

    np.savez_compressed(path, **arrays)
    return path

def default_scratch_weights_path(experiment_id: str) -> Path:
    return SCRATCH_MODEL_DIR / f"{experiment_id}_scratch_sgd.npz"

```

extract_inception_features.py

```
extract_inception_features.py
```

```

from __future__ import annotations

import json
from datetime import datetime, timezone
from pathlib import Path

import numpy as np

from common.image_io import extract_features, save_features
from captioning.dataset import prepare_flickr8k_splits, save_json

REPO_ROOT = Path(__file__).resolve().parents[2]
DEFAULT_RAW_DIR = REPO_ROOT / "data" / "raw" / "flickr8k"
DEFAULT_SPLITS_DIR = REPO_ROOT / "artifacts" / "captioning" / "splits"
DEFAULT_FEATURES_DIR = REPO_ROOT / "artifacts" / "captioning" / "features"
FEATURE_BASENAME = "inception_v3_flickr8k"
BATCH_SIZE = 32
LIMIT: int | None = None
PREPARE_ONLY = False

def build_encoder():
    try:
        from tensorflow.keras.applications import InceptionV3
    except ModuleNotFoundError as error:
        raise ModuleNotFoundError(
            "TensorFlow is required for InceptionV3 feature extraction. "
            "Install dependencies with: pip install -r requirements.txt"
        ) from error

    # agak beda sama kalau misal yang di example geeksforgeeks, soalnya kalau
    # di geeksforgeeks dia ga ngasih params include_top=False, jadi dia basically
    # tetep ngelakuin predict using the inception model. Kemarin tanya tanya sama
    # gemini sama gpt sih katanya gak efisien, mending langsung pake
    include_top=True, jadi yaudah gw pake aja ini
    encoder = InceptionV3(weights="imagenet", include_top=False,
        pooling="avg")

```



```

encoder.trainable = False
return encoder

def inception_preprocess(batch: np.ndarray) -> np.ndarray:
    try:
        from tensorflow.keras.applications.inception_v3 import
preprocess_input
    except ModuleNotFoundError as error:
        raise ModuleNotFoundError(
            "TensorFlow is required for InceptionV3 preprocessing. "
            "Install dependencies with: pip install -r requirements.txt"
        ) from error

    return preprocess_input(batch)

def main() -> None:
    splits = prepare_flickr8k_splits(DEFAULT_RAW_DIR, DEFAULT_SPLITS_DIR)
    if PREPARE_ONLY:
        print(f"Saved splits and captions: {DEFAULT_SPLITS_DIR}")
        return

    image_ids = splits["train"] + splits["val"] + splits["test"] +
splits["unused"]
    if LIMIT is not None:
        if LIMIT <= 0:
            raise ValueError("LIMIT must be greater than 0")
        image_ids = image_ids[:LIMIT]

    image_paths = [DEFAULT_RAW_DIR / "Images" / image_id for image_id in
image_ids]
    encoder = build_encoder()
    features = extract_features(
        image_paths=image_paths,
        encoder=encoder,
        target_size=(299, 299),

```

```

        batch_size=BATCH_SIZE,
        normalize=False,
        preprocess_fn=inception_preprocess,
        verbose=0,
    ).astype(np.float32, copy=False)

    suffix = "" if LIMIT is None else f"_limit{LIMIT}"
    features_path = DEFAULT_FEATURES_DIR /
f"{FEATURE_BASENAME}{suffix}_features.npy"
    image_ids_path = DEFAULT_FEATURES_DIR /
f"{FEATURE_BASENAME}{suffix}_image_ids.json"
    metadata_path = DEFAULT_FEATURES_DIR /
f"{FEATURE_BASENAME}{suffix}_metadata.json"

    save_features(features, features_path)
    save_json(image_ids, image_ids_path)
    save_json(
        {
            "encoder": "InceptionV3",
            "weights": "imagenet",
            "include_top": False,
            "pooling": "avg",
            "trainable": False,
            "input_size": [299, 299],
            "preprocess_fn":
"tensorflow.keras.applications.inception_v3.preprocess_input",
            "feature_shape": list(features.shape),
            "feature_dtype": str(features.dtype),
            "feature_stage": "before_decoder_projection",
            "image_count": len(image_ids),
            "split_counts": {name: len(ids) for name, ids in splits.items()},
            "created_at_utc": datetime.now(timezone.utc).isoformat(),
        },
        metadata_path,
    )

    print(f"Saved features: {features_path}")

```

```
print(f"Saved image ids: {image_ids_path}")
print(f"Saved metadata: {metadata_path}")

if __name__ == "__main__":
    main()
```

preprocess_captions.py

```
preprocess_captions.py

from __future__ import annotations

import json
import re
from collections import Counter
from pathlib import Path

from captioning.dataset import CaptionMap, save_json

REPO_ROOT = Path(__file__).resolve().parents[2]
DEFAULT_SPLITS_DIR = REPO_ROOT / "artifacts" / "captioning" / "splits"
DEFAULT_OUTPUT_DIR = REPO_ROOT / "artifacts" / "captioning" / "preprocessed"
PAD_TOKEN = "<pad>"
START_TOKEN = "<start>"
END_TOKEN = "<end>"
UNK_TOKEN = "<unk>"
SPECIAL_TOKENS = [PAD_TOKEN, START_TOKEN, END_TOKEN, UNK_TOKEN]

# captionnya cuma di-preprocess biar consist of english letter + space doang
# sih (sama semuanya jadi lower letter)
# kalau yang di geeksforgeeks dia ngelakuinnya gak di lower, underscore masih
# persist soalnya pakai \w
def clean_caption(caption: str) -> list[str]:
```

```

    text = caption.lower()
    text = re.sub(r"^[a-z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text.split() if text else []

def tokenize_caption(caption: str) -> list[str]:
    return [START_TOKEN, *clean_caption(caption), END_TOKEN]

def load_caption_map(path: str | Path) -> CaptionMap:
    with Path(path).open("r", encoding="utf-8") as file:
        data = json.load(file)

    return {image_id: list(captions) for image_id, captions in data.items()}

def build_vocabulary(train_captions: CaptionMap) -> tuple[dict[str, int],
dict[str, str]]:
    counter: Counter[str] = Counter()
    for captions in train_captions.values():
        for caption in captions:
            counter.update(clean_caption(caption))

    tokens = [*SPECIAL_TOKENS, *sorted(counter)]
    word_to_idx = {token: index for index, token in enumerate(tokens)}
    idx_to_word = {str(index): token for token, index in word_to_idx.items()}
    return word_to_idx, idx_to_word

def encode_caption(caption: str, word_to_idx: dict[str, int]) -> list[int]:
    unk_idx = word_to_idx[UNK_TOKEN]
    return [word_to_idx.get(token, unk_idx) for token in
tokenize_caption(caption)]

def encode_caption_map(

```

```

        captions_by_image: CaptionMap,
        word_to_idx: dict[str, int],
    ) -> dict[str, list[list[int]]]:
        encoded: dict[str, list[list[int]]] = {}
        for image_id, captions in captions_by_image.items():
            encoded[image_id] = [encode_caption(caption, word_to_idx) for caption
in captions]
        return encoded

def pad_sequence(
    sequence: list[int],
    max_len: int,
    pad_idx: int,
    end_idx: int,
) -> tuple[list[int], bool]:
    if len(sequence) > max_len:
        padded = sequence[:max_len]
        padded[-1] = end_idx
        return padded, True

    return [*sequence, *([pad_idx] * (max_len - len(sequence)))], False

def pad_caption_map(
    encoded_by_image: dict[str, list[list[int]]],
    max_len: int,
    pad_idx: int,
    end_idx: int,
) -> tuple[dict[str, list[list[int]]], int]:
    padded: dict[str, list[list[int]]] = {}
    truncated_count = 0
    for image_id, sequences in encoded_by_image.items():
        padded_sequences = []
        for sequence in sequences:
            padded_sequence, was_truncated = pad_sequence(
                sequence,

```

```

        max_len=max_len,
        pad_idx=pad_idx,
        end_idx=end_idx,
    )
    padded_sequences.append(padded_sequence)
    truncated_count += int(was_truncated)
padded[image_id] = padded_sequences

return padded, truncated_count

def max_caption_length(captions_by_image: CaptionMap) -> int:
    return max(
        len(tokenize_caption(caption))
        for captions in captions_by_image.values()
        for caption in captions
    )

def main() -> None:
    train_captions = load_caption_map(DEFAULT_SPLITS_DIR /
"train_captions.json")
    word_to_idx, idx_to_word = build_vocabulary(train_captions)
    max_len = max_caption_length(train_captions)
    pad_idx = word_to_idx[PAD_TOKEN]
    end_idx = word_to_idx[END_TOKEN]

    save_json(word_to_idx, DEFAULT_OUTPUT_DIR / "word_to_idx.json")
    save_json(idx_to_word, DEFAULT_OUTPUT_DIR / "idx_to_word.json")

    split_counts: dict[str, int] = {}
    truncated_counts: dict[str, int] = {}
    for split_name in ("train", "val", "test"):
        captions = load_caption_map(DEFAULT_SPLITS_DIR /
f"{split_name}_captions.json")
        encoded = encode_caption_map(captions, word_to_idx)
        padded, truncated_count = pad_caption_map(

```

```

        encoded,
        max_len=max_len,
        pad_idx=pad_idx,
        end_idx=end_idx,
    )
    save_json(encoded, DEFAULT_OUTPUT_DIR /
f"{split_name}_encoded_captions.json")
    save_json(padded, DEFAULT_OUTPUT_DIR /
f"{split_name}_padded_captions.json")
    split_counts[split_name] = sum(len(items) for items in
encoded.values())
    truncated_counts[split_name] = truncated_count

save_json(
    {
        "vocab_size": len(word_to_idx),
        "special_tokens": SPECIAL_TOKENS,
        "pad_idx": word_to_idx[PAD_TOKEN],
        "start_idx": word_to_idx[START_TOKEN],
        "end_idx": word_to_idx[END_TOKEN],
        "unk_idx": word_to_idx[UNK_TOKEN],
        "max_seq_len_train": max_len,
        "sequence_stage": "integer_encoded_and_padded",
        "split_caption_counts": split_counts,
        "truncated_caption_counts": truncated_counts,
    },
    DEFAULT_OUTPUT_DIR / "caption_preprocessing_metadata.json",
)

print(f"Saved caption preprocessing artifacts: {DEFAULT_OUTPUT_DIR}")
print(f"Vocabulary size: {len(word_to_idx)}")

if __name__ == "__main__":
    main()

```

build_teacher_forcing.py

```
build_teacher_forcing.py

from __future__ import annotations

import json
from pathlib import Path

import numpy as np

from captioning.dataset import save_json

REPO_ROOT = Path(__file__).resolve().parents[2]
DEFAULT_FEATURES_DIR = REPO_ROOT / "artifacts" / "captioning" / "features"
DEFAULT_PREPROCESSED_DIR = REPO_ROOT / "artifacts" / "captioning" /
"preprocessed"
DEFAULT_OUTPUT_DIR = REPO_ROOT / "artifacts" / "captioning" /
"teacher_forcing"
FEATURE_BASENAME = "inception_v3_flickr8k"

def load_json(path: str | Path):
    with Path(path).open("r", encoding="utf-8") as file:
        return json.load(file)

def build_split_arrays(
    padded_captions: dict[str, list[list[int]]],
    image_id_to_feature_idx: dict[str, int],
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    feature_indices: list[int] = []
    caption_inputs: list[list[int]] = []
    caption_targets: list[list[int]] = []

    for image_id, sequences in padded_captions.items():
```



```

        if image_id not in image_id_to_feature_idx:
            raise ValueError(f"Missing feature index for image: {image_id}")

        feature_idx = image_id_to_feature_idx[image_id]
        for sequence in sequences:
            if len(sequence) < 2:
                raise ValueError(f"Sequence must contain at least 2 tokens:
{image_id}")

            feature_indices.append(feature_idx)
            caption_inputs.append(sequence[:-1])
            caption_targets.append(sequence[1:])

    return (
        np.asarray(feature_indices, dtype=np.int32),
        np.asarray(caption_inputs, dtype=np.int32),
        np.asarray(caption_targets, dtype=np.int32),
    )

def save_split_npz(
    output_path: str | Path,
    feature_indices: np.ndarray,
    caption_inputs: np.ndarray,
    caption_targets: np.ndarray,
) -> None:
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    np.savez_compressed(
        path,
        feature_indices=feature_indices,
        caption_inputs=caption_inputs,
        caption_targets=caption_targets,
    )

def main() -> None:
    image_ids = load_json(DEFAULT_FEATURES_DIR /

```

```

f"{FEATURE_BASENAME}_image_ids.json")
    feature_metadata = load_json(DEFAULT_FEATURES_DIR /
f"{FEATURE_BASENAME}_metadata.json")
    preprocessing_metadata = load_json(
        DEFAULT_PREPROCESSED_DIR / "caption_preprocessing_metadata.json"
    )
    image_id_to_feature_idx = {
        image_id: index for index, image_id in enumerate(image_ids)
    }

    split_shapes = {}
    for split_name in ("train", "val", "test"):
        padded_captions = load_json(
            DEFAULT_PREPROCESSED_DIR / f"{split_name}_padded_captions.json"
        )
        feature_indices, caption_inputs, caption_targets = build_split_arrays(
            padded_captions,
            image_id_to_feature_idx,
        )
        save_split_npz(
            DEFAULT_OUTPUT_DIR / f"{split_name}_teacher_forcing.npz",
            feature_indices=feature_indices,
            caption_inputs=caption_inputs,
            caption_targets=caption_targets,
        )
        split_shapes[split_name] = {
            "feature_indices": list(feature_indices.shape),
            "caption_inputs": list(caption_inputs.shape),
            "caption_targets": list(caption_targets.shape),
        }

    save_json(
        {
            "feature_file": str(
                DEFAULT_FEATURES_DIR / f"{FEATURE_BASENAME}_features.npy"
            ),
            "feature_shape": feature_metadata["feature_shape"],

```

```

        "feature_stage": feature_metadata["feature_stage"],
        "max_seq_len": preprocessing_metadata["max_seq_len_train"],
        "decoder_timesteps": preprocessing_metadata["max_seq_len_train"] -
1,
        "pad_idx": preprocessing_metadata["pad_idx"],
        "start_idx": preprocessing_metadata["start_idx"],
        "end_idx": preprocessing_metadata["end_idx"],
        "unk_idx": preprocessing_metadata["unk_idx"],
        "array_dtypes": {
            "feature_indices": "int32",
            "caption_inputs": "int32",
            "caption_targets": "int32",
        },
        "split_shapes": split_shapes,
    },
    DEFAULT_OUTPUT_DIR / "teacher_forcing_metadata.json",
)

print(f"Saved teacher-forcing artifacts: {DEFAULT_OUTPUT_DIR}")

if __name__ == "__main__":
    main()

```

train_decoders.py

```

train_decoders.py

from __future__ import annotations

import csv
import json
import time
from dataclasses import asdict, dataclass
from pathlib import Path

```

```

import numpy as np
import tensorflow as tf
from tqdm import tqdm
from tqdm.keras import TqdmCallback

from captioning.decoder import build_lstm_decoder, build_simple_rnn_decoder

REPO_ROOT = Path(__file__).resolve().parents[2]
DEFAULT_PREPROCESSED_DIR = REPO_ROOT / "artifacts" / "captioning" /
"preprocessed"
DEFAULT_TEACHER_FORCING_DIR = REPO_ROOT / "artifacts" / "captioning" /
"teacher_forcing"
DEFAULT_EXPERIMENT_DIR = REPO_ROOT / "artifacts" / "captioning" /
"experiments"
DEFAULT_MODEL_DIR = REPO_ROOT / "artifacts" / "captioning" / "models"
DEFAULT_HISTORY_DIR = REPO_ROOT / "artifacts" / "captioning" / "histories"

MODEL_KIND = "both" # "both", "rnn", or "lstm"
EXPERIMENT_ID: str | None = None
EPOCHS = 20
BATCH_SIZE = 64
LIMIT_TRAIN_SAMPLES: int | None = None
LIMIT_VAL_SAMPLES: int | None = None
WRITE_CONFIGS_ONLY = False

@dataclass(frozen=True)
class ExperimentConfig:
    experiment_id: str
    model_kind: str
    recurrent_layers: int
    hidden_units: int
    embed_dim: int = 256
    learning_rate: float = 1e-3

```

```

@property
def hidden_units_per_layer(self) -> tuple[int, ...]:
    return (self.hidden_units,) * self.recurrent_layers

def default_experiment_configs() -> list[ExperimentConfig]:
    configs: list[ExperimentConfig] = []
    for model_kind in ("rnn", "lstm"):
        for recurrent_layers in (1, 2, 3):
            for hidden_units in (128, 512):
                configs.append(
                    ExperimentConfig(
                        experiment_id=(
f"{model_kind}_layers{recurrent_layers}_hidden{hidden_units}"
                                ),
                        model_kind=model_kind,
                        recurrent_layers=recurrent_layers,
                        hidden_units=hidden_units,
                    )
                )
    return configs

def load_json(path: str | Path):
    with Path(path).open("r", encoding="utf-8") as file:
        return json.load(file)

def save_json(data: object, output_path: str | Path) -> None:
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as file:
        json.dump(data, file, indent=2)
        file.write("\n")

```

```

def save_config_table(
    configs: list[ExperimentConfig],
    output_path: str | Path,
) -> None:
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    fieldnames = [
        "experiment_id",
        "model_kind",
        "recurrent_layers",
        "hidden_units",
        "embed_dim",
        "learning_rate",
    ]
    with path.open("w", encoding="utf-8", newline="") as file:
        writer = csv.DictWriter(file, fieldnames=fieldnames)
        writer.writeheader()
        for config in configs:
            writer.writerow(asdict(config))

def load_training_metadata(
    preprocessed_dir: str | Path,
    teacher_forcing_dir: str | Path,
) -> dict[str, object]:
    preprocessing = load_json(Path(preprocessed_dir) /
"caption_preprocessing_metadata.json")
    teacher_forcing = load_json(Path(teacher_forcing_dir) /
"teacher_forcing_metadata.json")
    return {
        "vocab_size": preprocessing["vocab_size"],
        "pad_idx": teacher_forcing["pad_idx"],
        "decoder_timesteps": teacher_forcing["decoder_timesteps"],
        "feature_dim": teacher_forcing["feature_shape"][-1],
        "feature_file": teacher_forcing["feature_file"],
    }

```

```

def load_split_arrays(
    teacher_forcing_dir: str | Path,
    split_name: str,
    features: np.ndarray,
    limit_samples: int | None = None,
) -> tuple[tuple[np.ndarray, np.ndarray], np.ndarray]:
    path = Path(teacher_forcing_dir) / f"{split_name}_teacher_forcing.npz"
    data = np.load(path)
    feature_indices = data["feature_indices"]
    caption_inputs = data["caption_inputs"]
    caption_targets = data["caption_targets"]

    if limit_samples is not None:
        feature_indices = feature_indices[:limit_samples]
        caption_inputs = caption_inputs[:limit_samples]
        caption_targets = caption_targets[:limit_samples]

    return (features[feature_indices], caption_inputs), caption_targets


def build_model(
    config: ExperimentConfig,
    metadata: dict[str, object],
) -> tf.keras.Model:
    builder = build_simple_rnn_decoder if config.model_kind == "rnn" else
    build_lstm_decoder
    return builder(
        vocab_size=int(metadata["vocab_size"]),
        decoder_timesteps=int(metadata["decoder_timesteps"]),
        pad_idx=int(metadata["pad_idx"]),
        feature_dim=int(metadata["feature_dim"]),
        embed_dim=config.embed_dim,
        hidden_units=config.hidden_units_per_layer,
        learning_rate=config.learning_rate,
    )

```

```

def train_experiment(
    config: ExperimentConfig,
    metadata: dict[str, object],
    teacher_forcing_dir: str | Path,
    model_dir: str | Path,
    history_dir: str | Path,
    epochs: int,
    batch_size: int,
    limit_train_samples: int | None = None,
    limit_val_samples: int | None = None,
) -> dict[str, object]:
    features = np.load(str(metadata["feature_file"])).astype("float32",
copy=False)
    train_x, train_y = load_split_arrays(
        teacher_forcing_dir,
        "train",
        features,
        limit_samples=limit_train_samples,
    )
    val_x, val_y = load_split_arrays(
        teacher_forcing_dir,
        "val",
        features,
        limit_samples=limit_val_samples,
    )

    model = build_model(config, metadata)
    started_at = time.perf_counter()
    history = model.fit(
        train_x,
        train_y,
        validation_data=(val_x, val_y),
        epochs=epochs,
        batch_size=batch_size,
        verbose=1
    )

```



```

elapsed_seconds = time.perf_counter() - started_at

model_path = Path(model_dir) / f"{config.experiment_id}.weights.h5"
model_path.parent.mkdir(parents=True, exist_ok=True)
model.save_weights(model_path)

history_payload = {
    "config": asdict(config),
    "epochs": epochs,
    "batch_size": batch_size,
    "elapsed_seconds": elapsed_seconds,
    "history": {key: [float(value) for value in values] for key, values in
history.history.items()}},
    "weights_file": str(model_path),
}
history_path = Path(history_dir) / f"{config.experiment_id}_history.json"
save_json(history_payload, history_path)

return {
    "experiment_id": config.experiment_id,
    "model_kind": config.model_kind,
    "recurrent_layers": config.recurrent_layers,
    "hidden_units": config.hidden_units,
    "epochs": epochs,
    "batch_size": batch_size,
    "elapsed_seconds": elapsed_seconds,
    "final_loss": float(history.history["loss"][-1]),
    "final_val_loss": float(history.history["val_loss"][-1]),
    "weights_file": str(model_path),
    "history_file": str(history_path),
}

def select_configs(
    configs: list[ExperimentConfig],
    model_kind: str,
    experiment_id: str | None,

```

```

) -> list[ExperimentConfig]:
    selected = configs
    if model_kind != "both":
        selected = [config for config in selected if config.model_kind ==
model_kind]
    if experiment_id is not None:
        selected = [config for config in selected if config.experiment_id ==
experiment_id]
    if not selected:
        raise ValueError("No experiment configs matched the requested
filters")
    return selected

def main() -> None:
    configs = select_configs(
        default_experiment_configs(),
        model_kind=MODEL_KIND,
        experiment_id=EXPERIMENT_ID,
    )
    save_config_table(configs, DEFAULT_EXPERIMENT_DIR /
"decoder_experiment_configs.csv")
    save_json(
        [asdict(config) for config in configs],
        DEFAULT_EXPERIMENT_DIR / "decoder_experiment_configs.json",
    )

    if WRITE_CONFIGS_ONLY:
        print(f"Saved {len(configs)} experiment configs:
{DEFAULT_EXPERIMENT_DIR}")
        return

    metadata = load_training_metadata(DEFAULT_PREPROCESSED_DIR,
DEFAULT_TEACHER_FORCING_DIR)
    summaries = []
    for config in tqdm(configs, desc="Training decoder experiments",
unit="experiment"):

```

```

print(f"Training {config.experiment_id}")
summaries.append(
    train_experiment(
        config=config,
        metadata=metadata,
        teacher_forcing_dir=DEFAULT_TEACHER_FORCING_DIR,
        model_dir=DEFAULT_MODEL_DIR,
        history_dir=DEFAULT_HISTORY_DIR,
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        limit_train_samples=LIMIT_TRAIN_SAMPLES,
        limit_val_samples=LIMIT_VAL_SAMPLES,
    )
)

save_json(summaries, DEFAULT_EXPERIMENT_DIR /
"decoder_training_summary.json")
print(
    "Saved training summary: "
    f"{DEFAULT_EXPERIMENT_DIR / 'decoder_training_summary.json'}"
)

if __name__ == "__main__":
    main()

```

decoder.py

```

decoder.py

from __future__ import annotations

from collections.abc import Sequence

import tensorflow as tf
from tensorflow import keras

```

```

from tensorflow.keras import layers

DEFAULT_FEATURE_DIM = 2048

def masked_sparse_categorical_crossentropy(
    pad_idx: int,
) -> keras.losses.Loss:
    class MaskedSparseCategoricalCrossentropy(keras.losses.Loss):
        def __init__(self) -> None:
            super().__init__(name="masked_sparse_categorical_crossentropy")
            self._loss = keras.losses.SparseCategoricalCrossentropy(
                reduction="none"
            )

        def call(self, y_true: tf.Tensor, y_pred: tf.Tensor) -> tf.Tensor:
            losses = self._loss(y_true, y_pred)
            mask = tf.cast(tf.not_equal(y_true, pad_idx), losses.dtype)
            losses = losses * mask
            return tf.math.divide_no_nan(
                tf.reduce_sum(losses),
                tf.reduce_sum(mask),
            )

        def get_config(self) -> dict[str, int]:
            return {"pad_idx": pad_idx}

    return MaskedSparseCategoricalCrossentropy()

def compile_caption_decoder(
    model: keras.Model,
    pad_idx: int,
    learning_rate: float = 1e-3,
) -> keras.Model:
    model.compile(

```

```

        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss=masked_sparse_categorical_crossentropy(pad_idx),
    )
    return model

def build_simple_rnn_decoder(
    vocab_size: int,
    decoder_timesteps: int,
    pad_idx: int,
    feature_dim: int = DEFAULT_FEATURE_DIM,
    embed_dim: int = 256,
    hidden_units: int | Sequence[int] = 256,
    learning_rate: float = 1e-3,
    compile_model: bool = True,
) -> keras.Model:
    model = _build_preinject_decoder(
        recurrent_layer=layers.SimpleRNN,
        recurrent_name="simple_rnn",
        vocab_size=vocab_size,
        decoder_timesteps=decoder_timesteps,
        feature_dim=feature_dim,
        embed_dim=embed_dim,
        hidden_units=hidden_units,
    )
    if compile_model:
        compile_caption_decoder(model, pad_idx=pad_idx,
learning_rate=learning_rate)
    return model

def build_lstm_decoder(
    vocab_size: int,
    decoder_timesteps: int,
    pad_idx: int,
    feature_dim: int = DEFAULT_FEATURE_DIM,
    embed_dim: int = 256,

```

```

        hidden_units: int | Sequence[int] = 256,
        learning_rate: float = 1e-3,
        compile_model: bool = True,
    ) -> keras.Model:
        model = _build_preinject_decoder(
            recurrent_layer=layers.LSTM,
            recurrent_name="lstm",
            vocab_size=vocab_size,
            decoder_timesteps=decoder_timesteps,
            feature_dim=feature_dim,
            embed_dim=embed_dim,
            hidden_units=hidden_units,
        )
        if compile_model:
            compile_caption_decoder(model, pad_idx=pad_idx,
learning_rate=learning_rate)
        return model

def build_init_inject_simple_rnn_decoder(
    vocab_size: int,
    decoder_timesteps: int,
    pad_idx: int,
    feature_dim: int = DEFAULT_FEATURE_DIM,
    embed_dim: int = 256,
    hidden_units: int | Sequence[int] = 256,
    learning_rate: float = 1e-3,
    compile_model: bool = True,
) -> keras.Model:
    model = _build_init_inject_decoder(
        recurrent_layer=layers.SimpleRNN,
        recurrent_name="simple_rnn",
        vocab_size=vocab_size,
        decoder_timesteps=decoder_timesteps,
        feature_dim=feature_dim,
        embed_dim=embed_dim,
        hidden_units=hidden_units,
    )

```

```

    )
    if compile_model:
        compile_caption_decoder(model, pad_idx=pad_idx,
learning_rate=learning_rate)
    return model

def build_init_inject_lstm_decoder(
    vocab_size: int,
    decoder_timesteps: int,
    pad_idx: int,
    feature_dim: int = DEFAULT_FEATURE_DIM,
    embed_dim: int = 256,
    hidden_units: int | Sequence[int] = 256,
    learning_rate: float = 1e-3,
    compile_model: bool = True,
) -> keras.Model:
    model = _build_init_inject_decoder(
        recurrent_layer=layers.LSTM,
        recurrent_name="lstm",
        vocab_size=vocab_size,
        decoder_timesteps=decoder_timesteps,
        feature_dim=feature_dim,
        embed_dim=embed_dim,
        hidden_units=hidden_units,
    )
    if compile_model:
        compile_caption_decoder(model, pad_idx=pad_idx,
learning_rate=learning_rate)
    return model

def _build_preinject_decoder(
    recurrent_layer: type[layers.Layer],
    recurrent_name: str,
    vocab_size: int,
    decoder_timesteps: int,

```

```

        feature_dim: int,
        embed_dim: int,
        hidden_units: int | Sequence[int],
    ) -> keras.Model:
        units_per_layer = _normalize_hidden_units(hidden_units)

        image_features = keras.Input(
            shape=(feature_dim,),
            name="image_features",
        )
        caption_tokens = keras.Input(
            shape=(decoder_timesteps,),
            dtype="int32",
            name="caption_tokens",
        )

        projected_image = layers.Dense(
            embed_dim,
            activation="tanh",
            name="image_projection",
        )(image_features)
        image_timestep = layers.Reshape((1, embed_dim), name="image_timestep")(
            projected_image
        )
        token_embeddings = layers.Embedding(
            vocab_size,
            embed_dim,
            mask_zero=False,
            name="token_embedding",
        )(caption_tokens)
        decoder_inputs = layers.Concatenate(axis=1, name="preinject_sequence")(
            [image_timestep, token_embeddings]
        )

        x = decoder_inputs
        for layer_index, units in enumerate(units_per_layer, start=1):
            x = recurrent_layer(

```



```

        units,
        return_sequences=True,
        name=f"{recurrent_name}_{layer_index}",
    )(x)

caption_steps = layers.Lambda(
    lambda values: values[:, 1:, :],
    name="drop_image_timestep",
)(x)
token_distribution = layers.Dense(
    vocab_size,
    activation="softmax",
    name="token_distribution",
)(caption_steps)

return keras.Model(
    inputs=[image_features, caption_tokens],
    outputs=token_distribution,
    name=f"preinject_{recurrent_name}_decoder",
)

def _build_init_inject_decoder(
    recurrent_layer: type[layers.Layer],
    recurrent_name: str,
    vocab_size: int,
    decoder_timesteps: int,
    feature_dim: int,
    embed_dim: int,
    hidden_units: int | Sequence[int],
) -> keras.Model:
    units_per_layer = _normalize_hidden_units(hidden_units)

    image_features = keras.Input(
        shape=(feature_dim,),
        name="image_features",
    )

```

```

caption_tokens = keras.Input(
    shape=(decoder_timesteps,),
    dtype="int32",
    name="caption_tokens",
)

x = layers.Embedding(
    vocab_size,
    embed_dim,
    mask_zero=False,
    name="token_embedding",
)(caption_tokens)

for layer_index, units in enumerate(units_per_layer, start=1):
    initial_hidden = layers.Dense(
        units,
        activation="tanh",
        name=f"image_to_h{layer_index}",
    )(image_features)
    initial_state = [initial_hidden]
    if recurrent_layer is layers.LSTM:
        initial_cell = layers.Dense(
            units,
            activation="tanh",
            name=f"image_to_c{layer_index}",
        )(image_features)
        initial_state.append(initial_cell)

    x = recurrent_layer(
        units,
        return_sequences=True,
        name=f"{recurrent_name}_{layer_index}",
    )(x, initial_state=initial_state)

token_distribution = layers.Dense(
    vocab_size,
    activation="softmax",

```

```

        name="token_distribution",
    )(x)

    return keras.Model(
        inputs=[image_features, caption_tokens],
        outputs=token_distribution,
        name=f"initinject_{recurrent_name}_decoder",
    )

def _normalize_hidden_units(hidden_units: int | Sequence[int]) -> tuple[int,
...]:
    if isinstance(hidden_units, int):
        units = (hidden_units,)
    else:
        units = tuple(hidden_units)

    if not units:
        raise ValueError("hidden_units must contain at least one layer")
    if any(unit <= 0 for unit in units):
        raise ValueError("hidden_units values must be positive")
    return units

```

src/cnn/nn/layers.py

```

src/cnn/nn/layers.py

import numpy as np

from ..core.base import Layer, WeightedLayer
from ..core.ops import (
    ensure_4d_batch,
    extract_image_patches,
    normalize_tuple,
    restore_batch_dim,
)

```

```

from .activations import get_activation

class ActivationLayer(Layer):
    def __init__(self, activation, name=None):
        super().__init__(name=name)
        self.activation = activation or "linear"
        self.activation_fn = get_activation(self.activation)

    def forward(self, inputs):
        return self.activation_fn(np.asarray(inputs))

    def get_config(self):
        config = super().get_config()
        config.update({"activation": self.activation})
        return config

class Conv2D(WeightedLayer):
    def __init__(
        self,
        filters,
        kernel_size,
        strides=(1, 1),
        padding="valid",
        activation=None,
        use_bias=True,
        dilation_rate=(1, 1),
        name=None,
    ):
        super().__init__(name=name)
        self.filters = int(filters)
        self.kernel_size = normalize_tuple(kernel_size, 2, "kernel_size")
        self.strides = normalize_tuple(strides, 2, "strides")
        self.padding = padding.lower()
        self.activation = activation or "linear"
        self.activation_fn = get_activation(self.activation)
        self.use_bias = use_bias
        self.dilation_rate = normalize_tuple(dilation_rate, 2,

```

```

"dilation_rate")
    self.kernel = None
    self.bias = None

    def set_weights(self, weights):
        if len(weights) not in {1, 2}:
            raise ValueError(f"Conv2D expects 1 or 2 tensors, got
{len(weights)}")

        kernel = np.asarray(weights[0], dtype=np.float32)
        if kernel.ndim != 4:
            raise ValueError(f"Conv2D kernel must be 4D, got {kernel.shape}")
        if kernel.shape[:2] != self.kernel_size:
            raise ValueError(
                f"Conv2D kernel size mismatch, expected {self.kernel_size},
got {kernel.shape[:2]}")
        )
        if kernel.shape[-1] != self.filters:
            raise ValueError(
                f"Conv2D filter count mismatch, expected {self.filters}, got
{kernel.shape[-1]}")
        )

        self.kernel = kernel
        self.bias = None
        if len(weights) == 2:
            bias = np.asarray(weights[1], dtype=np.float32)
            if bias.shape != (self.filters,):
                raise ValueError(
                    f"Conv2D bias must have shape {(self.filters,)}, got
{bias.shape}")
                )
            self.bias = bias
        elif self.use_bias:
            self.bias = np.zeros((self.filters,), dtype=np.float32)

    def get_weights(self):

```

```

        if self.kernel is None:
            return []
        if self.bias is None:
            return [self.kernel.copy()]
        return [self.kernel.copy(), self.bias.copy()]

    def forward(self, inputs):
        if self.kernel is None:
            raise RuntimeError("Conv2D weights are not initialized")

        patches, squeeze_batch = extract_image_patches(
            inputs=inputs,
            kernel_size=self.kernel_size,
            strides=self.strides,
            padding=self.padding,
            dilation_rate=self.dilation_rate,
        )
        outputs = np.einsum("nhwklc,klcf->nhwf", patches, self.kernel,
optimize=True)
        if self.bias is not None:
            outputs = outputs + self.bias.reshape((1, 1, 1, -1))
        outputs = self.activation_fn(outputs)
        return restore_batch_dim(outputs, squeeze_batch)

    def get_config(self):
        config = super().get_config()
        config.update(
            {
                "filters": self.filters,
                "kernel_size": self.kernel_size,
                "strides": self.strides,
                "padding": self.padding,
                "activation": self.activation,
                "use_bias": self.use_bias,
                "dilation_rate": self.dilation_rate,
            }
        )

```

```

        return config

class LocallyConnected2D(WeightedLayer):
    def __init__(
        self,
        filters,
        kernel_size,
        strides=(1, 1),
        padding="valid",
        activation=None,
        use_bias=True,
        dilation_rate=(1, 1),
        name=None,
    ):
        super().__init__(name=name)
        self.filters = int(filters)
        self.kernel_size = normalize_tuple(kernel_size, 2, "kernel_size")
        self.strides = normalize_tuple(strides, 2, "strides")
        self.padding = padding.lower()
        self.activation = activation or "linear"
        self.activation_fn = get_activation(self.activation)
        self.use_bias = use_bias
        self.dilation_rate = normalize_tuple(dilation_rate, 2,
"dilation_rate")
        self.kernel = None
        self.bias = None

    def set_weights(self, weights):
        if len(weights) not in {1, 2}:
            raise ValueError(
                f"LocallyConnected2D expects 1 or 2 tensors, got
{len(weights)}"
            )

        kernel = np.asarray(weights[0], dtype=np.float32)
        if kernel.ndim != 3:
            raise ValueError(f"LocallyConnected2D kernel must be 3D, got

```

```

{kernel.shape}")
    if kernel.shape[-1] != self.filters:
        raise ValueError(
            "LocallyConnected2D filter count mismatch, "
            f"expected {self.filters}, got {kernel.shape[-1]}"
        )

    self.kernel = kernel
    self.bias = None
    if len(weights) == 2:
        bias = np.asarray(weights[1], dtype=np.float32)
        if bias.ndim not in {1, 2}:
            raise ValueError(
                "LocallyConnected2D bias must be 1D or 2D, "
                f"got shape {bias.shape}"
            )
        self.bias = bias
    elif self.use_bias:
        self.bias = np.zeros((self.filters,), dtype=np.float32)

def get_weights(self):
    if self.kernel is None:
        return []
    if self.bias is None:
        return [self.kernel.copy()]
    return [self.kernel.copy(), self.bias.copy()]

def forward(self, inputs):
    if self.kernel is None:
        raise RuntimeError("LocallyConnected2D weights are not
initialized")

    patches, squeeze_batch = extract_image_patches(
        inputs=inputs,
        kernel_size=self.kernel_size,
        strides=self.strides,
        padding=self.padding,

```



```

        dilation_rate=self.dilation_rate,
    )
    batch_size, output_height, output_width = patches.shape[:3]
    flattened_patches = patches.reshape(batch_size, output_height *
output_width, -1)

    expected_shape = (
        output_height * output_width,
        flattened_patches.shape[-1],
        self.filters,
    )
    if self.kernel.shape != expected_shape:
        raise ValueError(
            "LocallyConnected2D kernel shape mismatch, "
            f"expected {expected_shape}, got {self.kernel.shape}"
        )

    outputs = np.einsum("npc,pcf->npf", flattened_patches, self.kernel,
optimize=True)
    if self.bias is not None:
        if self.bias.ndim == 1:
            outputs = outputs + self.bias.reshape((1, 1, -1))
        else:
            outputs = outputs + self.bias.reshape((1, output_height *
output_width, -1))

    outputs = outputs.reshape(batch_size, output_height, output_width,
self.filters)
    outputs = self.activation_fn(outputs)
    return restore_batch_dim(outputs, squeeze_batch)

def get_config(self):
    config = super().get_config()
    config.update(
        {
            "filters": self.filters,
            "kernel_size": self.kernel_size,

```

```

        "strides": self.strides,
        "padding": self.padding,
        "activation": self.activation,
        "use_bias": self.use_bias,
        "dilation_rate": self.dilation_rate,
    }
)
return config

class _Pooling2D(Layer):
    def __init__(
        self,
        pool_size=(2, 2),
        strides=None,
        padding="valid",
        name=None,
    ):
        super().__init__(name=name)
        self.pool_size = normalize_tuple(pool_size, 2, "pool_size")
        self.strides = normalize_tuple(strides or pool_size, 2, "strides")
        self.padding = padding.lower()

    def _pool(self, patches):
        raise NotImplementedError

    def forward(self, inputs):
        patches, squeeze_batch = extract_image_patches(
            inputs=inputs,
            kernel_size=self.pool_size,
            strides=self.strides,
            padding=self.padding,
        )
        outputs = self._pool(patches)
        return restore_batch_dim(outputs, squeeze_batch)

    def get_config(self):
        config = super().get_config()

```

```

        config.update(
            {
                "pool_size": self.pool_size,
                "strides": self.strides,
                "padding": self.padding,
            }
        )
    return config

class MaxPooling2D(_Pooling2D):
    def _pool(self, patches):
        return np.max(patches, axis=(3, 4))

class AveragePooling2D(_Pooling2D):
    def _pool(self, patches):
        return np.mean(patches, axis=(3, 4))

class GlobalAveragePooling2D(Layer):
    def forward(self, inputs):
        array, squeeze_batch = ensure_4d_batch(inputs)
        outputs = np.mean(array, axis=(1, 2))
        return restore_batch_dim(outputs, squeeze_batch)

class GlobalMaxPooling2D(Layer):
    def forward(self, inputs):
        array, squeeze_batch = ensure_4d_batch(inputs)
        outputs = np.max(array, axis=(1, 2))
        return restore_batch_dim(outputs, squeeze_batch)

class Flatten(Layer):
    def forward(self, inputs):
        array = np.asarray(inputs)
        if array.ndim == 1:
            return array
        if array.ndim == 2:
            return array
        if array.ndim == 3:

```

```

        return array.reshape(-1, order="C")
    if array.ndim < 2:
        raise ValueError(f"Flatten expects rank >= 2, got {array.shape}")
    return array.reshape(array.shape[0], -1, order="C")

class Dense(WeightedLayer):
    def __init__(
        self,
        units,
        activation=None,
        use_bias=True,
        name=None,
    ):
        super().__init__(name=name)
        self.units = int(units)
        self.activation = activation or "linear"
        self.activation_fn = get_activation(self.activation)
        self.use_bias = use_bias
        self.kernel = None
        self.bias = None

    def set_weights(self, weights):
        if len(weights) not in {1, 2}:
            raise ValueError(f"Dense expects 1 or 2 tensors, got {len(weights)}")

        kernel = np.asarray(weights[0], dtype=np.float32)
        if kernel.ndim != 2:
            raise ValueError(f"Dense kernel must be 2D, got {kernel.shape}")
        if kernel.shape[-1] != self.units:
            raise ValueError(f"Dense units mismatch, expected {self.units}, got {kernel.shape[-1]}")

        self.kernel = kernel
        self.bias = None
        if len(weights) == 2:
            bias = np.asarray(weights[1], dtype=np.float32)

```

```

        if bias.shape != (self.units,):
            raise ValueError(f"Dense bias must have shape {(self.units,)},
got {bias.shape}")
        self.bias = bias
    elif self.use_bias:
        self.bias = np.zeros((self.units,), dtype=np.float32)

def get_weights(self):
    if self.kernel is None:
        return []
    if self.bias is None:
        return [self.kernel.copy()]
    return [self.kernel.copy(), self.bias.copy()]

def forward(self, inputs):
    if self.kernel is None:
        raise RuntimeError("Dense weights are not initialized")

    array = np.asarray(inputs, dtype=np.float32)
    if array.shape[-1] != self.kernel.shape[0]:
        raise ValueError(
            "Dense input feature mismatch, "
            f"expected {self.kernel.shape[0]}, got {array.shape[-1]}"
        )

    outputs = np.tensordot(array, self.kernel, axes=([-1], [0]))
    if self.bias is not None:
        outputs = outputs + self.bias
    return self.activation_fn(outputs)

def get_config(self):
    config = super().get_config()
    config.update(
        {
            "units": self.units,
            "activation": self.activation,
            "use_bias": self.use_bias,

```

```
    }  
    )  
    return config
```

src/cnn/nn/keras_layers.py

```
src/cnn/nn/keras_layers.py
```

```
import tensorflow as tf  
  
from ..core.ops import compute_conv_output_length, normalize_tuple  
  
@tf.keras.utils.register_keras_serializable(package="cnn")  
class KerasLocallyConnected2D(tf.keras.layers.Layer):  
    def __init__(  
        self,  
        filters,  
        kernel_size,  
        strides=(1, 1),  
        padding="valid",  
        activation=None,  
        use_bias=True,  
        dilation_rate=(1, 1),  
        kernel_initializer="glorot_uniform",  
        bias_initializer="zeros",  
        name=None,  
        **kwargs,  
    ):  
        super().__init__(name=name, **kwargs)  
        self.filters = int(filters)  
        self.kernel_size = normalize_tuple(kernel_size, 2, "kernel_size")  
        self.strides = normalize_tuple(strides, 2, "strides")  
        self.padding = padding.lower()  
        self.activation = tf.keras.activations.get(activation)  
        self.activation_name = tf.keras.activations.serialize(self.activation)  
        self.use_bias = use_bias
```

```

        self.dilation_rate = normalize_tuple(dilation_rate, 2,
"dilation_rate")
        self.kernel_initializer =
tf.keras.initializers.get(kernel_initializer)
        self.bias_initializer = tf.keras.initializers.get(bias_initializer)
        self.output_height = None
        self.output_width = None

    def build(self, input_shape):
        if len(input_shape) != 4:
            raise ValueError(
                "KerasLocallyConnected2D expects inputs with rank 4 "
                f"(batch, height, width, channels), got {input_shape}"
            )

        input_height = int(input_shape[1])
        input_width = int(input_shape[2])
        channels = int(input_shape[3])

        self.output_height = compute_conv_output_length(
            input_length=input_height,
            kernel_size=self.kernel_size[0],
            stride=self.strides[0],
            padding=self.padding,
            dilation=self.dilation_rate[0],
        )
        self.output_width = compute_conv_output_length(
            input_length=input_width,
            kernel_size=self.kernel_size[1],
            stride=self.strides[1],
            padding=self.padding,
            dilation=self.dilation_rate[1],
        )
        if self.output_height <= 0 or self.output_width <= 0:
            raise ValueError(
                "Invalid output shape for KerasLocallyConnected2D. "
                "Check kernel_size, strides, padding, dilation_rate, and input

```

```

shape."
        )

        patch_dim = self.kernel_size[0] * self.kernel_size[1] * channels
        self.kernel = self.add_weight(
            name="kernel",
            shape=(self.output_height * self.output_width, patch_dim,
self.filters),
            initializer=self.kernel_initializer,
            trainable=True,
        )
        if self.use_bias:
            self.bias = self.add_weight(
                name="bias",
                shape=(self.output_height * self.output_width, self.filters),
                initializer=self.bias_initializer,
                trainable=True,
            )
        else:
            self.bias = None
        super().build(input_shape)

    def call(self, inputs):
        patches = tf.image.extract_patches(
            images=inputs,
            sizes=[1, self.kernel_size[0], self.kernel_size[1], 1],
            strides=[1, self.strides[0], self.strides[1], 1],
            rates=[1, self.dilation_rate[0], self.dilation_rate[1], 1],
            padding=self.padding.upper(),
        )
        batch_size = tf.shape(patches)[0]
        patches = tf.reshape(
            patches,
            [batch_size, self.output_height * self.output_width,
tf.shape(patches)[-1]],
        )
        outputs = tf.einsum("bpc,pcf->bpf", patches, self.kernel)

```



```

        if self.bias is not None:
            outputs = outputs + self.bias[tf.newaxis, ...]
        outputs = tf.reshape(outputs, [batch_size, self.output_height,
self.output_width, self.filters])
        if self.activation is not None:
            outputs = self.activation(outputs)
        return outputs

    def compute_output_shape(self, input_shape):
        return tf.TensorShape((input_shape[0], self.output_height,
self.output_width, self.filters))

    def get_config(self):
        config = super().get_config()
        config.update(
            {
                "filters": self.filters,
                "kernel_size": self.kernel_size,
                "strides": self.strides,
                "padding": self.padding,
                "activation": self.activation_name,
                "use_bias": self.use_bias,
                "dilation_rate": self.dilation_rate,
                "kernel_initializer":
tf.keras.initializers.serialize(self.kernel_initializer),
                "bias_initializer":
tf.keras.initializers.serialize(self.bias_initializer),
            }
        )
        return config

```

src/cnn/core/ops.py

```
src/cnn/core/ops.py
```

```

from math import ceil
import numpy as np

def normalize_tuple(value, length, name):
    if isinstance(value, int):
        values = (value,) * length
    else:
        values = tuple(int(item) for item in value)

    if len(values) != length:
        raise ValueError(f"{name} must have length {length}, got {values}")

    if any(item <= 0 for item in values):
        raise ValueError(f"{name} must contain positive integers, got {values}")

    return values

def effective_kernel_size(kernel_size, dilation): return kernel_size +
(kernel_size - 1) * (dilation - 1)
def compute_conv_output_length(input_length, kernel_size, stride, padding,
dilation=1):
    effective_kernel = effective_kernel_size(kernel_size, dilation)
    normalized_padding = padding.lower()
    if normalized_padding == "same":
        return ceil(input_length / stride)
    if normalized_padding == "valid":
        return (input_length - effective_kernel) // stride + 1
    raise ValueError(f"Unsupported padding mode: {padding}")

def compute_padding(input_length, kernel_size, stride, padding, dilation=1):
    normalized_padding = padding.lower()
    effective_kernel = effective_kernel_size(kernel_size, dilation)

    if normalized_padding == "valid":
        return 0, 0
    if normalized_padding != "same":

```

```

        raise ValueError(f"Unsupported padding mode: {padding}")

    output_length = ceil(input_length / stride)
    total_padding = max((output_length - 1) * stride + effective_kernel -
input_length, 0)
    padding_before = total_padding // 2
    padding_after = total_padding - padding_before
    return padding_before, padding_after

def ensure_4d_batch(inputs):
    array = np.asarray(inputs)
    squeeze_batch = False

    if array.ndim == 3:
        array = array[np.newaxis, ...]
        squeeze_batch = True

    if array.ndim != 4:
        raise ValueError(
            "Expected image tensor with shape (batch, height, width, channels)
"
            f"or (height, width, channels), got {array.shape}"
        )

    return array, squeeze_batch

def restore_batch_dim(outputs, squeeze_batch):
    if squeeze_batch:
        return outputs[0]
    return outputs

def extract_image_patches(inputs, kernel_size, strides, padding,
dilation_rate=(1, 1)):
    array, squeeze_batch = ensure_4d_batch(inputs)
    kernel_height, kernel_width = kernel_size
    stride_height, stride_width = strides
    dilation_height, dilation_width = dilation_rate

```

```

height = array.shape[1]
width = array.shape[2]
pad_top, pad_bottom = compute_padding(
    input_length=height,
    kernel_size=kernel_height,
    stride=stride_height,
    padding=padding,
    dilation=dilation_height,
)
pad_left, pad_right = compute_padding(
    input_length=width,
    kernel_size=kernel_width,
    stride=stride_width,
    padding=padding,
    dilation=dilation_width,
)

padded = np.pad(
    array,
    pad_width=((0, 0), (pad_top, pad_bottom), (pad_left, pad_right), (0,
0)),
    mode="constant",
    constant_values=0.0,
)

effective_kernel_height = effective_kernel_size(kernel_height,
dilation_height)
effective_kernel_width = effective_kernel_size(kernel_width,
dilation_width)

output_height = compute_conv_output_length(
    input_length=height,
    kernel_size=kernel_height,
    stride=stride_height,
    padding=padding,
    dilation=dilation_height,

```

```

    )
    output_width = compute_conv_output_length(
        input_length=width,
        kernel_size=kernel_width,
        stride=stride_width,
        padding=padding,
        dilation=dilation_width,
    )

    if output_height <= 0 or output_width <= 0:
        raise ValueError(
            "Invalid output size. Check kernel_size, strides, padding, and
input shape."
        )

    windows = np.lib.stride_tricks.sliding_window_view(
        padded,
        window_shape=(effective_kernel_height, effective_kernel_width),
        axis=(1, 2),
    )
    windows = windows[:, ::stride_height, ::stride_width, :, :, :]
    windows = windows[:, :output_height, :output_width, :, :, :]
    patches = np.moveaxis(windows, 3, -1)
    patches = patches[:, :, :, ::dilation_height, ::dilation_width, :]
    return patches, squeeze_batch

```

src/cnn/data/image.py

```
src/cnn/data/image.py
```

```

from pathlib import Path

import numpy as np
from PIL import Image

```

```

def iter_image_batches(image_paths, target_size, batch_size, normalize=True,
dtype=np.float32):
    if batch_size <= 0:
        raise ValueError("batch_size must be greater than 0")

    pending_paths = []
    for image_path in image_paths:
        pending_paths.append(image_path)
        if len(pending_paths) == batch_size:
            yield load_image_batch(
                image_paths=pending_paths,
                target_size=target_size,
                normalize=normalize,
                dtype=dtype,
            )
            pending_paths = []

    if pending_paths:
        yield load_image_batch(
            image_paths=pending_paths,
            target_size=target_size,
            normalize=normalize,
            dtype=dtype,
        )

def load_image(image_path, target_size, normalize=True, dtype=np.float32):
    path = Path(image_path)
    if not path.exists():
        raise FileNotFoundError(f"Image not found: {path}")

    height, width = target_size
    with Image.open(path) as image:
        image = image.convert("RGB")
        image = image.resize((width, height), Image.Resampling.BILINEAR)
        array = np.asarray(image, dtype=dtype)

    if normalize:

```

```

        array = array / np.array(255.0, dtype=dtype)

    return array.astype(dtype, copy=False)

def load_image_batch(image_paths, target_size, normalize=True,
dtype=np.float32):
    arrays = [
        load_image(
            image_path=path,
            target_size=target_size,
            normalize=normalize,
            dtype=dtype,
        )
        for path in image_paths
    ]

    if not arrays:
        height, width = target_size
        return np.empty((0, height, width, 3), dtype=dtype)

    return np.stack(arrays, axis=0)

def extract_features(
    image_paths,
    encoder,
    target_size,
    batch_size=32,
    normalize=True,
    preprocess_fn=None,
    verbose=0,
):
    feature_batches = []

    for batch in iter_image_batches(
        image_paths=image_paths,
        target_size=target_size,
        batch_size=batch_size,

```

```

        normalize=normalize,
    ):
        if preprocess_fn is not None:
            batch = preprocess_fn(batch)

        features = encoder.predict(batch, verbose=verbose)
        feature_batches.append(np.asarray(features))

    if not feature_batches:
        return np.empty((0,), dtype=np.float32)

    return np.concatenate(feature_batches, axis=0)

def save_features(features, output_path):
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    np.save(path, features)

def extract_and_save_features(
    image_paths,
    encoder,
    target_size,
    output_path,
    batch_size=32,
    normalize=True,
    preprocess_fn=None,
    verbose=0,
):
    features = extract_features(
        image_paths,
        encoder,
        target_size,
        batch_size,
        normalize,
        preprocess_fn,
        verbose,
    )

```



```
save_features(features, output_path)
return features
```

src/cnn/data/dataset.py

```
src/cnn/data/dataset.py
```

```
import json
from dataclasses import dataclass
from functools import partial
from pathlib import Path

import numpy as np
import tensorflow as tf
from sklearn.model_selection import StratifiedShuffleSplit

REPO_ROOT = Path(__file__).resolve().parents[3]
DEFAULT_DATASET_ROOTS = (
    REPO_ROOT / "data" / "intel",
    REPO_ROOT / "data" / "raw" / "intel",
)

@dataclass(slots=True)
class ClassificationSplit:
    image_paths: list
    labels: np.ndarray
    class_names: list
    name: str

    @property
    def size(self):
        return len(self.image_paths)

@dataclass(slots=True)
class ClassificationDataset:
    train: ClassificationSplit
```

```

val: ClassificationSplit
test: ClassificationSplit

@property
def class_names(self):
    return self.train.class_names

@property
def num_classes(self):
    return len(self.class_names)

def resolve_dataset_root(dataset_root=None, default_candidates=None):
    if dataset_root is not None:
        candidate = Path(dataset_root)
        if candidate.exists():
            return candidate

        repo_relative = REPO_ROOT / candidate
        if repo_relative.exists():
            return repo_relative

        raise FileNotFoundError(f"Dataset root not found: {candidate}")

    candidates = tuple(default_candidates or DEFAULT_DATASET_ROOTS)
    for candidate in candidates:
        if candidate.exists():
            return candidate

    searched = ", ".join(str(path) for path in candidates)
    raise FileNotFoundError(
        "Dataset root not found. Expected one of: "
        f"{searched}"
    )

def _is_classification_split_dir(directory):
    if directory is None or not directory.exists() or not directory.is_dir():
        return False

```

```

    for child in directory.iterdir():
        if not child.is_dir():
            continue
        if _sorted_image_paths(child):
            return True
    return False

def _find_split_directory(root, candidates):
    for candidate in candidates:
        path = root / candidate
        if _is_classification_split_dir(path):
            return path
    return None

def _sorted_image_paths(directory):
    extensions = ("*.jpg", "*.jpeg", "*.png", "*.bmp")
    image_paths = []
    for pattern in extensions:
        image_paths.extend(directory.glob(pattern))
    return sorted(image_paths)

def _load_split_directory(split_dir, class_names=None, split_name="split"):
    if class_names is None:
        class_names = sorted(path.name for path in split_dir.iterdir() if
path.is_dir())

    image_paths = []
    labels = []
    for label, class_name in enumerate(class_names):
        class_dir = split_dir / class_name
        if not class_dir.exists():
            raise FileNotFoundError(f"Missing class directory: {class_dir}")
        class_image_paths = _sorted_image_paths(class_dir)
        image_paths.extend(class_image_paths)
        labels.extend([label] * len(class_image_paths))

```

```

    return ClassificationSplit(
        image_paths=image_paths,
        labels=np.asarray(labels, dtype=np.int64),
        class_names=list(class_names),
        name=split_name,
    )

def _make_split_subset(train_split, subset_indices, name):
    return ClassificationSplit(
        image_paths=[train_split.image_paths[index] for index in
subset_indices],
        labels=train_split.labels[subset_indices],
        class_names=train_split.class_names,
        name=name,
    )

def _split_training_set(train_split, validation_split, random_state):
    splitter = StratifiedShuffleSplit(
        n_splits=1,
        test_size=validation_split,
        random_state=random_state,
    )
    indices = np.arange(train_split.size)
    train_indices, val_indices = next(splitter.split(indices,
train_split.labels))
    train_subset = _make_split_subset(train_split, train_indices, "train")
    val_subset = _make_split_subset(train_split, val_indices, "val")
    return train_subset, val_subset

def _load_tf_example(path, label, height, width, normalize, preprocess_fn):
    image_bytes = tf.io.read_file(path)
    image = tf.io.decode_image(image_bytes, channels=3,
expand_animations=False)
    image = tf.image.resize(image, [height, width], method="bilinear")
    image = tf.cast(image, tf.float32)
    if normalize:
        image = image / 255.0

```

```

        if preprocess_fn is not None:
            image = preprocess_fn(image)
        return image, label

def _serialize_split(split):
    return {
        "name": split.name,
        "size": split.size,
        "class_names": split.class_names,
        "image_paths": [str(image_path) for image_path in split.image_paths],
        "labels": split.labels.tolist(),
    }

def load_image_classification_dataset(
    dataset_root=None,
    validation_split=0.2,
    random_state=42,
    train_candidates=("train", "seg_train/seg_train", "seg_train"),
    val_candidates=("val", "validation", "seg_val/seg_val", "seg_val",
"seg_validation"),
    test_candidates=("test", "seg_test/seg_test", "seg_test"),
    default_root_candidates=None,
):
    root = resolve_dataset_root(dataset_root,
default_candidates=default_root_candidates)

    train_dir = _find_split_directory(
        root,
        train_candidates,
    )
    val_dir = _find_split_directory(
        root,
        val_candidates,
    )
    test_dir = _find_split_directory(
        root,
        test_candidates,

```

```

    )

    if train_dir is None or test_dir is None:
        raise FileNotFoundError(
            "Expected image classification dataset directories for train and
test splits. "
            "Supported names include train/seg_train and test/seg_test, "
            "including nested Kaggle-style folders like seg_train/seg_train."
        )

    base_train_split = _load_split_directory(train_dir, split_name="train")
    if val_dir is not None:
        val_split = _load_split_directory(
            val_dir,
            class_names=base_train_split.class_names,
            split_name="val",
        )
        train_split = base_train_split
    else:
        train_split, val_split = _split_training_set(
            train_split=base_train_split,
            validation_split=validation_split,
            random_state=random_state,
        )

    test_split = _load_split_directory(
        test_dir,
        class_names=base_train_split.class_names,
        split_name="test",
    )

    return ClassificationDataset(train=train_split, val=val_split,
test=test_split)

def build_tf_dataset(
    split,
    target_size,

```

```

        batch_size=32,
        shuffle=False,
        shuffle_buffer_size=None,
        normalize=True,
        preprocess_fn=None,
        cache=False,
        prefetch=True,
    ):
        if batch_size <= 0:
            raise ValueError("batch_size must be greater than 0")

        height, width = target_size
        path_ds = tf.data.Dataset.from_tensor_slices(
            ([str(path) for path in split.image_paths],
split.labels.astype(np.int32))
        )
        load_example = partial(
            _load_tf_example,
            height=height,
            width=width,
            normalize=normalize,
            preprocess_fn=preprocess_fn,
        )
        dataset = path_ds.map(load_example, num_parallel_calls=tf.data.AUTOTUNE)
        if shuffle:
            if shuffle_buffer_size is None:
                shuffle_buffer_size = min(split.size, max(2048, batch_size * 64))
            dataset = dataset.shuffle(buffer_size=shuffle_buffer_size,
reshuffle_each_iteration=True)
        if cache:
            if isinstance(cache, str):
                dataset = dataset.cache(cache)
            else:
                dataset = dataset.cache()
        dataset = dataset.batch(batch_size).prefetch(tf.data.AUTOTUNE if prefetch
else 1)
        return dataset

```

```
def save_dataset_manifest(dataset, output_path):
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)

    manifest = {
        "train": _serialize_split(dataset.train),
        "val": _serialize_split(dataset.val),
        "test": _serialize_split(dataset.test),
    }
    path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="utf-8")
```

src/common/image_io.py

```
src/common/image_io.py
```

```
from __future__ import annotations

from pathlib import Path
from typing import Callable, Iterable

import numpy as np
from PIL import Image

ArrayPreprocessFn = Callable[[np.ndarray], np.ndarray]

def load_image(
    image_path: str | Path,
    target_size: tuple[int, int],
    normalize: bool = True,
    dtype: np.dtype | str = np.float32,
) -> np.ndarray:
    path = Path(image_path)
    if not path.exists():
```



```

        raise FileNotFoundError(f"Image not found: {path}")

    height, width = target_size

    with Image.open(path) as image:
        image = image.convert("RGB")
        image = image.resize((width, height), Image.Resampling.BILINEAR)
        array = np.asarray(image, dtype=dtype)

    if normalize:
        array = array / np.array(255.0, dtype=dtype)

    return array.astype(dtype, copy=False)

def load_image_batch(
    image_paths: Iterable[str | Path],
    target_size: tuple[int, int],
    normalize: bool = True,
    dtype: np.dtype | str = np.float32,
) -> np.ndarray:
    arrays = [
        load_image(
            image_path=path,
            target_size=target_size,
            normalize=normalize,
            dtype=dtype,
        )
        for path in image_paths
    ]

    if not arrays:
        height, width = target_size
        channels = 3
        return np.empty((0, height, width, channels), dtype=dtype)

    return np.stack(arrays, axis=0)

```

```

def extract_features(
    image_paths: list[str | Path],
    encoder,
    target_size: tuple[int, int],
    batch_size: int = 32,
    normalize: bool = True,
    preprocess_fn: ArrayPreprocessFn | None = None, # placeholder, ntar ini
    verbose: int = 0,
) -> np.ndarray:
    if batch_size <= 0:
        raise ValueError("batch_size must be greater than 0")

    feature_batches: list[np.ndarray] = []

    for start in range(0, len(image_paths), batch_size):
        batch_paths = image_paths[start : start + batch_size]
        batch = load_image_batch(
            image_paths=batch_paths,
            target_size=target_size,
            normalize=normalize,
        )

        if preprocess_fn is not None:
            batch = preprocess_fn(batch)

        features = encoder.predict(batch, verbose=verbose)
        feature_batches.append(np.asarray(features))

    if not feature_batches:
        return np.empty((0,), dtype=np.float32)

    return np.concatenate(feature_batches, axis=0)

```

```
def save_features(features: np.ndarray, output_path: str | Path) -> None:
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    np.save(path, features)
```

src/cnn/training/serialization.py

```
src/cnn/training/serialization.py
```

```
import json
from pathlib import Path

import numpy as np

from ..core.model import Sequential
from ..nn.keras_layers import KerasLocallyConnected2D
from ..nn.metrics import SparseMacroF1
from ..nn.layers import (
    ActivationLayer,
    AveragePooling2D,
    Conv2D,
    Dense,
    Flatten,
    GlobalAveragePooling2D,
    GlobalMaxPooling2D,
    LocallyConnected2D,
    MaxPooling2D,
)
from .train import build_cnn_classifier, compile_cnn_classifier

def load_keras_model(model_or_path, compile=True):
    try:
        import tensorflow as tf
```

```

except ModuleNotFoundError as error:
    raise ModuleNotFoundError(
        "TensorFlow is required to load a Keras model. "
        "Install dependencies with: pip install -r requirements.txt"
    ) from error

if hasattr(model_or_path, "layers"):
    return model_or_path

path = Path(model_or_path)
if not path.exists():
    raise FileNotFoundError(f"Keras model not found: {path}")

return tf.keras.models.load_model(
    path,
    custom_objects={
        "KerasLocallyConnected2D": KerasLocallyConnected2D,
        "SparseMacroF1": SparseMacroF1,
    },
    compile=compile,
)

def load_saved_classifier(model_dir, model_name, compile=False):
    directory = Path(model_dir)
    final_model_path = directory / f"{model_name}_final.keras"
    if final_model_path.exists():
        return load_keras_model(final_model_path, compile=compile)

    summary_path = directory / f"{model_name}_summary.json"
    if not summary_path.exists():
        raise FileNotFoundError(f"Model summary not found: {summary_path}")

    summary = json.loads(summary_path.read_text(encoding="utf-8"))
    config = summary["config"]
    keras_model = build_cnn_classifier(
        input_shape=tuple(config["input_shape"]),
        num_classes=int(summary["num_classes"]),

```

```

        conv_filters=tuple(config["conv_filters"]),
        kernel_sizes=tuple(tuple(kernel) for kernel in
config["kernel_sizes"]),
        pooling_type=str(config["pooling_type"]),
        dense_units=tuple(config["dense_units"]),
        activation=str(config["activation"]),
        conv_padding=str(config["conv_padding"]),
        use_global_pooling=bool(config["use_global_pooling"]),
        local_connectivity=bool(config.get("local_connectivity", False)),
        name=str(config["name"]),
    )

    weights_h5_path = directory / f"{model_name}.weights.h5"
    weights_json_path = directory / f"{model_name}.weights.json"
    if weights_h5_path.exists():
        keras_model.load_weights(weights_h5_path)
    elif weights_json_path.exists():
        keras_model.load_weights(weights_json_path)
    else:
        raise FileNotFoundError(
            f"No supported weight artifact found for {model_name} in
{directory}"
        )

    if compile:
        compile_cnn_classifier(
            model=keras_model,
            num_classes=int(keras_model.output_shape[-1]),
            learning_rate=float(config["learning_rate"]),
        )
    return keras_model

def _serialize_activation(activation):
    try:
        import tensorflow as tf
    except ModuleNotFoundError as error:
        raise ModuleNotFoundError(

```

```

        "TensorFlow is required to inspect Keras activations. "
        "Install dependencies with: pip install -r requirements.txt"
    ) from error

    serialized = tf.keras.activations.serialize(activation)
    if isinstance(serialized, dict):
        return str(serialized.get("class_name", "linear")).lower()
    return str(serialized).lower()

def keras_to_scratch(model_or_path, name=None):
    try:
        import tensorflow as tf
    except ModuleNotFoundError as error:
        raise ModuleNotFoundError(
            "TensorFlow is required to convert a Keras model. "
            "Install dependencies with: pip install -r requirements.txt"
        ) from error

    keras_model = load_keras_model(model_or_path, compile=False)
    scratch_layers = []

    for layer in keras_model.layers:
        if isinstance(layer, tf.keras.layers.InputLayer):
            continue

        if isinstance(layer, tf.keras.layers.Conv2D):
            scratch_layer = Conv2D(
                filters=layer.filters,
                kernel_size=layer.kernel_size,
                strides=layer.strides,
                padding=layer.padding,
                activation=_serialize_activation(layer.activation),
                use_bias=layer.use_bias,
                dilation_rate=layer.dilation_rate,
                name=layer.name,
            )
            scratch_layer.set_weights([np.asarray(weight) for weight in

```

```

layer.get_weights())
    elif isinstance(layer, KerasLocallyConnected2D):
        scratch_layer = LocallyConnected2D(
            filters=layer.filters,
            kernel_size=layer.kernel_size,
            strides=layer.strides,
            padding=layer.padding,
            activation=layer.activation_name,
            use_bias=layer.use_bias,
            dilation_rate=layer.dilation_rate,
            name=layer.name,
        )
        scratch_layer.set_weights([np.asarray(weight) for weight in
layer.get_weights()])
    elif isinstance(layer, tf.keras.layers.MaxPooling2D):
        scratch_layer = MaxPooling2D(
            pool_size=layer.pool_size,
            strides=layer.strides,
            padding=layer.padding,
            name=layer.name,
        )
    elif isinstance(layer, tf.keras.layers.AveragePooling2D):
        scratch_layer = AveragePooling2D(
            pool_size=layer.pool_size,
            strides=layer.strides,
            padding=layer.padding,
            name=layer.name,
        )
    elif isinstance(layer, tf.keras.layers.GlobalAveragePooling2D):
        scratch_layer = GlobalAveragePooling2D(name=layer.name)
    elif isinstance(layer, tf.keras.layers.GlobalMaxPooling2D):
        scratch_layer = GlobalMaxPooling2D(name=layer.name)
    elif isinstance(layer, tf.keras.layers.Flatten):
        scratch_layer = Flatten(name=layer.name)
    elif isinstance(layer, tf.keras.layers.Dense):
        scratch_layer = Dense(
            units=layer.units,

```

```

        activation=_serialize_activation(layer.activation),
        use_bias=layer.use_bias,
        name=layer.name,
    )
    scratch_layer.set_weights([np.asarray(weight) for weight in
layer.get_weights()])
    elif isinstance(layer, tf.keras.layers.ReLU):
        scratch_layer = ActivationLayer(activation="relu",
name=layer.name)
    elif isinstance(layer, tf.keras.layers.Softmax):
        scratch_layer = ActivationLayer(activation="softmax",
name=layer.name)
    elif isinstance(layer, tf.keras.layers.Activation):
        scratch_layer = ActivationLayer(
            activation=_serialize_activation(layer.activation),
            name=layer.name,
        )
    else:
        raise TypeError(
            f"Unsupported Keras layer for scratch conversion:
{layer.__class__.__name__}"
        )

    scratch_layers.append(scratch_layer)

return Sequential(layers=scratch_layers, name=name or keras_model.name)

```

src/cnn/training/train.py

```
src/cnn/training/train.py
```

```

import json
import random
from pathlib import Path

import numpy as np

```



```

import tensorflow as tf

from ..nn.keras_layers import KerasLocallyConnected2D
from ..nn.metrics import SparseMacroF1

def set_random_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)

def _normalize_sequence(values, length, name):
    if isinstance(values, int):
        return [values] * length
    if isinstance(values, tuple) and values and all(isinstance(item, int) for
item in values):
        return [values] * length

    normalized = list(values)
    if len(normalized) != length:
        raise ValueError(f"{name} must have length {length}, got
{len(normalized)}")
    return normalized

def build_cnn_classifier(
    input_shape,
    num_classes,
    conv_filters,
    kernel_sizes,
    pooling_type="max",
    dense_units=(128,),
    activation="relu",
    classifier_activation="softmax",
    pooling_size=(2, 2),
    pooling_strides=None,
    conv_padding="same",
    use_global_pooling=False,
    local_connectivity=False,

```

```

        dropout_rates=None,
        name="cnn_classifier",
    ):
        if num_classes <= 0:
            raise ValueError("num_classes must be greater than 0")
        if not conv_filters:
            raise ValueError("conv_filters must not be empty")

        kernel_sizes = _normalize_sequence(kernel_sizes, len(conv_filters),
"kernel_sizes")
        dropout_rates = list(dropout_rates or [])
        pooling_type_normalized = pooling_type.lower()
        if pooling_type_normalized not in {"max", "avg", "average"}:
            raise ValueError("pooling_type must be 'max' or 'avg'")

        classifier = tf.keras.Sequential(name=name)
        classifier.add(tf.keras.layers.Input(shape=input_shape, name="input"))

        for index, (filters, kernel_size) in enumerate(zip(conv_filters,
kernel_sizes, strict=True)):
            layer_name = f"block{index + 1}"
            if local_connectivity:
                classifier.add(
                    KerasLocallyConnected2D(
                        filters=filters,
                        kernel_size=kernel_size,
                        padding=conv_padding,
                        activation=activation,
                        name=f"{layer_name}_local",
                    )
                )
            else:
                classifier.add(
                    tf.keras.layers.Conv2D(
                        filters=filters,
                        kernel_size=kernel_size,
                        padding=conv_padding,

```

```

        activation=activation,
        name=f"{layer_name}_conv",
    )
)

if pooling_type_normalized == "max":
    classifier.add(
        tf.keras.layers.MaxPooling2D(
            pool_size=pooling_size,
            strides=pooling_strides,
            name=f"{layer_name}_pool",
        )
    )
else:
    classifier.add(
        tf.keras.layers.AveragePooling2D(
            pool_size=pooling_size,
            strides=pooling_strides,
            name=f"{layer_name}_pool",
        )
    )

if use_global_pooling:
    classifier.add(tf.keras.layers.GlobalAveragePooling2D(name="global_pool"))
else:
    classifier.add(tf.keras.layers.Flatten(name="flatten"))

for index, units in enumerate(dense_units):
    classifier.add(
        tf.keras.layers.Dense(units, activation=activation,
name=f"dense_{index + 1}")
    )
    if index < len(dropout_rates):
        classifier.add(
            tf.keras.layers.Dropout(dropout_rates[index],
name=f"dropout_{index + 1}")

```

```

        )

    classifier.add(
        tf.keras.layers.Dense(
            num_classes,
            activation=classifier_activation,
            name="classifier",
        )
    )
    return classifier

def compile_cnn_classifier(
    model,
    num_classes,
    learning_rate=1e-3,
):
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
        loss=tf.keras.losses.SparseCategoricalCrossentropy(),
        metrics=[
            tf.keras.metrics.SparseCategoricalAccuracy(name="accuracy"),
            SparseMacroF1(num_classes=num_classes, name="macro_f1"),
        ],
    )
    return model

def _format_value(value):
    if isinstance(value, str):
        return f"'{value}'"
    return repr(value)

def _format_activation_name(activation):
    if activation in (None, "linear"):
        return None
    if isinstance(activation, dict):
        return activation.get("config", {}).get("activation")
    return activation

```

```

def format_model_architecture(model, variable_name="classifier"):
    lines = [f'{variable_name} = tf.keras.Sequential(name="{model.name}")']
    input_shape = tuple(int(value) for value in model.input_shape[1:])

    for index, layer in enumerate(model.layers):
        config = layer.get_config()
        args = []

        if isinstance(layer, tf.keras.layers.Conv2D):
            args.append(str(layer.filters))
            args.append(repr(tuple(layer.kernel_size)))
            if index == 0:
                args.append(f"input_shape={repr(input_shape)}")
            if layer.padding != "valid":
                args.append(f"padding='{layer.padding}'")
            activation = _format_activation_name(config.get("activation"))
            if activation is not None:
                args.append(f"activation='{activation}'")
            layer_code = f"Conv2D({' , '.join(args)})"
        elif isinstance(layer, KerasLocallyConnected2D):
            args.append(str(layer.filters))
            args.append(repr(tuple(layer.kernel_size)))
            if index == 0:
                args.append(f"input_shape={repr(input_shape)}")
            if layer.padding != "valid":
                args.append(f"padding='{layer.padding}'")
            if layer.activation_name not in (None, "linear"):
                args.append(f"activation='{layer.activation_name}'")
            layer_code = f"KerasLocallyConnected2D({' , '.join(args)})"
        elif isinstance(layer, tf.keras.layers.MaxPooling2D):
            args.append(f"pool_size={repr(tuple(layer.pool_size))}")
            if layer.strides is not None:
                args.append(f"strides={repr(tuple(layer.strides))}")
            layer_code = f"MaxPooling2D({' , '.join(args)})"
        elif isinstance(layer, tf.keras.layers.AveragePooling2D):
            args.append(f"pool_size={repr(tuple(layer.pool_size))}")

```

```

        if layer.strides is not None:
            args.append(f"strides={repr(tuple(layer.strides))}")
            layer_code = f"AveragePooling2D({'', '.join(args)} )"
        elif isinstance(layer, tf.keras.layers.GlobalAveragePooling2D):
            layer_code = "GlobalAveragePooling2D()"
        elif isinstance(layer, tf.keras.layers.GlobalMaxPooling2D):
            layer_code = "GlobalMaxPooling2D()"
        elif isinstance(layer, tf.keras.layers.Flatten):
            layer_code = "Flatten()"
        elif isinstance(layer, tf.keras.layers.Dense):
            args.append(str(layer.units))
            activation = _format_activation_name(config.get("activation"))
            if activation is not None:
                args.append(f"activation='{activation}'")
            layer_code = f"Dense({'', '.join(args)} )"
        elif isinstance(layer, tf.keras.layers.Dropout):
            layer_code = f"Dropout({layer.rate})"
        elif isinstance(layer, tf.keras.layers.ReLU):
            layer_code = "ReLU()"
        elif isinstance(layer, tf.keras.layers.Softmax):
            layer_code = "Softmax()"
        elif isinstance(layer, tf.keras.layers.Activation):
            activation = _format_activation_name(config.get("activation"))
            layer_code = f"Activation('{activation}')"
        else:
            layer_code = f"{layer.__class__.__name__}({'',
'.join(f'{key}={_format_value(value)}' for key, value in config.items() if key
!= 'name')})"

        lines.append(f"{variable_name}.add(tf.keras.layers.{layer_code})")

    return "\n".join(lines)

def summarize_model_layers(model):
    summary = []
    for layer in model.layers:
        config = layer.get_config()

```

```

        summary.append(
            {
                "name": layer.name,
                "type": layer.__class__.__name__,
                "trainable": bool(layer.trainable),
                "params": int(layer.count_params()),
                "filters": config.get("filters"),
                "kernel_size": config.get("kernel_size"),
                "pool_size": config.get("pool_size"),
                "strides": config.get("strides"),
                "padding": config.get("padding"),
                "units": config.get("units"),
                "activation": config.get("activation"),
            }
        )
    return summary

def evaluate_cnn_classifier(
    model,
    dataset,
    verbose=0,
):
    metric_values = model.evaluate(dataset, return_dict=True, verbose=verbose)
    return {key: float(value) for key, value in metric_values.items()}

def save_history(history, output_path):
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    serializable = {key: [float(value) for value in values] for key, values in
history.history.items()}
    path.write_text(json.dumps(serializable, indent=2) + "\n",
encoding="utf-8")

```

src/cnn/training/experiments.py

```
src/cnn/training/experiments.py
```

```

import json
from dataclasses import asdict, dataclass
from pathlib import Path

import numpy as np
import tensorflow as tf

from ..data import build_tf_dataset, iter_image_batches
from ..nn import macro_f1_score
from .serialization import keras_to_scratch
from .train import build_cnn_classifier, compile_cnn_classifier,
evaluate_cnn_classifier, save_history

@dataclass(slots=True)
class CNNExperimentConfig:
    name: str
    conv_filters: tuple
    kernel_sizes: tuple
    pooling_type: str
    dense_units: tuple = (128,)
    batch_size: int = 32
    epochs: int = 15
    learning_rate: float = 1e-3
    input_shape: tuple = (150, 150, 3)
    use_global_pooling: bool = False
    conv_padding: str = "same"
    activation: str = "relu"
    local_connectivity: bool = False

def _normalize_per_layer(values, length, name):
    if isinstance(values, int):
        normalized = [values] * length
    elif isinstance(values, tuple) and values and isinstance(values[0], int):
        normalized = [tuple(int(item) for item in values)] * length
    else:
        normalized = list(values)

```



```

        if len(normalized) != length:
            raise ValueError(f"{name} must have length {length}, got
{len(normalized)}")
        return tuple(normalized)

def build_experiment_grid(
    layer_counts,
    filter_variants,
    kernel_variants,
    pooling_types,
    *,
    dense_units=(128,),
    batch_size=32,
    epochs=15,
    learning_rate=1e-3,
    input_shape=(150, 150, 3),
    use_global_pooling=False,
):
    experiments = []

    for layer_count in layer_counts:
        for filter_variant in filter_variants:
            conv_filters = tuple(int(value) for value in
_normalize_per_layer(filter_variant, layer_count, "filters"))
            for kernel_variant in kernel_variants:
                kernel_sizes = tuple(
                    tuple(int(item) for item in kernel_size)
                    for kernel_size in _normalize_per_layer(kernel_variant,
layer_count, "kernel_sizes")
                )
                for pooling_type in pooling_types:
                    name = (
                        f"cnn_l{layer_count}"
                        f"_f{'-'.join(str(value) for value in conv_filters)}"
                        f"_k{'-'.join(f'{kernel[0]}x{kernel[1]}' for kernel in
kernel_sizes)}"
                        f"_pooling_type}"

```

```

        )
        experiments.append(
            CNNEExperimentConfig(
                name=name,
                conv_filters=conv_filters,
                kernel_sizes=kernel_sizes,
                pooling_type=str(pooling_type),
                dense_units=tuple(int(value) for value in
dense_units),

                batch_size=batch_size,
                epochs=epochs,
                learning_rate=learning_rate,
                input_shape=tuple(int(value) for value in
input_shape),

                use_global_pooling=use_global_pooling,
            )
        )

    return experiments

def create_default_callbacks(output_dir, model_name, monitor="val_macro_f1"):
    output_path = Path(output_dir)
    weights_dir = output_path / "weights"
    logs_dir = output_path / "logs"
    weights_dir.mkdir(parents=True, exist_ok=True)
    logs_dir.mkdir(parents=True, exist_ok=True)

    checkpoint_path = weights_dir / f"{model_name}.weights.h5"
    csv_log_path = logs_dir / f"{model_name}.csv"
    return [
        tf.keras.callbacks.ModelCheckpoint(
            filepath=checkpoint_path,
            monitor=monitor,
            mode="max",
            save_best_only=True,
            save_weights_only=True,
            verbose=1,

```

```

    ),
    tf.keras.callbacks.EarlyStopping(
        monitor=monitor,
        mode="max",
        patience=3,
        restore_best_weights=True,
        verbose=1,
    ),
    tf.keras.callbacks.CSVLogger(csv_log_path),
]

def train_experiment(
    config: CNNEExperimentConfig,
    train_split,
    val_split,
    *,
    num_classes,
    output_dir,
    preprocess_fn=None,
    normalize=True,
):
    model = build_cnn_classifier(
        input_shape=config.input_shape,
        num_classes=num_classes,
        conv_filters=config.conv_filters,
        kernel_sizes=config.kernel_sizes,
        pooling_type=config.pooling_type,
        dense_units=config.dense_units,
        use_global_pooling=config.use_global_pooling,
        conv_padding=config.conv_padding,
        activation=config.activation,
        local_connectivity=config.local_connectivity,
        name=config.name,
    )
    compile_cnn_classifier(
        model=model,
        num_classes=num_classes,

```

```

        learning_rate=config.learning_rate,
    )

    target_size = config.input_shape[:2]
    train_dataset = build_tf_dataset(
        train_split,
        target_size=target_size,
        batch_size=config.batch_size,
        shuffle=True,
        normalize=normalize,
        preprocess_fn=preprocess_fn,
    )
    val_dataset = build_tf_dataset(
        val_split,
        target_size=target_size,
        batch_size=config.batch_size,
        shuffle=False,
        normalize=normalize,
        preprocess_fn=preprocess_fn,
    )

    callbacks = create_default_callbacks(output_dir=output_dir,
model_name=config.name)
    history = model.fit(
        train_dataset,
        validation_data=val_dataset,
        epochs=config.epochs,
        callbacks=callbacks,
        verbose=1,
    )

    output_path = Path(output_dir)
    output_path.mkdir(parents=True, exist_ok=True)
    save_history(history, output_path / f"{config.name}_history.json")
    model.save(output_path / f"{config.name}_final.keras")
    model.save_weights(output_path / f"{config.name}.weights.h5")

```

```

metrics = evaluate_cnn_classifier(model, val_dataset, verbose=0)
summary = {
    "config": asdict(config),
    "validation_metrics": metrics,
    "parameter_count": int(model.count_params()),
    "best_epoch": int(np.argmax(history.history["val_macro_f1"]) + 1),
    "best_val_macro_f1": float(np.max(history.history["val_macro_f1"])),
}
(output_path / f"{config.name}_summary.json").write_text(
    json.dumps(summary, indent=2) + "\n",
    encoding="utf-8",
)
return model, history, summary

def summarize_experiments(summary_paths):
    summaries = []
    for summary_path in summary_paths:
        path = Path(summary_path)
        if not path.exists():
            continue
        summaries.append(json.loads(path.read_text(encoding="utf-8")))
    return summaries

def predict_scratch_split(
    scratch_model,
    split,
    *,
    target_size,
    batch_size=32,
    normalize=True,
    preprocess_fn=None,
):
    predictions = []
    for batch in iter_image_batches(
        image_paths=split.image_paths,
        target_size=target_size,
        batch_size=batch_size,

```

```

        normalize=normalize,
    ):
        if preprocess_fn is not None:
            batch = preprocess_fn(batch)
            predictions.append(np.asarray(scratch_model.predict(batch,
batch_size=batch_size)))

        if not predictions:
            return np.empty((0,), dtype=np.float32)
        return np.concatenate(predictions, axis=0)

def compare_keras_and_scratch(
    keras_model,
    split,
    *,
    target_size,
    batch_size=32,
    normalize=True,
    preprocess_fn=None,
):
    keras_dataset = build_tf_dataset(
        split,
        target_size=target_size,
        batch_size=batch_size,
        shuffle=False,
        normalize=normalize,
        preprocess_fn=preprocess_fn,
    )
    keras_predictions = keras_model.predict(keras_dataset, verbose=0)
    scratch_model = keras_to_scratch(keras_model)
    scratch_predictions = predict_scratch_split(
        scratch_model,
        split,
        target_size=target_size,
        batch_size=batch_size,
        normalize=normalize,
        preprocess_fn=preprocess_fn,

```

```

    )
    return {
        "keras_macro_f1": macro_f1_score(split.labels, keras_predictions,
num_classes=keras_predictions.shape[-1]),
        "scratch_macro_f1": macro_f1_score(split.labels, scratch_predictions,
num_classes=scratch_predictions.shape[-1]),
        "max_probability_gap": float(np.max(np.abs(keras_predictions -
scratch_predictions))),
        "mean_probability_gap": float(np.mean(np.abs(keras_predictions -
scratch_predictions))),
    }

def compare_shared_and_non_shared(
    shared_model,
    non_shared_model,
    split,
    *,
    target_size,
    batch_size=32,
    normalize=True,
):
    shared_predictions = shared_model.predict(
        build_tf_dataset(
            split,
            target_size=target_size,
            batch_size=batch_size,
            shuffle=False,
            normalize=normalize,
        ),
        verbose=0,
    )
    non_shared_predictions = non_shared_model.predict(
        build_tf_dataset(
            split,
            target_size=target_size,
            batch_size=batch_size,
            shuffle=False,

```

```

        normalize=normalize,
    ),
    verbose=0,
)
return {
    "shared_macro_f1": macro_f1_score(split.labels, shared_predictions,
num_classes=shared_predictions.shape[-1]),
    "non_shared_macro_f1": macro_f1_score(split.labels,
non_shared_predictions, num_classes=non_shared_predictions.shape[-1]),
    "shared_params": int(shared_model.count_params()),
    "non_shared_params": int(non_shared_model.count_params()),
}

```

src/cnn/visualization.py

```
src/cnn/visualization.py
```

```

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

from .nn.keras_layers import KerasLocallyConnected2D

def _import_tensorflow():
    try:
        import tensorflow as tf
    except ModuleNotFoundError as error:
        raise ModuleNotFoundError(
            "TensorFlow is required for CNN visualization utilities. "
            "Install dependencies with: pip install -r requirements.txt"
        ) from error
    return tf

def _prepare_image_batch(images):

```



```

batch = np.asarray(images, dtype=np.float32)
if batch.ndim == 3:
    batch = batch[np.newaxis, ...]
if batch.ndim != 4:
    raise ValueError(
        "Expected image batch with shape (batch, height, width, channels)
"
        f"or single image (height, width, channels), got {batch.shape}"
    )
return batch

def _call_model_once(model, batch):
    model(batch, training=False)
    return model

def _dense_logits(tf, layer, inputs):
    outputs = tf.tensordot(inputs, layer.kernel, axes=[[-1], [0]])
    if layer.use_bias:
        outputs = tf.nn.bias_add(outputs, layer.bias)
    return outputs

def _forward_to_class_scores(tf, model, start_index, inputs):
    outputs = inputs
    last_index = len(model.layers) - 1

    for index, layer in enumerate(model.layers[start_index:],
start=start_index):
        is_last_layer = index == last_index

        if is_last_layer and isinstance(layer, tf.keras.layers.Dense):
            activation = getattr(layer, "activation", None)
            if activation == tf.keras.activations.softmax:
                return _dense_logits(tf, layer, outputs)

        if is_last_layer and isinstance(layer, tf.keras.layers.Softmax):
            return outputs

```

```

        if is_last_layer and isinstance(layer, tf.keras.layers.Activation):
            if getattr(layer, "activation", None) ==
tf.keras.activations.softmax:
                return outputs

        outputs = layer(outputs)

    return outputs

def list_feature_layers(model):
    tf = _import_tensorflow()
    feature_layer_types = (tf.keras.layers.Conv2D, KerasLocallyConnected2D)
    return [layer.name for layer in model.layers if isinstance(layer,
feature_layer_types)]

def get_last_feature_layer(model):
    feature_layers = list_feature_layers(model)
    if not feature_layers:
        raise ValueError("Model does not contain a convolutional feature
layer.")
    return feature_layers[-1]

def extract_feature_maps(model, images, layer_names=None):
    tf = _import_tensorflow()
    batch = _prepare_image_batch(images)
    _call_model_once(model, batch)

    selected_layers = list(layer_names or list_feature_layers(model))
    if not selected_layers:
        raise ValueError("No feature layers selected for feature map
extraction.")

    feature_model = tf.keras.Model(
        inputs=model.inputs,
        outputs=[model.get_layer(layer_name).output for layer_name in
selected_layers],
    )

```

```

        outputs = feature_model(batch, training=False)
        if not isinstance(outputs, (list, tuple)):
            outputs = [outputs]
        return {name: np.asarray(output) for name, output in zip(selected_layers,
outputs)}

def plot_feature_maps(
    feature_maps,
    layer_name=None,
    image_index=0,
    max_maps=16,
    cols=4,
    normalize_each=True,
):
    if isinstance(feature_maps, dict):
        if layer_name is None:
            layer_name = next(iter(feature_maps))
        maps = np.asarray(feature_maps[layer_name])
    else:
        maps = np.asarray(feature_maps)
        layer_name = layer_name or "feature_maps"

    if maps.ndim != 4:
        raise ValueError(f"Expected feature maps with shape (batch, height,
width, channels), got {maps.shape}")
    if image_index >= maps.shape[0]:
        raise IndexError(f"image_index {image_index} out of range for batch
size {maps.shape[0]}")

    maps = maps[image_index]
    num_maps = min(max_maps, maps.shape[-1])
    rows = int(np.ceil(num_maps / cols))
    fig, axes = plt.subplots(rows, cols, figsize=(cols * 3, rows * 3))
    axes = np.atleast_1d(axes).reshape(rows, cols)

    for index in range(rows * cols):
        axis = axes.flat[index]

```

```

        axis.axis("off")
        if index >= num_maps:
            continue

        feature_map = maps[:, :, index]
        if normalize_each:
            feature_min = feature_map.min()
            feature_max = feature_map.max()
            if feature_max > feature_min:
                feature_map = (feature_map - feature_min) / (feature_max -
feature_min)
            axis.imshow(feature_map, cmap="viridis")
            axis.set_title(f"{layer_name}[{index}]")

    fig.tight_layout()
    return fig

def make_gradcam_heatmap(model, image, layer_name=None, class_index=None):
    tf = _import_tensorflow()
    batch = _prepare_image_batch(image)
    if batch.shape[0] != 1:
        raise ValueError("Grad-CAM expects a single image. Pass one image, not
a batch.")

    _call_model_once(model, batch)
    layer_name = layer_name or get_last_feature_layer(model)
    batch_tensor = tf.convert_to_tensor(batch, dtype=tf.float32)
    target_layer_index = next(
        (index for index, layer in enumerate(model.layers) if layer.name ==
layer_name),
        None,
    )
    if target_layer_index is None:
        raise ValueError(f"Layer not found in model: {layer_name}")

    conv_model = tf.keras.Model(
        inputs=model.inputs,

```

```

        outputs=model.get_layer(layer_name).output,
    )

    with tf.GradientTape() as tape:
        conv_output = conv_model(batch_tensor, training=False)
        tape.watch(conv_output)
        scores = _forward_to_class_scores(
            tf,
            model,
            target_layer_index + 1,
            conv_output,
        )
        if class_index is None:
            class_index = int(tf.argmax(scores[0]).numpy())
        score = scores[:, class_index]

    gradients = tape.gradient(score, conv_output)
    pooled_gradients = tf.reduce_mean(gradients, axis=(0, 1, 2))
    conv_output = conv_output[0]
    heatmap = tf.reduce_sum(conv_output * pooled_gradients[tf.newaxis,
tf.newaxis, :], axis=-1)
    heatmap = tf.nn.relu(heatmap)
    max_value = tf.reduce_max(heatmap)
    if float(max_value.numpy()) > 0.0:
        heatmap = heatmap / max_value

    return np.asarray(heatmap), class_index, layer_name

def resize_heatmap(heatmap, target_size):
    tf = _import_tensorflow()
    resized = tf.image.resize(
        np.asarray(heatmap, dtype=np.float32)[..., np.newaxis],
        target_size,
        method="bilinear",
    )
    return np.asarray(resized[..., 0])

```

```

def overlay_heatmap(image, heatmap, alpha=0.4, cmap="jet"):
    image_array = np.asarray(image, dtype=np.float32)
    if image_array.ndim != 3:
        raise ValueError(f"Expected image with shape (height, width,
channels), got {image_array.shape}")

    if image_array.max(initial=0.0) > 1.0:
        image_array = image_array / 255.0

    resized_heatmap = resize_heatmap(heatmap, image_array.shape[:2])
    colored_heatmap = plt.get_cmap(cmap)(resized_heatmap)[...,
:3].astype(np.float32)
    overlay = (1.0 - alpha) * image_array + alpha * colored_heatmap
    return np.clip(overlay, 0.0, 1.0)

def plot_gradcam(image, heatmap, alpha=0.4, cmap="jet", title=None):
    overlay = overlay_heatmap(image, heatmap, alpha=alpha, cmap=cmap)
    fig, axes = plt.subplots(1, 3, figsize=(12, 4))
    axes[0].imshow(np.asarray(image))
    axes[0].set_title("Image")
    axes[1].imshow(heatmap, cmap=cmap)
    axes[1].set_title("Grad-CAM")
    axes[2].imshow(overlay)
    axes[2].set_title(title or "Overlay")
    for axis in axes:
        axis.axis("off")
    fig.tight_layout()
    return fig

def save_figure(fig, output_path):
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(path, bbox_inches="tight")
    return path

```